CURSO-LIBRO DE ESTUDIO

Matemáticas, programación y teoría de códigos

De 3.º de ESO a la investigación sobre códigos lineales sobre GF(7)

Edición: Entrega 1 · generada el 13/10/2026

Estado del libro: 1 de 286 capítulos redactados. Este documento contiene **todo lo escrito hasta ahora**: el plan completo del libro (índice de 286 capítulos), el mapa de prerrequisitos, el sistema de estudio, los capítulos redactados, los apéndices (glosario, fórmulas, errores frecuentes, bibliografía), el registro de progreso y el código de verificación. Los capítulos pendientes se añadirán en las próximas entregas.

En esta versión **PDF** las soluciones aparecen **desplegadas** en recuadros titulados «▼ Solución A1», «▼ Solución A2»… Tápalas con un papel y no las mires hasta haber intentado cada ejercicio.

Redactado con ayuda de una IA (Claude, de Anthropic). Todos los cálculos se comprueban con los programas del apéndice de código.

Índice de este documento

Matemáticas, programación y teoría de códigos

- Por dónde empezar
- Descargar el libro
- Estructura del libro
- Estado actual
- Organización del repositorio
- Verificar los cálculos
- Sobre la autoría y el uso de IA

Cómo estudiar este libro

- 1. El ciclo de cada capítulo
- 2. Los tres niveles de ejercicios
- 3. Criterio de dominio
- 4. Repaso espaciado
- 5. Cómo pedir corrección a tu tutor (o a una IA)
- 6. Tipos de afirmaciones: aprende a distinguirlas
- 7. Convenios de notación del libro
- 8. El código del libro
- 9. Uso honesto de la IA

Índice general del libro

- Leyenda
- Documentos de acompañamiento
- PARTE I. FUNDAMENTOS MATEMÁTICOS DE ESO
- PARTE II. PROGRAMACIÓN Y PENSAMIENTO COMPUTACIONAL
- PARTE III. MATEMÁTICAS DISCRETAS Y ÁLGEBRA LINEAL
- PARTE IV. TEORÍA DE NÚMEROS Y POLINOMIOS
- PARTE V. TEORÍA DE CÓDIGOS DESDE CERO
- PARTE VI. CONSTRUIR UN BUSCADOR REAL DE CÓDIGOS
- PARTE VII. INVESTIGACIÓN MATEMÁTICA AUTÓNOMA
- APÉNDICES
- Tamaño del plan

Mapa de prerrequisitos y orden de estudio

- 1. Mapa general de módulos
- 2. Tabla de prerrequisitos por módulo
- 3. Cadenas críticas de conceptos

4. Comprobación de prerrequisitos que faltaban en el temario original
5. Orden de estudio recomendado
6. Qué mirar antes de empezar cada capítulo

Capítulo 1.1 — Los conjuntos de números: $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ y $\mathbb{R}$

Cómo está organizado este capítulo
1. Título y objetivos
2. Prerrequisitos
3. Explicación intuitiva
4. Definiciones formales
5. Esquema visual y mapa conceptual
6. Fórmulas importantes, símbolo a símbolo
7. Ejemplo básico resuelto paso a paso
8. Ejemplo intermedio resuelto paso a paso
9. Ejemplo avanzado aplicado a tu investigación
10. Errores frecuentes
11. Resumen para memorizar
12. Tabla de fórmulas y propiedades
13. Ejercicios sin resolver
14. Soluciones razonadas
15. Preguntas de comprensión
16. Prueba de dominio
17. Relación con el siguiente capítulo

Ampliaciones (opcionales en una primera lectura)
Anexo — Respuestas de comprensión, prueba de dominio y refuerzo
Referencias del capítulo
Registro personal del capítulo 1.1

Apéndice A — Glosario

Apéndice B — Hoja de fórmulas

Capítulo 1.1 — Conjuntos de números

Apéndice C — Errores frecuentes y cómo corregirlos

Capítulo 1.1 — Conjuntos de números

Apéndice D — Bibliografía comentada

1. Teoría de códigos (libros de texto)
2. Campos finitos, álgebra y matemáticas discretas
3. Tablas de códigos, bases de datos y artículos relacionados con GF(7)
4. Complejidad del cálculo de la distancia mínima
5. Programación y cálculo numérico

6. Software de álgebra que se usará para verificación independiente (Módulo 28)
7. Currículo escolar
Fuentes por capítulo
Registro de verificaciones de fuentes

Registro de progreso

1. Índice general (resumen)
2. Registro de redacción
3. Conceptos ya explicados en el libro
4. Decisiones de organización del libro
5. Registro de aprendizaje (lo rellenas tú)

Código del libro

Carpetas
Cómo ejecutar
Registro de ejecuciones
Limitaciones conocidas

Apéndice de código: programas del libro

`codigo/ejemplos/cap_1_1_rincon_del_ordenador.py`
`codigo/verificaciones/verificar_cap_1_1.py`

Matemáticas, programación y teoría de códigos

Un curso-libro progresivo: de 3.º de ESO a la investigación sobre códigos lineales sobre GF(7)

Este repositorio contiene un **libro de estudio completo**, escrito por entregas, para aprender las matemáticas y la programación necesarias para **comprender, desarrollar, verificar y mejorar** una investigación sobre códigos lineales con parámetros [n, k, d] sobre el campo finito GF(7): calcular distancias mínimas, generar matrices, buscar mejores códigos y comprobar con rigor si un resultado es realmente nuevo.

El punto de partida es **3.º de ESO** (Andalucía). Cada concepto se explica desde cero, con ejemplos resueltos paso a paso, ejercicios de tres niveles con soluciones razonadas, pruebas de dominio y programas en Python que **verifican** los cálculos.

Por dónde empezar

1. Lee **Cómo estudiar este libro** (10 minutos): explica el sistema de aprendizaje y el criterio para avanzar.
2. Echa un vistazo al **índice general** y al **mapa de prerrequisitos y orden de estudio**.
3. Empieza por el **Capítulo 1.1 — Los conjuntos de números: $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ y $\mathbb{R}$**.
4. Anota tu avance en **PROGRESO.md**.

Descargar el libro

En la carpeta `descargas/` tienes todo lo escrito hasta ahora en tres formatos:

Archivo	Para qué sirve
`libro-gf7-entrega-1.pdf`	Leer en el móvil, la tableta o imprimir. Las soluciones aparecen desplegadas en recuadros (tápalas antes de intentar cada ejercicio).
`libro-gf7-entrega-1.html`	Un solo archivo que se abre con cualquier navegador, sin conexión. Las soluciones están plegadas y se despliegan con un clic.
`libro-gf7-entrega-1-fuentes.zip`	Todo el repositorio: textos en Markdown, programas de Python, PDF y HTML.

Las descargas se regeneran con cada entrega mediante `python3 herramientas/construir_libro.py --entrega N --fecha DD/MM/AAAA` (necesita pandoc y Chromium).

Estructura del libro

Parte	Contenido	Módulos
I	Fundamentos matemáticos de ESO	1–5
II	Programación y pensamiento computacional (Python)	6–9
III	Matemáticas discretas y álgebra lineal (también sobre campos finitos)	10–13
IV	Teoría de números y polinomios; construcción de campos finitos	14–16
V	**Teoría de códigos desde cero** (prioridad central)	17–23
VI	Construir un buscador real de códigos	24–28
VII	Investigación matemática autónoma y proyecto final	29–32

Estado actual

Entrega 1 (12/10/2026): índice completo (286 capítulos), mapa de prerrequisitos, sistema de estudio, apéndices iniciales y **capítulo 1.1 completo**. El detalle está en PROGRESO.md.

Organización del repositorio

```
README.md                  ← este archivo
PROGRESO.md                ← registro de redacción y de aprendizaje
libro/
  00-indice-general.md
  00-mapa-de-prerrequisitos.md
  00-como-estudiar-este-libro.md
  parte-1-fundamentos-eso/
    modulo-01-numeros-y-operaciones/
      cap-1-1-conjuntos-de-numeros.md
apendices/
  glosario.md · hoja-de-formulas.md · errores-frecuentes.md · bibliografia.md
codigo/
  README.md
  ejemplos/                ← pequeños programas que aparecen en los capítulos
  verificaciones/          ← programas que recalculan los resultados de cada capítulo
descargas/                 ← el libro en PDF, HTML y ZIP
herramientas/
  construir_libro.py       ← genera las descargas a partir de los textos
```

Verificar los cálculos

Solo hace falta Python 3 (sin bibliotecas externas):

```
python3 codigo/verificaciones/verificar_cap_1_1.py
```

Sobre la autoría y el uso de IA

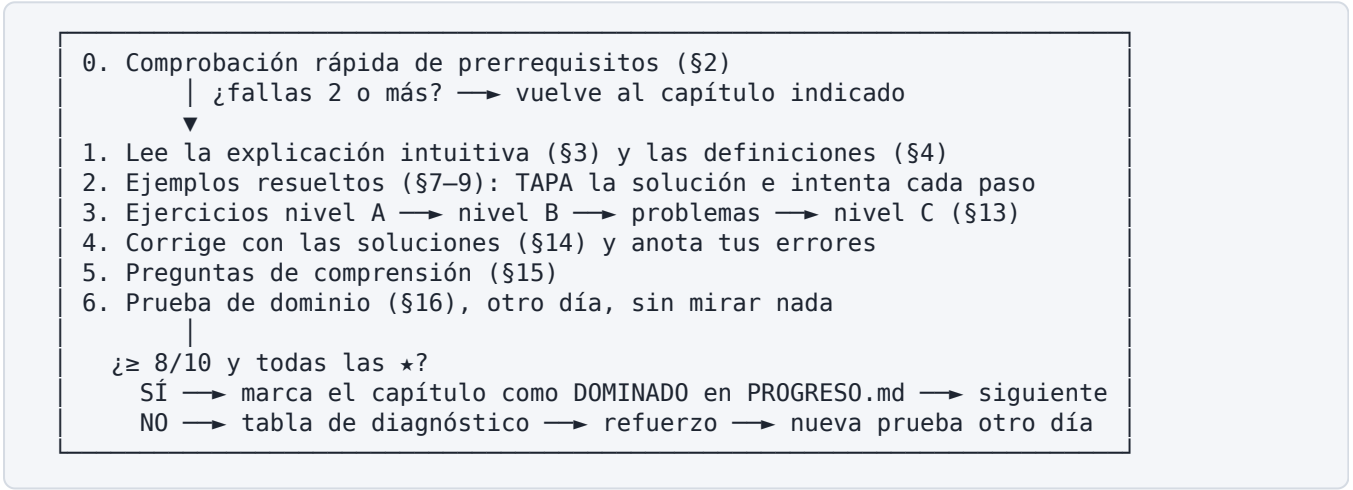
El texto de este libro se ha redactado con ayuda de una IA (Claude, de Anthropic), siguiendo un plan pedagógico definido por el autor del repositorio. Todos los cálculos numéricos se comprueban con programas incluidos en `codigo/verificaciones/`, y cada referencia bibliográfica indica si se ha podido verificar. Si encuentras un error, anótalo: detectar errores (también los de una IA) es una de las habilidades que este libro quiere enseñar.

Cómo estudiar este libro

> **Idea central:** completar un capítulo no es lo mismo que dominarlo. Este libro está diseñado para que avances **solo cuando has demostrado que entiendes**, no cuando has terminado de leer.

---

1. El ciclo de cada capítulo



Consejos prácticos:

- Estudia con **papel y lápiz**. Las matemáticas se aprenden haciendo, no leyendo.
- Las soluciones están **plegadas** (en GitHub y en la versión HTML aparecen como recuadros cerrados con un triángulo, por ejemplo «Solución A1»; en el PDF aparecen abiertos, y conviene taparlos con un papel). Ábrelas solo después de intentarlo de verdad. Equivocarse y descubrir por qué es la parte del estudio que más enseña.
- Lleva un **cuaderno de errores**: para cada error, escribe (1) qué hiciste, (2) por qué estaba mal, (3) qué concepto fallaba, (4) un ejercicio parecido resuelto bien.
- Haz la prueba de dominio **otro día**, no justo después de estudiar: así compruebas lo que recuerdas de verdad.

---

2. Los tres niveles de ejercicios

Nivel	Qué exige	Ejemplo (capítulo 1.1)
A	Comprender y aplicar directamente una definición o un procedimiento.	Clasificar $-12/4$ en $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$.
B	Combinar varios conceptos y resolver problemas que no has visto antes.	Decidir sin dividir si 21/56 da decimal exacto (hay que simplificar **y** aplicar el criterio).
C	Investigar: recoger datos, formular una conjetura, comprobarla, intentar refutarla y justificar las conclusiones.	Descubrir la relación entre el periodo de $1/p$ y $p - 1$.

Además, cada capítulo incluye **2 problemas de aplicación práctica** y **1 problema combinado** con capítulos anteriores. Desde el capítulo 1.2, cada capítulo tendrá una sección **«Recupera»** con problemas nuevos sobre lo estudiado semanas atrás.

---

3. Criterio de dominio

Un capítulo está **dominado** cuando:

1. Has obtenido **8 puntos o más sobre 10** en la prueba de dominio.
2. Has respondido **correctamente todas las preguntas marcadas con ★** (conceptos críticos para lo que viene después).
3. Puedes **explicar con tus palabras** las respuestas a las preguntas de comprensión.

Si no se cumplen, **no avances** a los capítulos que dependen de él (consulta el mapa de prerrequisitos). Usa la **tabla de diagnóstico** del capítulo: te dice qué concepto falla, qué sección repasar y qué ejercicio de refuerzo hacer.

Prueba de salto

Si un capítulo trata algo que ya has estudiado en clase, puedes intentar **directamente** su prueba de dominio. Si sacas **9 o más y todas las ★**, márcalo como dominado y lee solo su **ejemplo avanzado** (§9, la conexión con la investigación) y su sección de errores frecuentes. Esto permite avanzar rápido por la Parte I sin saltarse lagunas.

---

4. Repaso espaciado

Lo que no se repasa se olvida. Calendario recomendado para cada capítulo dominado:

Cuándo	Qué hacer
Al día siguiente	Rehacer 2 ejercicios que fallaste.
A la semana	Resumen para memorizar (§11) de memoria + 1 problema «Recupera» del capítulo siguiente.
Al mes	Repaso acumulativo (R*n*) del módulo.
Al final de cada Parte	Examen acumulativo (EP-*X*).

---

5. Cómo pedir corrección a tu tutor (o a una IA)

Las soluciones del libro muestran **una** forma correcta, pero no pueden ver **tu** razonamiento. Cuando quieras que te corrijan, envía:

```
Capítulo: 1.1      Ejercicio: A4 d)
Mi respuesta: 0,3(12) = 312/990 = 52/165
Mi razonamiento: multipliqué por 1000 y resté…
Duda concreta: no sé por qué no coincide al dividir.
```

Quien te corrija debe: (1) señalar **el concepto exacto** que falla (por ejemplo, «olvidaste restar el anteperiodo»), (2) explicarlo, y (3) darte **otro ejercicio parecido** para comprobar que lo has entendido. Si usas una IA para corregir, **comprueba sus cálculos tú mismo**: las IA también se equivocan (lo estudiarás a fondo en el capítulo 28.7).

---

6. Tipos de afirmaciones: aprende a distinguirlas

En todo el libro, cada afirmación lleva una etiqueta. Esta distinción es la base del trabajo científico:

Etiqueta	Qué significa	¿Se puede usar como verdad?
[DEFINICIÓN]	Acuerdo sobre el significado de una palabra o símbolo.	Es un acuerdo; comprueba que el autor que lees usa la misma.
[PROPIEDAD — demostrada] / **[TEOREMA]**	Demostrada en el libro.	Sí.
[PROPIEDAD — demostración pendiente: …]	Verdadera y conocida; se demostrará más adelante.	Sí, pero sabiendo que aún no has visto por qué.
[HECHO ACEPTADO]	Demostrado por matemáticos; demostración fuera del alcance del libro.	Sí, citando la fuente.
[CONJETURA]	Parece verdadera, pero no está demostrada.	**No.** Hay que demostrarla o refutarla.
[VERIFICADO POR PROGRAMA]	Comprobado en un número finito de casos.	Es evidencia, **no** demostración (salvo que el programa haya revisado **todos** los casos posibles de un problema finito, y el programa sea correcto).
[EJEMPLO]	Un caso concreto.	Ilustra; no demuestra afirmaciones generales.
[AVANCE]	Anticipo de un capítulo futuro.	No se exige dominarlo todavía.

---

7. Convenios de notación del libro

- **0 ∈ ℕ**: en este libro los naturales empiezan en 0.
- **Coma decimal** en el texto (0,75); **punto** en los programas (`0.75`).
- Listas de decimales separadas por **punto y coma**: 0,5; 1,2.
- **Periodo entre paréntesis**: 0,41(6) = 0,41666…
- Signo menos tipográfico «−» en el texto; guion «-» en el código.
- Las fórmulas se escriben con símbolos Unicode (ℕ, ∈, ⊂, ≤, √, ⟺) en lugar de LaTeX, para que se lean bien en cualquier visor.
- Los capítulos se citan como «capítulo 13.4» (Módulo 13, capítulo 4).

---

8. El código del libro

- Todo el código usa **solo la biblioteca estándar de Python 3**: no hay que instalar nada más.
- Cada capítulo con cálculos tiene un programa de **verificación** en `codigo/verificaciones/` que recalcula sus resultados por un procedimiento independiente.
- Cuando el libro muestra la salida de un programa, indica **con qué versión de Python se ejecutó y cuándo**. Si una salida no se ha ejecutado realmente, se dice explícitamente.
- Hasta el Módulo 6 no necesitas entender el código: basta con ejecutarlo (`python3 nombre_del_archivo.py`) y ver que todo sale «OK».

---

9. Uso honesto de la IA

Este libro se ha redactado con ayuda de una IA (Claude, de Anthropic), y está pensado para que tú aprendas a **no depender** de ella:

- Usa la IA como **tutor**: para que te explique, te proponga ejercicios o revise tu razonamiento. No para que haga los ejercicios por ti.
- **Verifica** todo resultado importante por un segundo camino (a mano, con otro programa o con una fuente fiable).
- En tu investigación, **registra** qué partes ha hecho una IA y cuáles tú (capítulo 31.9).

Índice general del libro

Matemáticas, programación y teoría de códigos: de 3.º de ESO a la investigación sobre códigos lineales sobre GF(7)

> Este índice es el **plan completo** del libro. Los capítulos marcados con ✅ ya están redactados (con enlace); los marcados con ⏳ están pendientes y se irán escribiendo por entregas, en el orden de estudio recomendado (ver mapa de prerrequisitos). El estado actualizado está siempre en `PROGRESO.md`.

Leyenda

Marca	Significado
★	**Núcleo**: imprescindible para entender y desarrollar tu investigación.
◇	**Ampliación**: recomendable, pero no bloquea el avance hacia el proyecto.
[+]	**Añadido** al temario original porque es un prerrequisito que faltaba (justificación en el mapa de prerrequisitos).
✅ / ⏳	Redactado / pendiente de redactar.
R*n*	Repaso acumulativo.
EP-*X*	Examen acumulativo y prueba de dominio de la Parte *X*.

Numeración. «Capítulo 13.4» significa «Módulo 13, capítulo 4». Cada capítulo sigue el formato de 17 secciones descrito en Cómo estudiar este libro. Los capítulos de laboratorio y de proyecto adaptan ese formato al trabajo con código.

Documentos de acompañamiento

- Cómo estudiar este libro — sistema de aprendizaje, niveles A/B/C, pruebas de dominio, cómo pedir corrección.
- Mapa de prerrequisitos y orden de estudio
- Registro de progreso
- Apéndices: Glosario · Hoja de fórmulas · Errores frecuentes · Bibliografía
- Código del libro (programas de verificación y ejemplos)

- **Descargas:** el libro en PDF, HTML y ZIP en la carpeta `descargas/` del repositorio (ver el README).

PARTE I. FUNDAMENTOS MATEMÁTICOS DE ESO

Objetivo: dominar las herramientas básicas sin lagunas.

Módulo 1. Números y operaciones

- 1.1 ★ ✅ Los conjuntos de números: $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ y $\mathbb{R}$
- 1.2 ★ ⏳ Números enteros: signos, valor absoluto, orden y jerarquía de operaciones
- 1.3 ★ ⏳ Fracciones: equivalencia, simplificación y operaciones
- 1.4 ★ ⏳ Números decimales: operaciones, redondeo, truncamiento y [+] partes enteras $\lfloor x \rfloor$ y $\lceil x \rceil$
- 1.5 ★ ⏳ Potencias de exponente natural y entero; propiedades
- 1.6 ◇ ⏳ Raíces y radicales; potencias de exponente fraccionario
- 1.7 ★ ⏳ Notación científica y órdenes de magnitud (¿cuánto es 7^{30}?)
- 1.8 ★ ⏳ Errores, aproximaciones y estimaciones (error absoluto y relativo; estimar antes de calcular)
- R1 ⏳ Repaso acumulativo del Módulo 1

Módulo 2. Divisibilidad y aritmética

- 2.1 ★ ⏳ Múltiplos, divisores y criterios de divisibilidad
- 2.2 ★ ⏳ Números primos y criba de Eratóstenes
- 2.3 ★ ⏳ Factorización en primos y teorema fundamental de la aritmética
- 2.4 ★ ⏳ Máximo común divisor y mínimo común múltiplo
- 2.5 ★ ⏳ División euclídea: cociente y resto (también con números negativos)
- 2.6 ★ ⏳ Algoritmo de Euclides
- 2.7 ★ ⏳ [+] Identidad de Bézout y algoritmo de Euclides extendido
- 2.8 ★ ⏳ Introducción a la aritmética modular: relojes, calendarios y restos
- R2 ⏳ Repaso acumulativo de los Módulos 1–2

Módulo 3. Álgebra

- 3.1 ★ ⏳ Variables y expresiones; traducir del lenguaje común al algebraico
- 3.2 ★ ⏳ Monomios y polinomios: vocabulario, grado y valor numérico
- 3.3 ★ ⏳ Suma, resta y producto de polinomios
- 3.4 ★ ⏳ Identidades notables

- 3.5 ★ ⏳ División de polinomios y regla de Ruffini
- 3.6 ★ ⏳ Factorización de polinomios y raíces
- 3.7 ★ ⏳ Ecuaciones de primer grado
- 3.8 ★ ⏳ Ecuaciones de segundo grado
- 3.9 ★ ⏳ Sistemas de ecuaciones lineales (2×2 y 3×3): sustitución, igualación y reducción
- 3.10 ◇ ⏳ Inecuaciones y desigualdades
- 3.11 ★ ⏳ Resolución de problemas: del enunciado a la ecuación
- R3 ⏳ Repaso acumulativo de los Módulos 1–3

Módulo 4. Funciones, patrones y razonamiento

- 4.1 ★ ⏳ Coordenadas cartesianas y gráficas
- 4.2 ★ ⏳ Relaciones y funciones: dominio, recorrido y tablas de valores
- 4.3 ★ ⏳ Funciones lineales y afines
- 4.4 ★ ⏳ Funciones cuadráticas y exponenciales: cómo crecen
- 4.5 ★ ⏳ [+] Logaritmos
- 4.6 ★ ⏳ Sucesiones y patrones: aritméticas, geométricas y recurrentes
- 4.7 ★ ⏳ [+] Notación de sumatorio (Σ) y productorio (Π)
- 4.8 ★ ⏳ Lógica básica: proposiciones, «y», «o», «no», «si… entonces»
- 4.9 ★ ⏳ Cuantificadores intuitivos: «para todo» y «existe»
- 4.10 ★ ⏳ Cómo construir y analizar un argumento matemático
- R4 ⏳ Repaso acumulativo de los Módulos 1–4

Módulo 5. Combinatoria y probabilidad

- 5.1 ★ ⏳ Principios de la suma y del producto
- 5.2 ★ ⏳ Variaciones y permutaciones (con y sin repetición); factorial
- 5.3 ★ ⏳ Combinaciones, números combinatorios y triángulo de Pascal
- 5.4 ★ ⏳ [+] Binomio de Newton y su significado combinatorio
- 5.5 ★ ⏳ Recuento sistemático: árboles, tablas y listados ordenados
- 5.6 ★ ⏳ Probabilidad básica y regla de Laplace
- 5.7 ◇ ⏳ Espacios muestrales, sucesos y probabilidad condicionada
- 5.8 ◇ ⏳ [+] Valor esperado (esperanza)
- 5.9 ★ ⏳ Explosión combinatoria: por qué algunos problemas son difíciles para un ordenador
- EP-I ⏳ Examen acumulativo y prueba de dominio de la Parte I

PARTE II. PROGRAMACIÓN Y PENSAMIENTO COMPUTACIONAL

Objetivo: desarrollar las herramientas necesarias para experimentar con códigos.

Módulo 6. Python desde cero

- 6.1 ★ ⏳ [+] Instalar Python, usar el intérprete y escribir tu primer programa
- 6.2 ★ ⏳ Variables y tipos de datos (`int`, `float`, `bool`, `str`)
- 6.3 ★ ⏳ Operadores aritméticos (incluidos `//` y `%`), de comparación y lógicos
- 6.4 ★ ⏳ Condicionales
- 6.5 ★ ⏳ Bucles `for` y `while`
- 6.6 ★ ⏳ Funciones: parámetros, valor de retorno y ámbito
- 6.7 ★ ⏳ Listas y tuplas
- 6.8 ★ ⏳ Conjuntos y diccionarios
- 6.9 ★ ⏳ Cadenas de texto y salida con formato
- 6.10 ★ ⏳ Entrada y salida; lectura y escritura de archivos (texto, CSV y JSON)
- 6.11 ★ ⏳ [+] Errores y excepciones: cómo leer un mensaje de error
- 6.12 ★ ⏳ [+] Módulos y organización de un programa en varios archivos

Módulo 7. Programación matemática

- 7.1 ★ ⏳ Aritmética modular en Python
- 7.2 ★ ⏳ Euclides y Euclides extendido en Python; inversos modulares
- 7.3 ★ ⏳ Funciones de comprobación (predicados)
- 7.4 ★ ⏳ Generar combinaciones, permutaciones y productos (a mano y con `itertools`)
- 7.5 ★ ⏳ Búsqueda exhaustiva
- 7.6 ★ ⏳ Recursividad
- 7.7 ★ ⏳ Aserciones e invariantes
- 7.8 ★ ⏳ Pruebas unitarias con `unittest`
- 7.9 ★ ⏳ Números aleatorios, semillas y reproducibilidad
- 7.10 ★ ⏳ [+] Vectores y matrices como listas de listas

Módulo 8. Algoritmos y complejidad

- 8.1 ★ ⏳ Qué es un algoritmo: corrección y terminación
- 8.2 ★ ⏳ Búsqueda exhaustiva y búsqueda inteligente (poda, simetrías)
- 8.3 ★ ⏳ Complejidad temporal y espacial: contar operaciones
- 8.4 ★ ⏳ Notación O grande
- 8.5 ★ ⏳ Cuellos de botella: medir con `time`, `timeit` y `cProfile`

- 8.6 ★ ⏳ Optimizar sin cambiar el resultado matemático (precálculo, tablas, evitar trabajo repetido)
- 8.7 ◇ ⏳ [+] Representación binaria: bits, bytes, `float` y enteros como vectores
- 8.8 ◇ ⏳ Introducción a C para optimizar (solo después de dominar Python)

Módulo 9. Investigación computacional

- 9.1 ★ ⏳ Formular hipótesis comprobables
- 9.2 ★ ⏳ Diseñar experimentos
- 9.3 ★ ⏳ Separar generación, cálculo y verificación
- 9.4 ★ ⏳ Pruebas independientes y «oráculos»
- 9.5 ★ ⏳ Registro: semillas, parámetros, versiones y resultados
- 9.6 ★ ⏳ Comparar algoritmos con justicia
- 9.7 ★ ⏳ Errores de implementación típicos y cómo cazarlos
- 9.8 ★ ⏳ Documentar y reproducir un resultado
- EP-II ⏳ Examen acumulativo de la Parte II (incluye un mini-proyecto)

PARTE III. MATEMÁTICAS DISCRETAS Y ÁLGEBRA LINEAL

Objetivo: adquirir el lenguaje matemático que aparece en la teoría de códigos.

Módulo 10. Lógica y demostraciones

- 10.1 ★ ⏳ Proposiciones, conectores y tablas de verdad
- 10.2 ★ ⏳ Implicación, recíproca, contrarrecíproca y equivalencia
- 10.3 ★ ⏳ Condiciones necesarias y suficientes
- 10.4 ★ ⏳ Cuantificadores y cómo negarlos
- 10.5 ★ ⏳ Demostración directa
- 10.6 ★ ⏳ Demostración por contraposición
- 10.7 ★ ⏳ Reducción al absurdo (irracionalidad de √n, infinitud de los primos)
- 10.8 ★ ⏳ Inducción matemática
- 10.9 ★ ⏳ Contraejemplos
- 10.10 ★ ⏳ Evidencia computacional frente a demostración
- 10.11 ★ ⏳ [+] Cómo redactar una demostración

Módulo 11. Conjuntos y relaciones

- 11.1 ★ ⏳ Conjuntos, pertenencia y subconjuntos

- 11.2 ★ ⏳ Operaciones con conjuntos
- 11.3 ★ ⏳ Producto cartesiano; palabras de longitud n como elementos de A^n
- 11.4 ★ ⏳ Relaciones y funciones: inyectivas, sobreyectivas y biyectivas
- 11.5 ★ ⏳ Relaciones de equivalencia y clases de equivalencia (las clases de restos)
- 11.6 ★ ⏳ [+] Cardinal de conjuntos finitos y principio del palomar
- 11.7 ★ ⏳ Estructuras finitas: operaciones internas y tablas de operación
- 11.8 ★ ⏳ [+] Permutaciones como funciones; cómo actúan sobre palabras

Módulo 12. Vectores y matrices

- 12.1 ★ ⏳ Vectores y operaciones
- 12.2 ★ ⏳ Combinaciones lineales
- 12.3 ★ ⏳ Matrices: dimensiones, notación y tipos
- 12.4 ★ ⏳ Suma de matrices y producto por un número
- 12.5 ★ ⏳ Producto de matrices, matriz identidad y traspuesta
- 12.6 ★ ⏳ Sistemas lineales en forma matricial
- 12.7 ★ ⏳ Eliminación de Gauss y forma escalonada (reducida)
- 12.8 ★ ⏳ Rango de una matriz
- 12.9 ★ ⏳ Dependencia e independencia lineal
- 12.10 ★ ⏳ Espacio generado y subespacios
- 12.11 ★ ⏳ Bases y dimensión
- 12.12 ★ ⏳ Aplicaciones lineales: núcleo, imagen y teorema del rango
- 12.13 ◇ ⏳ [+] Matriz inversa y determinantes
- 12.14 ★ ⏳ [+] Producto escalar y ortogonalidad

Módulo 13. Álgebra lineal sobre campos finitos

- 13.1 ★ ⏳ Qué es un campo: los axiomas, uno a uno
- 13.2 ★ ⏳ Campos y no-campos: $\mathbb{Q}$, $\mathbb{Z}$, el «reloj de 6» y el «reloj de 7»
- 13.3 ★ ⏳ Aritmética módulo un primo: tablas de GF(2), GF(3), GF(5) y GF(7)
- 13.4 ★ ⏳ Inversos multiplicativos módulo p (por tabla y por Euclides extendido)
- 13.5 ★ ⏳ Vectores sobre GF(q): operaciones y recuento (q^n vectores)
- 13.6 ★ ⏳ Matrices sobre GF(p)
- 13.7 ★ ⏳ Eliminación de Gauss módulo p
- 13.8 ★ ⏳ Rango, independencia y dimensión sobre GF(7)
- 13.9 ★ ⏳ Subespacios de $GF(q)^n$: un subespacio de dimensión k tiene q^k vectores
- 13.10 ★ ⏳ Ortogonalidad sobre campos finitos: vectores ortogonales a sí mismos
- 13.11 ★ ⏳ $\mathbb{R}$ frente a GF(7): diferencias y trampas

- 13.12 ★ ⏳ Laboratorio: biblioteca de álgebra lineal sobre GF(p) con pruebas
- EP-III ⏳ Examen acumulativo de la Parte III

PARTE IV. TEORÍA DE NÚMEROS Y POLINOMIOS

Objetivo: preparar los conceptos necesarios para comprender campos finitos y familias de códigos.

Módulo 14. Aritmética modular avanzada

- 14.1 ★ ⏳ Congruencias y sus propiedades
- 14.2 ★ ⏳ Inversos modulares: cuándo existen y cómo calcularlos
- 14.3 ★ ⏳ Ecuaciones de congruencia lineales
- 14.4 ◇ ⏳ Sistemas de congruencias y teorema chino del resto
- 14.5 ★ ⏳ Pequeño teorema de Fermat y teorema de Euler
- 14.6 ★ ⏳ [+] Introducción a los grupos: orden de un elemento y raíces primitivas
- 14.7 ★ ⏳ Exponenciación rápida
- 14.8 ◇ ⏳ Aplicaciones informáticas: sumas de comprobación, ISBN, hashing

Módulo 15. Polinomios

- 15.1 ★ ⏳ Polinomios con coeficientes en un campo
- 15.2 ★ ⏳ Operaciones y grado
- 15.3 ★ ⏳ División euclídea de polinomios
- 15.4 ★ ⏳ Raíces y teorema del factor; un polinomio de grado m tiene como mucho m raíces
- 15.5 ★ ⏳ Máximo común divisor de polinomios y algoritmo de Euclides
- 15.6 ★ ⏳ Factorización y polinomios irreducibles
- 15.7 ★ ⏳ Aritmética módulo un polinomio
- 15.8 ★ ⏳ [+] Evaluación (Horner) e interpolación de Lagrange
- 15.9 ★ ⏳ Laboratorio: polinomios sobre GF(p) en Python

Módulo 16. Construcción de campos finitos

- 16.1 ★ ⏳ [+] Anillos y campos: el marco general
- 16.2 ★ ⏳ Característica; por qué un campo finito tiene p^m elementos
- 16.3 ★ ⏳ GF(p) = $\mathbb{Z}/p\mathbb{Z}$: GF(7) en detalle
- 16.4 ★ ⏳ Campos de extensión GF(p)[x]/(f(x)) y polinomios irreducibles
- 16.5 ◇ ⏳ GF(4), GF(8) y GF(9) a mano; por qué el «reloj de 8» no es GF(8)

- 16.6 ◇ ⏳ Elementos primitivos y tablas de logaritmos discretos
- 16.7 ◇ ⏳ [+] Polinomios mínimos y clases ciclotómicas
- 16.8 ★ ⏳ Qué necesita tu proyecto actual y qué es ampliación posterior
- EP-IV ⏳ Examen acumulativo de la Parte IV

PARTE V. TEORÍA DE CÓDIGOS DESDE CERO

Prioridad central del libro.

Módulo 17. Qué son los códigos correctores de errores

- 17.1 ★ ⏳ Mensajes, símbolos, alfabetos y palabras
- 17.2 ★ ⏳ Codificación y decodificación; el canal con ruido
- 17.3 ★ ⏳ Errores de transmisión y el canal simétrico q-ario
- 17.4 ★ ⏳ Distancia de Hamming y sus propiedades
- 17.5 ★ ⏳ Distancia mínima de un código; parámetros (n, M, d)
- 17.6 ★ ⏳ Detección y corrección de errores; bolas de Hamming
- 17.7 ★ ⏳ Decodificación por el vecino más próximo
- 17.8 ◇ ⏳ Aplicaciones reales: códigos QR, CD/DVD, ISBN, comunicaciones espaciales, discos RAID

Módulo 18. Códigos lineales

- 18.1 ★ ⏳ Definición: un código lineal es un subespacio de $GF(q)^n$
- 18.2 ★ ⏳ La notación [n, k, d]_q: longitud, dimensión, distancia mínima y campo base
- 18.3 ★ ⏳ Número de palabras: q^k
- 18.4 ★ ⏳ Matrices generadoras
- 18.5 ★ ⏳ Generar todas las palabras de un código a partir de G
- 18.6 ★ ⏳ Codificar: mensaje × G
- 18.7 ★ ⏳ Tasa R = k/n y distancia relativa d/n
- 18.8 ★ ⏳ Taller: códigos diminutos sobre GF(2), GF(3) y GF(7)

Módulo 19. Distancia mínima y peso

- 19.1 ★ ⏳ Peso de Hamming
- 19.2 ★ ⏳ d(x, y) = wt(x − y)
- 19.3 ★ ⏳ Teorema: en un código lineal, distancia mínima = peso mínimo
- 19.4 ★ ⏳ Calcular d por enumeración, y su coste

- 19.5 ★ ⏳ [+] Ahorro por múltiplos escalares: basta mirar (q^k − 1)/(q − 1) palabras
- 19.6 ★ ⏳ [+] d mediante la matriz de control: columnas linealmente dependientes
- 19.7 ★ ⏳ Distribución de pesos y enumerador de pesos
- 19.8 ★ ⏳ Por qué calcular d es un problema difícil
- 19.9 ★ ⏳ Casos de prueba manuales y validación con programas independientes

Módulo 20. Matrices generadoras y matrices de control

- 20.1 ★ ⏳ Matriz generadora: dimensiones y rango
- 20.2 ★ ⏳ Forma sistemática [I_k | A]
- 20.3 ★ ⏳ Matriz de control H
- 20.4 ★ ⏳ Código dual y relación $G \cdot H^T = 0$
- 20.5 ★ ⏳ Síndromes
- 20.6 ★ ⏳ Detección y corrección con síndromes; líderes de clase
- 20.7 ★ ⏳ Comprobaciones algebraicas de una pareja (G, H)

Módulo 21. Límites y parámetros

- 21.1 ★ ⏳ Códigos equivalentes (permutaciones y matrices monomiales)
- 21.2 ★ ⏳ Cota de Singleton y códigos MDS
- 21.3 ★ ⏳ Cota de Hamming y códigos perfectos
- 21.4 ★ ⏳ [+] Cota de Griesmer
- 21.5 ★ ⏳ Cota de Gilbert–Varshamov, paso a paso
- 21.6 ◇ ⏳ [+] Cota de Plotkin
- 21.7 ◇ ⏳ Cotas por programación lineal (idea)
- 21.8 ★ ⏳ Tablas de mejores códigos conocidos: cómo leerlas
- 21.9 ★ ⏳ Mejorar un código: código válido, óptimo conocido y récord demostrado
- 21.10 ★ ⏳ Taller: comparar parámetros

Módulo 22. Familias de códigos

- 22.1 ★ ⏳ Códigos de repetición
- 22.2 ★ ⏳ Códigos de paridad (suma cero) sobre GF(q)
- 22.3 ★ ⏳ Códigos de Hamming q-arios
- 22.4 ★ ⏳ [+] Códigos símplex
- 22.5 ★ ⏳ Códigos cíclicos
- 22.6 ◇ ⏳ Códigos BCH
- 22.7 ★ ⏳ Códigos Reed–Solomon

- 22.8 ★ ⏳ Códigos construidos mediante matrices (circulantes, Vandermonde)
- 22.9 ★ ⏳ Códigos constacíclicos, cuasicíclicos y cuasitwisted
- 22.10 ★ ⏳ Mapa: qué familias necesita tu investigación y cuáles son extensiones

Módulo 23. Códigos sobre GF(7)

- 23.1 ★ ⏳ Aritmética con 0, 1, …, 6: suma, resta y producto
- 23.2 ★ ⏳ Inversos y división en GF(7)
- 23.3 ★ ⏳ Vectores y matrices sobre GF(7)
- 23.4 ★ ⏳ Construir códigos lineales sobre GF(7)
- 23.5 ★ ⏳ Rango sobre GF(7), a mano y por programa
- 23.6 ★ ⏳ Generar las palabras de un código sobre GF(7)
- 23.7 ★ ⏳ Calcular pesos y distancias sobre GF(7)
- 23.8 ★ ⏳ Ejemplos completos de parámetros [n, k, d]_7
- 23.9 ★ ⏳ Reed–Solomon y Hamming sobre GF(7)
- 23.10 ★ ⏳ Ejercicios de dificultad creciente
- EP-V ⏳ Examen acumulativo de la Parte V

PARTE VI. CONSTRUIR UN BUSCADOR REAL DE CÓDIGOS

Objetivo: aplicar las matemáticas a un programa propio.

Módulo 24. Arquitectura de un buscador

- 24.1 ★ ⏳ Separar generador de matrices, cálculo de parámetros y verificador
- 24.2 ★ ⏳ Representación eficiente de los elementos de GF(7)
- 24.3 ★ ⏳ Generación de matrices candidatas
- 24.4 ★ ⏳ Duplicados y equivalencias: cuándo y cómo eliminarlos
- 24.5 ★ ⏳ Recuento de palabras
- 24.6 ★ ⏳ Cálculo del peso y de la distancia mínima
- 24.7 ★ ⏳ Registro de resultados

Módulo 25. Algoritmos exactos

- 25.1 ★ ⏳ Búsqueda exhaustiva
- 25.2 ★ ⏳ Enumeración de combinaciones lineales; [+] códigos de Gray q-arios
- 25.3 ★ ⏳ Casos pequeños verificables a mano
- 25.4 ★ ⏳ Optimización de operaciones

- 25.5 ★ ⏳ Comprobaciones de rango
- 25.6 ◇ ⏳ [+] Idea del algoritmo de Brouwer–Zimmermann
- 25.7 ★ ⏳ Comparación con implementaciones independientes
- 25.8 ★ ⏳ Cuándo un algoritmo certifica un resultado y cuándo solo aporta evidencia parcial

Módulo 26. Heurísticas de búsqueda

- 26.1 ★ ⏳ Búsqueda aleatoria
- 26.2 ★ ⏳ Funciones objetivo
- 26.3 ★ ⏳ Hill climbing (ascenso de colinas)
- 26.4 ★ ⏳ Mutaciones de candidatos
- 26.5 ★ ⏳ Reinicios aleatorios
- 26.6 ★ ⏳ Diversidad de candidatos: exploración y explotación
- 26.7 ◇ ⏳ Cómo escapar de óptimos locales (recocido simulado, búsqueda tabú: idea)
- 26.8 ★ ⏳ Encontrar buenos candidatos no es demostrar que no existen mejores

Módulo 27. Extensión y construcción de códigos

- 27.1 ★ ⏳ Idea matemática de extender un código
- 27.2 ★ ⏳ Pinchar, acortar y extender: cómo cambian n, k y d
- 27.3 ★ ⏳ Condiciones que deben comprobarse
- 27.4 ◇ ⏳ [+] Construcciones (u | u + v), yuxtaposición y Construcción X
- 27.5 ★ ⏳ Ejemplos pequeños
- 27.6 ★ ⏳ Búsqueda de mejoras de parámetros
- 27.7 ★ ⏳ Verificación de resultados frente a métodos conocidos

Módulo 28. Validación científica

- 28.1 ★ ⏳ Revisión de resultados publicados
- 28.2 ★ ⏳ Verificación de parámetros y condiciones
- 28.3 ★ ⏳ Un dato de una tabla no es una demostración
- 28.4 ★ ⏳ Comprobación independiente de los cálculos
- 28.5 ★ ⏳ Versiones del programa y tests de regresión
- 28.6 ★ ⏳ Casos de prueba
- 28.7 ★ ⏳ Cómo detectar errores en resultados generados por una IA
- 28.8 ★ ⏳ Qué evidencia hace falta para afirmar un nuevo récord
- EP-VI ⏳ Examen acumulativo de la Parte VI

PARTE VII. INVESTIGACIÓN MATEMÁTICA AUTÓNOMA

Módulo 29. Cómo leer artículos científicos

- 29.1 ★ ⏳ Anatomía de un artículo: resumen, introducción, definiciones, teoremas, experimentos y conclusiones
- 29.2 ★ ⏳ Identificar la contribución principal
- 29.3 ★ ⏳ Entender la notación
- 29.4 ★ ⏳ Seguir una demostración
- 29.5 ★ ⏳ Reproducir un experimento
- 29.6 ★ ⏳ Identificar limitaciones

Módulo 30. Cómo diseñar una investigación

- 30.1 ★ ⏳ Elegir una pregunta concreta
- 30.2 ★ ⏳ Investigar el estado del arte
- 30.3 ★ ⏳ Establecer una hipótesis
- 30.4 ★ ⏳ Diseñar pruebas y medir resultados
- 30.5 ★ ⏳ Documentar experimentos negativos
- 30.6 ★ ⏳ Formular conclusiones proporcionadas a la evidencia
- 30.7 ★ ⏳ Una idea prometedora no es un descubrimiento demostrado

Módulo 31. Git y GitHub para matemáticas computacionales

- 31.1 ★ ⏳ Repositorios
- 31.2 ★ ⏳ Commits e historial de cambios
- 31.3 ★ ⏳ Ramas
- 31.4 ★ ⏳ README y documentación
- 31.5 ★ ⏳ Tests automáticos
- 31.6 ★ ⏳ Registro de experimentos
- 31.7 ★ ⏳ Reproducibilidad
- 31.8 ★ ⏳ Publicar código y documentación
- 31.9 ★ ⏳ Registrar las contribuciones humanas y el uso de IA con transparencia

Módulo 32. Proyecto final: un sistema de investigación de códigos lineales sobre GF(7)

Se construye progresivamente: cada pieza se empieza en cuanto se tienen sus prerrequisitos (ver mapa), y solo se amplía cuando las pruebas de la anterior son correctas.

- 32.1 ★ ⏳ Biblioteca de operaciones sobre GF(7)
- 32.2 ★ ⏳ Operaciones matriciales
- 32.3 ★ ⏳ Cálculo del rango
- 32.4 ★ ⏳ Generación de las palabras de un código
- 32.5 ★ ⏳ Pesos y distancia mínima exactos (casos verificables)
- 32.6 ★ ⏳ Pruebas unitarias
- 32.7 ★ ⏳ Verificador independiente para matrices pequeñas
- 32.8 ★ ⏳ Buscador de candidatos
- 32.9 ★ ⏳ Sistema de registro reproducible
- 32.10 ★ ⏳ Informe final: resultados, limitaciones y conclusiones

---

APÉNDICES

- A ✅ Glosario (crece con cada capítulo)
- B ✅ Hoja de fórmulas (crece con cada capítulo)
- C ✅ Errores frecuentes y cómo corregirlos (crece con cada capítulo)
- D ✅ Bibliografía comentada y estado de verificación de cada fuente
- E ✅ Registro de progreso
- F ⏳ [+] Plantilla de cuaderno de laboratorio (se publicará con el Módulo 9)

Tamaño del plan

Parte	Módulos	Capítulos	★ núcleo	◇ ampliación
I	1–5	46	42	4
II	6–9	38	36	2
III	10–13	45	44	1
IV	14–16	25	20	5
V	17–23	62	58	4
VI	24–28	38	35	3
VII	29–32	32	32	0
Total	**32**	**286**	**267**	**19**

Los capítulos tienen extensiones distintas: los de Parte I que ya conozcas de clase se pueden superar rápidamente con la **prueba de salto** (ver Cómo estudiar este libro); los de laboratorio son más cortos en teoría y más largos en código.

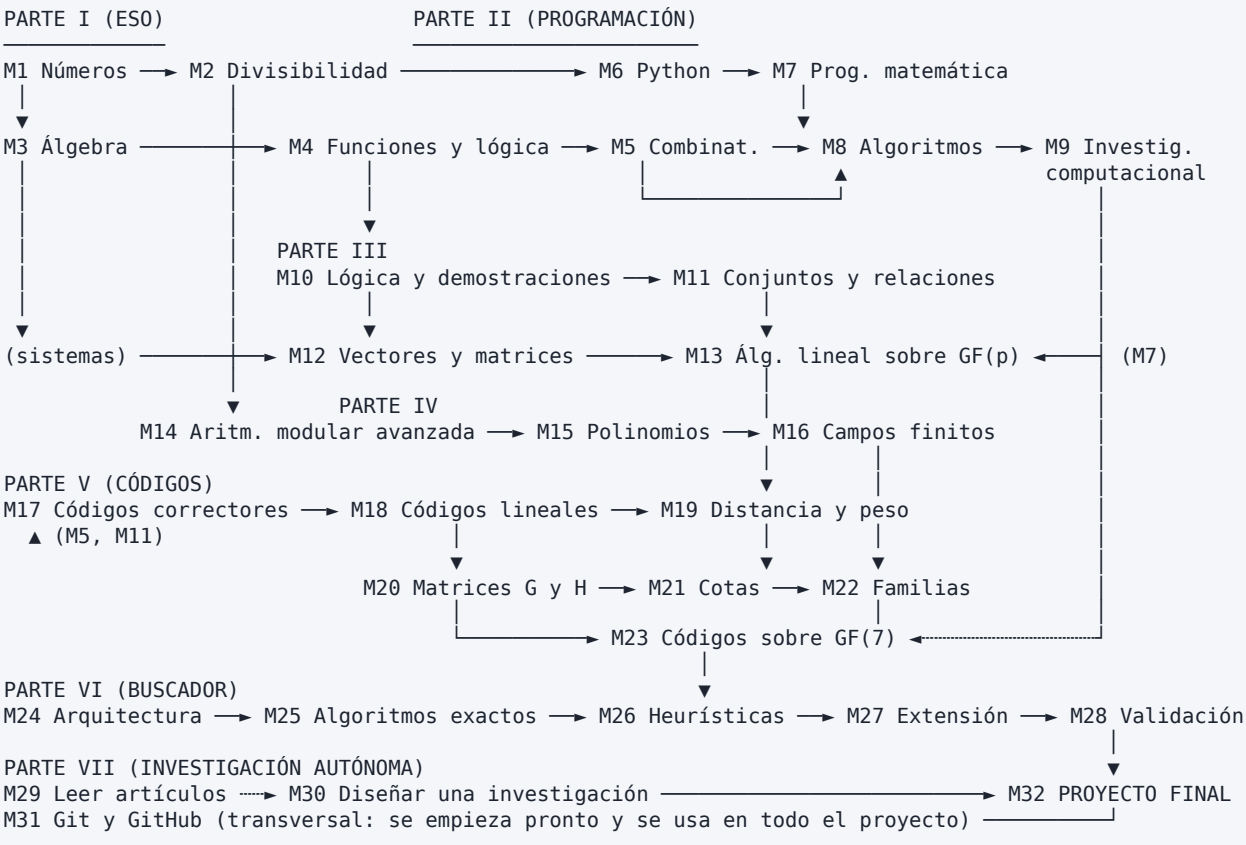
Mapa de prerrequisitos y orden de estudio

> Este documento responde a tres preguntas: **¿qué necesito saber antes de cada módulo?**, **¿falta algún prerrequisito en el temario?** y **¿en qué orden conviene estudiar?** Vuelve a él cada vez que empieces un módulo nuevo.

---

1. Mapa general de módulos

Una flecha **A** ⟶ **B** significa «A es prerrequisito de B». Las líneas punteadas (⋯) indican prerrequisitos recomendados pero no imprescindibles.



---

2. Tabla de prerrequisitos por módulo

Módulo	Prerrequisitos imprescindibles	Recomendados	Desbloquea
M1 Números	Primaria, 1.º–2.º ESO	—	M2, M3, M6
M2 Divisibilidad	M1 (1.1–1.3)	—	M6, M7, M13, M14
M3 Álgebra	M1	M2	M4, M12, M15
M4 Funciones, patrones y razonamiento	M1, M3 (3.1–3.8)	—	M5, M8, M10
M5 Combinatoria y probabilidad	M1, M4 (4.6–4.7)	M2	M8, M17, M19, M21
M6 Python	M1, M2 (2.5)	—	M7
M7 Programación matemática	M6, M2 (2.5–2.8)	M5 (5.1–5.3)	M8, M9, M13.12
M8 Algoritmos y complejidad	M7, M4 (4.4–4.5)	M5 (5.9)	M9, M24–M26
M9 Investigación computacional	M7, M8 (8.1–8.5)	M31 (31.1–31.3)	M24–M28
M10 Lógica y demostraciones	M4 (4.8–4.10)	M2	M11, M12, todos los teoremas posteriores
M11 Conjuntos y relaciones	M10 (10.1–10.4)	M2 (2.8)	M13, M17, M21
M12 Vectores y matrices	M3 (3.9), M10	M11	M13
M13 Álgebra lineal sobre GF(p)	M12, M2 (2.7–2.8), M11 (11.5)	M7	M18–M20, M23
M14 Aritmética modular avanzada	M2, M10, M11 (11.5)	M13	M16
M15 Polinomios	M3 (3.2–3.6), M13 (13.1–13.4)	M14	M16, M22
M16 Campos finitos	M14, M15	—	M22 (cíclicos, BCH, RS sobre extensiones)

Módulo	Prerrequisitos imprescindibles	Recomendados	Desbloquea
M17 Códigos correctores	M5 (5.1–5.6), M11 (11.3)	M10	M18
M18 Códigos lineales	M17, M13	—	M19, M20
M19 Distancia mínima y peso	M18, M5 (5.3)	M8 (8.3–8.4)	M20, M21, M25
M20 Matrices G y H	M19, M13 (13.7–13.10), M12 (12.12, 12.14)	—	M21, M23
M21 Cotas y parámetros	M20, M5 (5.3–5.4), M4 (4.5, 4.7), M1 (1.4: ⌈x⌉)	M11 (11.8)	M22, M27, M28
M22 Familias de códigos	M21; para 22.5–22.9: M15, M16	—	M23, M27
M23 Códigos sobre GF(7)	M13, M18–M20	M21, M22	M24
M24 Arquitectura de un buscador	M23, M9	M31	M25
M25 Algoritmos exactos	M24, M8	M19 (19.5–19.6)	M26
M26 Heurísticas	M25, M7 (7.9)	M5 (5.8)	M27
M27 Extensión y construcción	M21, M26	M22	M28
M28 Validación científica	M25–M27, M9	M10 (10.10)	M32
M29 Leer artículos	M10, M18–M21	M22	M30
M30 Diseñar una investigación	M9, M29	M28	M32
M31 Git y GitHub	M6	—	M9, M24, M32
M32 Proyecto final	M28, M30, M31	todos	—

---

3. Cadenas críticas de conceptos

Estas son las «cadenas» que llevan desde lo que sabes hoy hasta los conceptos centrales de tu investigación. Si un eslabón falla, todo lo que viene después se tambalea.

Cadena A — De los restos al campo GF(7):
1.1 (conjuntos de números, operación cerrada) → 1.2 (enteros) → 2.5 (división con resto) → 2.6 (Euclides) → 2.7 (Bézout) → 2.8 (aritmética modular) → 11.5 (clases de restos) → 13.1 (campo) → 13.3 (GF(7)) → 13.4 (inversos).

Cadena B — Álgebra lineal:
3.9 (sistemas) → 12.1–12.2 (vectores, combinaciones lineales) → 12.5 (producto de matrices) → 12.7 (Gauss) → 12.8 (rango) → 12.9–12.11 (independencia, base, dimensión) → 13.7–13.9 (lo mismo sobre GF(p)).

Cadena C — Códigos lineales:
11.3 (palabras como elementos de A^n) → 17.4 (distancia de Hamming) → 17.5 (distancia mínima) → 18.1–18.5 (códigos lineales y matriz generadora) → 19.1–19.4 (peso y cálculo de d) → 20.1–20.6 (G, H, síndromes) → 21 (cotas).

Cadena D — Programación para investigar:
6.x → 7.1–7.2 (aritmética modular en Python) → 7.4–7.5 (combinaciones y búsqueda exhaustiva) → 7.7–7.9 (aserciones, tests, semillas) → 8.3–8.6 (complejidad y optimización) → 9.x (método experimental) → 24–28 (buscador y validación).

Cadena E — De los polinomios a los códigos cíclicos:
3.2–3.6 (polinomios) → 15.x (polinomios sobre un campo) → 16.4 (extensiones) → 22.5 (cíclicos) → 22.9 (constacíclicos, cuasicíclicos, cuasitwisted).

Cadena F — Cotas:
1.4 ($\lceil x \rceil$) + 1.5 (potencias) + 4.5 (logaritmos) + 4.7 (Σ) + 5.3 (números combinatorios) → 17.6 (bolas de Hamming) → 21.2–21.5 (Singleton, Hamming, Griesmer, Gilbert–Varshamov).

---

4. Comprobación de prerrequisitos que faltaban en el temario original

Al diseñar el índice se revisó, concepto por concepto, **dónde se usa cada idea por primera vez y dónde se enseña antes**. Estas son las lagunas detectadas y cómo se han cubierto (marcadas con [+] en el índice):

Concepto	Se necesita en	Problema detectado	Solución
Inverso módulo p	13.4, 13.7 (Gauss mod p)	El Módulo 14 (aritmética modular avanzada) iba **después** del 13.	Nuevo cap. **2.7** (Bézout, Euclides extendido); 13.4 es autocontenido. El Módulo 14 profundiza después.
Partes enteras ⌊x⌋, ⌈x⌉	21.4 (cota de Griesmer), 19.5	No aparecían.	Añadidas al cap. **1.4**.
Logaritmos	8.4 (O grande), 18.7, 21.5	No aparecían.	Nuevo cap. **4.5**.
Notación Σ y Π	19.7, 21.3, 21.5	No aparecía.	Nuevo cap. **4.7**.
Binomio de Newton	19.7, 21.3	No aparecía.	Nuevo cap. **5.4**.
Valor esperado	26 (análisis de búsquedas aleatorias), 21.5	No aparecía.	Nuevo cap. **5.8** (◇).
Principio del palomar	1.1 (idea), 21, 25	No aparecía.	Nuevo cap. **11.6**.
Permutaciones como funciones	21.1 (códigos equivalentes), 24.4	No aparecía.	Nuevo cap. **11.8**.
Producto escalar y ortogonalidad	20.3–20.4 (matriz de control, código dual)	No aparecía.	Nuevos caps. **12.14** y **13.10**.
Determinantes	22.7–22.8 (Vandermonde, Reed–Solomon)	No aparecían.	Nuevo cap. **12.13** (◇).
Grupos, orden, raíces primitivas	16.6, 22.5–22.7	No aparecían.	Nuevo cap. **14.6**.
Interpolación de Lagrange	22.7 (Reed–Solomon)	No aparecía.	Nuevo cap. **15.8**.

Concepto	Se necesita en	Problema detectado	Solución
Anillos	16 (GF(p)[x]/(f))	No aparecían.	Nuevo cap. **16.1**.
Polinomios mínimos, clases ciclotómicas	22.5–22.6 (cíclicos, BCH)	No aparecían.	Nuevo cap. **16.7** (◇).
Ahorro por escalares (q^k − 1)/(q − 1)	24–25 (eficiencia del buscador)	No aparecía.	Nuevo cap. **19.5**.
d mediante columnas de H	20, 22.3 (Hamming)	No aparecía.	Nuevo cap. **19.6**.
Cotas de Griesmer y Plotkin	21.8 (lectura de tablas), 21.9	Faltaban: Griesmer es clave para códigos lineales.	Nuevos caps. **21.4** y **21.6** (◇).
Códigos símplex	22.3, 27	No aparecían.	Nuevo cap. **22.4**.
Códigos de Gray q-arios; Brouwer–Zimmermann	25 (cálculo eficiente de d)	No aparecían.	Añadidos en **25.2** y **25.6** (◇).
Construcciones (u | u+v), X	27.6 (mejora de parámetros)	No aparecían.	Nuevo cap. **27.4** (◇).
Instalación y errores en Python	6.x	Se daba por supuesto.	Nuevos caps. **6.1**, **6.11**, **6.12**.
Representación binaria	1.1 (por qué falla 0,1), 8.6	No aparecía.	Nuevo cap. **8.7** (◇).
Cómo redactar demostraciones	10–30	No aparecía.	Nuevo cap. **10.11**.

---

5. Orden de estudio recomendado

La numeración de los módulos es **temática**. El orden de estudio es **pedagógico**: intercala matemáticas y programación para que pronto puedas experimentar, y llega a los códigos sobre GF(7) **antes** de estudiar los campos de extensión (que tu proyecto actual no necesita).

Etapa	Qué estudiar	En paralelo	Hito al terminar
1. Base numérica	M1 → M2	Desde 2.5: empezar M6 (6.1–6.5)	Sabes operar con restos a mano y escribir programas sencillos.
2. Álgebra y Python	M3 → M4	M6 completo → M7; M31 (31.1–31.3)	**Hito 1:** calculas aritmética módulo 7 a mano y en Python, con tests.
3. Contar y razonar	M5 → M10 → M11	M8	Sabes contar posibilidades, estimar costes y escribir demostraciones sencillas.
4. Álgebra lineal	M12 → M13	M9	**Hito 2:** calculas rangos y resuelves sistemas sobre GF(7) a mano y con tu propia biblioteca probada.
5. Primer recorrido por los códigos	M17 → M18 → M19 → M20 → M23	M32.1–32.7 (primeras piezas del proyecto)	**Hito 3:** calculas los parámetros $[n, k, d]_7$ de códigos pequeños y **verificas** resultados generados por una IA en casos pequeños.
6. Teoría avanzada	M14 → M15 → M16 → M21 → M22	M29	**Hito 4:** comparas tus códigos con las cotas teóricas y con las tablas de mejores códigos conocidos.
7. Buscador e investigación	M24 → M25 → M26 → M27 → M28	M30, M31 completo	**Hito 5:** buscador reproducible, verificador independiente e informe con conclusiones proporcionadas a la evidencia (M32).

Estimación orientativa (es una estimación, no un compromiso): con una media de 2–3 sesiones de 45–60 minutos por capítulo y 4 sesiones por semana, el recorrido completo lleva **varios años**. Pero no tienes que esperar al final para hacer cosas reales: el **Hito 3** (calcular y verificar parámetros de códigos pequeños sobre GF(7)) llega a mitad del camino, y los capítulos de Parte I que ya domines de clase se pueden superar rápidamente con la **prueba de salto** (ver Cómo estudiar este libro).

---

6. Qué mirar antes de empezar cada capítulo

Cada capítulo tiene una sección **«2. Prerrequisitos»** con una **comprobación rápida**. Regla práctica:

- Si fallas **una** pregunta de la comprobación rápida: repasa ese punto y sigue.
- Si fallas **dos o más**: vuelve al capítulo indicado y repite su prueba de dominio antes de continuar.

Capítulo 1.1 — Los conjuntos de números: $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ y $\mathbb{R}$

Parte I · Fundamentos matemáticos de ESO · **Módulo 1** · Números y operaciones

Nivel: 3.º ESO (punto de partida del libro) · **Tipo:** ★ núcleo imprescindible

Tiempo orientativo: 5–6 sesiones de 45–60 minutos, repartidas en 4 unidades.

Verificación: todos los cálculos de este capítulo se han comprobado con el programa `codigo/verificaciones/verificar_cap_1_1.py` (ejecutado con Python 3.13.16 el 12/10/2026: todas las comprobaciones superadas).

Cómo está organizado este capítulo

Unidad	Qué estudias	Secciones	Ejercicios recomendados
A (sesiones 1–2)	Naturales, enteros y racionales; clasificar números	§3, §4.1–4.2, §5, Ejemplo 1.1	A1, A2
B (sesiones 2–3)	Fracciones y decimales en los dos sentidos	§4.3, Propiedades 1.4–1.6, Ejemplo 1.2	A3, A4, B2, B3
C (sesión 4)	Irracionales, números reales, orden y densidad	§4.2 (Def. 1.6), §4.4, Prop. 1.3	A5, B1, B4, B5, B6
D (sesión 5)	Operaciones cerradas, contraejemplos y conexión con GF(7)	§4.5, Ejemplo 1.3	A6, problemas, C1, C2
Cierre (sesión 6)	Comprensión y prueba de dominio	§15, §16	Prueba de dominio

Etiquetas que verás en todo el libro (para distinguir siempre qué tipo de afirmación es cada cosa):

Etiqueta	Significado
[DEFINICIÓN]	Un acuerdo sobre el significado de una palabra o símbolo. No se demuestra: se acuerda.
[PROPIEDAD — demostrada] / **[TEOREMA]**	Afirmación verdadera cuya demostración aparece en este libro (aquí o en el lugar indicado).
[PROPIEDAD — demostración pendiente: …]	Afirmación verdadera y conocida, cuya demostración necesita herramientas que todavía no tienes. Se indica dónde se demostrará.
[HECHO ACEPTADO]	Resultado demostrado por matemáticos, cuya demostración queda fuera del alcance de este libro.
[CONJETURA]	Afirmación que parece verdadera pero que **no** está demostrada (o que tú aún no has demostrado).
[VERIFICADO POR PROGRAMA]	Comprobado por ordenador en un número finito de casos. Es **evidencia**, no demostración.
[EJEMPLO]	Un caso concreto. Un ejemplo ilustra, pero no demuestra una afirmación general.
[AVANCE]	Idea de un capítulo futuro. No se exige dominarla todavía.

> 📌 **Las soluciones están plegadas.** En GitHub aparecen como recuadros cerrados con un triángulo (por ejemplo, «Solución A1»): ábrelos solo cuando hayas intentado el ejercicio por tu cuenta. Las respuestas de la prueba de dominio están en el **anexo final** del capítulo.

---

1. Título y objetivos

Título: Los conjuntos de números: naturales ($\mathbb{N}$), enteros ($\mathbb{Z}$), racionales ($\mathbb{Q}$) y reales ($\mathbb{R}$).

Al terminar este capítulo serás capaz de:

1. **Clasificar** cualquier número en $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, en los irracionales o en $\mathbb{R}$, y justificarlo.
2. **Explicar por qué existe cada conjunto**: qué problema no se podía resolver con el anterior.
3. **Convertir una fracción en decimal** y **predecir sin dividir** si el decimal será exacto, periódico puro o periódico mixto.

4. **Convertir un decimal exacto o periódico en fracción** (fracción generatriz) y comprobar el resultado por otro camino.
5. **Ordenar números** y situarlos en la recta numérica.
6. **Decidir si una operación es cerrada** en un conjunto y, si no lo es, dar un **contraejemplo**.
7. **Distinguir entre ejemplo, evidencia y demostración**, una habilidad que usarás durante todo el libro.
8. **Entender por qué los ordenadores se equivocan con algunos decimales** y por qué en tu investigación sobre GF(7) se trabaja solo con números enteros.
9. [AVANCE] **Ver por primera vez** un conjunto **finito** de números en el que se puede sumar, restar, multiplicar y dividir: la semilla de GF(7).

---

2. Prerrequisitos

Este es el primer capítulo, así que solo necesitas lo que se aprende en Primaria y en 1.º–2.º de ESO:

- Sumar, restar, multiplicar y dividir números naturales.
- Hacer una **división con resto** (por ejemplo, 47 entre 5 da cociente 9 y resto 2, porque $47 = 9 \cdot 5 + 2$).
- Saber qué es una **fracción** y simplificar fracciones sencillas.
- Haber usado **números negativos** en contextos como temperaturas o plantas de un edificio.
- Saber qué es un **número primo** (un natural mayor que 1 cuyos únicos divisores son 1 y él mismo: 2, 3, 5, 7, 11, …). Lo usaremos solo en una propiedad; se estudia a fondo en el Módulo 2.

Comprobación rápida de prerrequisitos

Responde sin calculadora. Si fallas **dos o más**, repasa esos puntos con tu libro de clase antes de seguir (o pide ayuda a tu tutor). Los capítulos 1.2, 1.3 y 2.5 los tratarán de nuevo con detalle.

1. ¿Cuánto es $36 : 4$?
2. ¿Cuál es el resto de dividir 47 entre 5?
3. Simplifica $18/24$ todo lo posible.
4. ¿Qué temperatura es más baja: -8 °C o -3 °C?
5. ¿Es 9 un número primo?
6. Escribe 0,25 como fracción.

▼ **Ver respuestas de la comprobación rápida**

1. 9, porque 9 · 4 = 36.
2. Resto 2, porque 47 = 9 · 5 + 2 y 2 < 5.
3. 18/24 = 3/4 (dividiendo numerador y denominador entre 6).
4. −8 °C es más baja (está más lejos del cero hacia abajo).
5. No: 9 = 3 · 3, así que 3 es un divisor distinto de 1 y de 9.
6. 0,25 = 25/100 = 1/4.

Notación que usaremos (decisiones del libro):

- Escribimos los decimales con **coma**, como en España: 0,75. Cuidado: en Python (y en casi todos los lenguajes de programación) se escribe con **punto**: `0.75`.
- Cuando escribamos una lista de decimales, los separaremos con **punto y coma** para no confundirnos: 0,5; 1,2; 3,75.
- Los decimales periódicos se escriben aquí con el **periodo entre paréntesis**: 0,41(6) significa 0,41666…, y 0,(27) significa 0,272727… En tu libro de clase verás el periodo marcado con un arco encima de las cifras; significa lo mismo. Usamos paréntesis porque se pueden escribir con cualquier teclado y en un programa.
- Usamos el signo menos tipográfico «−» en el texto; en los programas se escribe con el guion «-».

---

3. Explicación intuitiva

3.1. La historia de los números es una historia de problemas sin respuesta

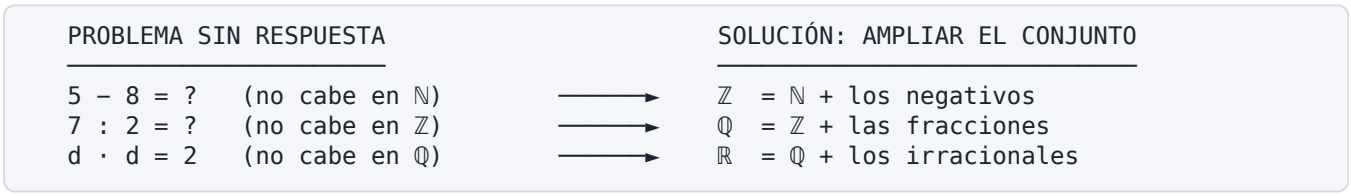
Imagina que los números son las **herramientas de una caja**. Cada vez que aparece un problema que las herramientas que tienes no pueden resolver, inventas una herramienta nueva y la añades a la caja. Las herramientas antiguas no se tiran: siguen dentro.

Paso 1. Contar. Para contar ovejas, alumnos o páginas usamos 0, 1, 2, 3, … Son los **números naturales**, ℕ. Con ellos siempre se puede **sumar** (3 + 5 = 8) y **multiplicar** (3 · 5 = 15) sin salir de ℕ.

Paso 2. Restar sin límite. ¿Cuánto es $5 - 8$? Con los naturales **no hay respuesta**: no puedes quitar 8 ovejas de un rebaño de 5. Pero en la vida real sí tiene sentido: si tienes 5 € y gastas 8 €, debes 3 €; si estás en la planta 5 y bajas 8 plantas, llegas al sótano -3. Para que la resta **siempre** tenga respuesta, se añaden los negativos: -1, -2, -3, … Así nacen los **números enteros**, $\mathbb{Z}$.

Paso 3. Repartir sin límite. ¿Cuánto es $7 : 2$? En los enteros **no hay respuesta exacta**: no existe un entero que multiplicado por 2 dé 7. Pero si repartes 7 pizzas entre 2 personas, cada una come 3 pizzas y media: 7/2. Para que la división (entre un número distinto de cero) **siempre** tenga respuesta, se añaden las fracciones. Así nacen los **números racionales**, $\mathbb{Q}$.

Paso 4. Medir sin límite. Dibuja un cuadrado de lado 1. ¿Cuánto mide su diagonal? Por el teorema de Pitágoras, su longitud d cumple $d^2 = 1^2 + 1^2 = 2$. Los griegos descubrieron algo sorprendente: **ninguna fracción** multiplicada por sí misma da exactamente 2. La diagonal existe (¡la puedes dibujar!), pero su longitud, $\sqrt{2}$, no es racional. Para que **toda longitud** tenga un número, se completan los racionales con los **irracionales** ($\sqrt{2}$, π, …). El conjunto completo son los **números reales**, $\mathbb{R}$.



Fíjate en el **patrón**: cada conjunto nuevo se inventa para que una operación que «fallaba» deje de fallar. Ese patrón es la idea más importante del capítulo.

3.2. El giro de tu investigación: ir en sentido contrario

En la teoría de códigos correctores de errores pasa algo curioso. No queremos conjuntos **infinitos** como $\mathbb{Q}$ o $\mathbb{R}$: un ordenador no puede guardar infinitos números ni infinitas cifras decimales. Queremos un conjunto **pequeño y finito** de símbolos (por ejemplo, solo siete: 0, 1, 2, 3, 4, 5 y 6) en el que, aun así, se pueda **sumar, restar, multiplicar y dividir** sin salirse nunca del conjunto, igual que en $\mathbb{Q}$.

¿Es posible? Piensa en los **días de la semana**. Si hoy es lunes, dentro de 10 días será jueves, porque $10 = 7 + 3$ y cada 7 días se vuelve al mismo día. Con los días de la semana «sumamos» y el resultado **siempre** vuelve a ser uno de los 7 días. Esa es la semilla de **GF(7)**, el campo finito de 7 elementos sobre el que trata tu investigación. Lo verás por primera vez en el Ejemplo 1.3 de este capítulo y lo estudiarás con todo detalle en los Módulos 2, 13 y 16.

3.3. Una segunda idea clave: exacto frente a aproximado

1/3 es un número **exacto**. Pero si lo escribes en decimal, 0,333..., necesitas **infinitas** cifras. Si te quedas con 0,333, estás usando una **aproximación**. Los ordenadores, con los decimales normales, hacen continuamente aproximaciones de este tipo, y eso puede producir resultados **falsos** en una investigación. En este capítulo aprenderás a reconocer cuándo un número tiene expresión decimal finita y cuándo no, y por qué en tu proyecto solo se usarán **números enteros**, con los que el ordenador es exacto.

---

4. Definiciones formales

4.1. Un poco de lenguaje de conjuntos

> **[DEFINICIÓN 1.1] Conjunto y elemento.** Un **conjunto** es una colección de objetos en la que está perfectamente claro qué objetos están dentro y cuáles no. Cada objeto del conjunto se llama **elemento**. Si x es un elemento del conjunto A escribimos **$x \in A$** («x pertenece a A»); si no lo es, escribimos **$x \notin A$** («x no pertenece a A»).

Un conjunto se puede describir **enumerando** sus elementos entre llaves, como {0, 1, 2, 3, 4, 5, 6}, o **describiendo una condición**, como «el conjunto de los números pares». El orden y las repeticiones no importan: {1, 2, 3} y {3, 1, 2, 2} son el mismo conjunto.

> **[DEFINICIÓN 1.2] Subconjunto.** Decimos que A es un **subconjunto** de B, y escribimos **$A \subset B$**, si todo elemento de A es también elemento de B.

- $\in$ relaciona un **elemento** con un **conjunto**: $2 \in \mathbb{N}$.
- $\subset$ relaciona un **conjunto** con otro **conjunto**: $\{2\} \subset \mathbb{N}$, $\mathbb{N} \subset \mathbb{Z}$.

(El estudio sistemático de los conjuntos llega en el Módulo 11. Por ahora basta con esto.)

4.2. Los cuatro conjuntos de números

> **[DEFINICIÓN 1.3] Números naturales.**
>
> **$\mathbb{N} = \{0, 1, 2, 3, 4, \ldots\}$**
>
> Los puntos suspensivos significan «y así sucesivamente, sin fin»: $\mathbb{N}$ es infinito.

⚠️ **Convenio importante.** Algunos libros no incluyen el 0 en $\mathbb{N}$ y empiezan en 1. **En este libro el 0 es natural**, porque en informática y en teoría de códigos contamos desde 0 (la palabra de peso 0, la posición 0 de una lista…). Cuando un autor necesita dejar claro que no incluye el 0 suele escribir $\mathbb{N}^*$ o $\mathbb{Z}^+$ (enteros positivos). **Lección de investigador:** antes de usar un resultado, comprueba siempre qué definiciones usa el autor. Dos libros pueden decir cosas que parecen contradictorias solo porque definen la misma palabra de forma distinta.

> **[DEFINICIÓN 1.4] Números enteros.**
>
> **$\mathbb{Z} = \{\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots\}$**
>
> Son los naturales junto con sus **opuestos**. El **opuesto** de un número a es el número $-a$ que cumple $a + (-a) = 0$. El opuesto de 5 es -5; el opuesto de -5 es 5; el opuesto de 0 es 0.

(La letra $\mathbb{Z}$ viene del alemán *Zahlen*, «números».)

> **[DEFINICIÓN 1.5] Números racionales.**
>
> **$\mathbb{Q} = \{\ a/b : a \in \mathbb{Z},\ b \in \mathbb{Z},\ b \neq 0\ \}$**
>
> Se lee: «$\mathbb{Q}$ es el conjunto de todos los números de la forma a/b, tales que a es entero, b es entero y b es distinto de cero». En la fracción a/b, a es el **numerador** y b el **denominador**.

Tres aclaraciones imprescindibles:

1. **Fracciones equivalentes.** Muchas fracciones representan el **mismo** número racional: $1/2 = 2/4 = 3/6 = -5/-10$. Dos fracciones a/b y c/d son iguales exactamente cuando **$a \cdot d = b \cdot c$** (los «productos cruzados» coinciden). Por ejemplo, $2/4 = 3/6$ porque $2 \cdot 6 = 12 = 4 \cdot 3$.
2. **Fracción irreducible.** Una fracción es **irreducible** si no se puede simplificar más, es decir, si el numerador y el denominador no tienen ningún divisor común mayor que 1. 3/4 es irreducible; 6/8 no lo es (ambos son divisibles entre

2). Todo número racional se puede escribir como fracción irreducible con denominador positivo, y de **una sola manera**.
3. **¿Por qué b ≠ 0?** Porque dividir entre 0 no tiene sentido: si 5/0 fuera un número x, tendría que cumplir x · 0 = 5, pero cualquier número multiplicado por 0 da 0. En cambio, 0/5 = 0 sin ningún problema (0 · 5 = 0).

(La letra ℚ viene de *quotient*, «cociente».)

> **[DEFINICIÓN 1.6] Números reales e irracionales.**
>
> Un **número real** es cualquier número que se puede escribir con una **expresión decimal**, finita o infinita, con signo. Cada número real corresponde a **un punto** de la recta numérica, y cada punto de la recta corresponde a un número real. El conjunto de los números reales se llama ℝ. Un número real es **irracional** si **no** es racional, es decir, si no se puede escribir como fracción de enteros. Al conjunto de los irracionales lo llamaremos 𝕀 (también se escribe ℝ \ ℚ, «ℝ menos ℚ»).

🔎 **Honestidad matemática.** La definición anterior es la adecuada para ESO, pero no es la definición rigurosa que se usa en la universidad. Definir ℝ con todo rigor (con las llamadas «cortaduras de Dedekind» o con «sucesiones de Cauchy») es un tema de primer curso de Análisis Matemático. Para la teoría de códigos **casi nunca necesitarás** ℝ: tu investigación vive en conjuntos finitos. Lo verás en la Ampliación B, al final del capítulo.

4.3. Expresiones decimales

[DEFINICIÓN 1.7] Tipos de expresión decimal.

- Un decimal es **exacto** (o finito) si tiene un número finito de cifras decimales: 0,175.
- Un decimal es **periódico** si, a partir de cierta cifra, un bloque de cifras se repite indefinidamente. Ese bloque se llama **periodo**.
  - **Periódico puro:** el periodo empieza justo después de la coma: 0,(428571) = 0,428571428571…
  - **Periódico mixto:** entre la coma y el periodo hay cifras que no se repiten, llamadas **anteperiodo**: en 0,41(6) = 0,41666…, el anteperiodo es 41 y el periodo es 6.
- Un decimal es **infinito no periódico** si tiene infinitas cifras y ningún bloque se repite indefinidamente: 1,4142135623… (√2).
- La **parte entera** es lo que hay antes de la coma: en 2,1(6) la parte entera es 2.
- La **fracción generatriz** de un decimal exacto o periódico es la fracción irreducible que tiene ese decimal.

⚠️ «Que se repitan cifras a simple vista» **no** basta para ser periódico. 0,1212 es **exacto** (tiene 4 cifras y se acaba); 0,(12) = 0,121212… es periódico. Y en 0,1010010001… se repiten muchos 0 y 1, pero **no hay un bloque fijo** que se repita: no es periódico.

4.4. El orden

[DEFINICIÓN 1.8] Orden. Para dos números reales a y b, escribimos **a < b** («a es menor que b») si la diferencia **b − a es positiva**. En la recta numérica, a < b significa que a está **a la izquierda** de b. Escribimos a ≤ b si a < b o a = b.

Consecuencias que usarás constantemente:

- Todo número negativo es menor que 0, y 0 es menor que todo número positivo.
- Entre negativos, **el que está más lejos del cero es el menor**: −7 < −3, porque −3 − (−7) = 4 es positivo.
- Para comparar dos fracciones con **denominadores positivos**: a/b < c/d exactamente cuando **a · d < c · b**. Por ejemplo, 3/7 < 4/9 porque 3 · 9 = 27 < 28 = 4 · 7.

- Para comparar decimales, se comparan las partes enteras y después cifra a cifra: $0{,}6 < 0{,}666\ldots < 0{,}67$.

4.5. Operaciones cerradas y contraejemplos

> **[DEFINICIÓN 1.9] Operación cerrada.** Un conjunto A es **cerrado** para una operación (por ejemplo, la suma) si, **siempre** que a y b son elementos de A, el resultado de operarlos también es elemento de A. En símbolos, para la suma: **para todo $a \in A$ y todo $b \in A$, se cumple $a + b \in A$.**

> **[DEFINICIÓN 1.10] Contraejemplo.** Un **contraejemplo** de una afirmación del tipo «para todo… se cumple…» es **un caso concreto** en el que la afirmación **falla**.

La regla de oro (la estudiarás a fondo en el Módulo 10):

Para demostrar que una afirmación «para todo…» es…	Hace falta…
FALSA	**Un solo** contraejemplo.
VERDADERA	Un **razonamiento general** (una demostración). Ningún número de ejemplos basta, porque los casos son infinitos.

Por ejemplo, «$\mathbb{N}$ es cerrado para la resta» es **falso**: basta el contraejemplo $2 - 5 = -3 \notin \mathbb{N}$. En cambio, para afirmar que «$\mathbb{Q}$ es cerrado para la suma» no bastan mil ejemplos: hay que dar un argumento que funcione para **todas** las fracciones (lo verás en la Propiedad 1.2).

4.6. Propiedades y teoremas

> **[PROPIEDAD 1.1 — demostrada] Inclusiones.** $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$.

Demostración.

- $\mathbb{N} \subset \mathbb{Z}$: por la Definición 1.4, $\mathbb{Z}$ contiene a todos los naturales.
- $\mathbb{Z} \subset \mathbb{Q}$: todo entero n se puede escribir como $n/1$, que es una fracción de enteros con denominador $1 \neq 0$.

- $\mathbb{Q} \subset \mathbb{R}$: todo racional a/b tiene una expresión decimal (se obtiene dividiendo a entre b; la Propiedad 1.4 dice qué forma tiene), así que es real. ∎

(El símbolo ∎ marca el final de una demostración.)

> **[PROPIEDAD 1.2 — demostrada] Tabla de cierre.**
>
> | Conjunto | Suma | Resta | Multiplicación | División (divisor ≠ 0) |
> |---|---|---|---|---|
> | $\mathbb{N}$ | Sí | **No** ($2 - 5 = -3$) | Sí | **No** ($1 : 2 = 1/2$) |
> | $\mathbb{Z}$ | Sí | Sí | Sí | **No** ($1 : 2 = 1/2$) |
> | $\mathbb{Q}$ | Sí | Sí | Sí | Sí |
> | $\mathbb{R}$ | Sí | Sí | Sí | Sí |

Demostración para $\mathbb{Q}$ (para $\mathbb{N}$ y $\mathbb{Z}$, los «Sí» los aceptamos por la experiencia de Primaria, y los «No» quedan demostrados por los contraejemplos de la tabla). Sean a/b y c/d dos racionales, con a, b, c, d enteros y $b \neq 0$, $d \neq 0$.

- **Suma:** $a/b + c/d = (a \cdot d + c \cdot b)/(b \cdot d)$. El numerador $a \cdot d + c \cdot b$ es entero (sumas y productos de enteros son enteros) y el denominador $b \cdot d$ es entero y **distinto de 0** (el producto de dos números distintos de cero no es cero). Luego el resultado es racional.
- **Resta:** $a/b - c/d = (a \cdot d - c \cdot b)/(b \cdot d)$, con el mismo razonamiento.
- **Multiplicación:** $(a/b) \cdot (c/d) = (a \cdot c)/(b \cdot d)$, con $b \cdot d \neq 0$.
- **División:** si además $c/d \neq 0$ (es decir, $c \neq 0$), $(a/b) : (c/d) = (a \cdot d)/(b \cdot c)$, y $b \cdot c \neq 0$. ∎

Fíjate en que la demostración **no usa ningún ejemplo concreto**: funciona para cualesquiera a, b, c, d. Eso es lo que la convierte en demostración.

> **[PROPIEDAD 1.3 — demostrada] Densidad de $\mathbb{Q}$.** Entre dos racionales distintos siempre hay otro racional. En concreto, si $a < b$ son racionales, su **punto medio** $(a + b)/2$ es racional y cumple $a < (a + b)/2 < b$.

Demostración. $(a + b)/2$ es racional porque es una suma de racionales dividida entre 2 (Propiedad 1.2). Además:
$(a + b)/2 - a = (b - a)/2$, que es positivo porque $b - a > 0$. Luego $a < (a + b)/2$.
$b - (a + b)/2 = (b - a)/2 > 0$. Luego $(a + b)/2 < b$. ∎

Consecuencia: entre dos racionales distintos hay **infinitos** racionales (aplica el punto medio una y otra vez). Compáralo con $\mathbb{Z}$: entre 3 y 4 **no hay ningún** entero.

> **[PROPIEDAD 1.4 — demostrada] Toda fracción da un decimal exacto o periódico.** Al dividir a entre b (enteros, b > 0) se obtiene siempre un decimal exacto o un decimal periódico. Además, **el periodo tiene como mucho b − 1 cifras.**

Demostración (argumento de los restos). Haz la división larga de a entre b. Después de calcular la parte entera, en cada paso «bajas un 0»: multiplicas el resto actual r por 10, lo divides entre b, escribes la cifra del cociente y obtienes un resto nuevo. Observa dos cosas:

1. **Cada resto es uno de los números 0, 1, 2, …, b − 1** (un resto siempre es menor que el divisor).
2. **La siguiente cifra y el siguiente resto dependen solo del resto actual** (se calculan con 10·r entre b).

Si en algún momento el resto es 0, la división termina: **decimal exacto**. Si nunca es 0, todos los restos están entre 1 y b − 1: solo hay **b − 1 valores posibles**. Si miras los b primeros restos, tienes b restos y solo b − 1 valores posibles, así que **dos de ellos tienen que coincidir** (es como meter b calcetines en b − 1 cajones: algún cajón tiene dos; esta idea se llama *principio del palomar* y la estudiarás en el capítulo 11.6). Cuando un resto se repite, por el punto 2, **todo lo que viene después se repite**: las cifras forman un periodo. Como la repetición ocurre dentro de los b primeros restos, el periodo tiene como mucho b − 1 cifras. ∎

> **[PROPIEDAD 1.5] Criterio del denominador.** Sea a/b una fracción **irreducible** con b > 1.
>
> - (i) Si los únicos factores primos de b son 2 y/o 5, el decimal es **exacto**. **[demostrada]**
> - (ii) Si b no tiene ni el factor 2 ni el factor 5, el decimal es **periódico puro**.
> - (iii) Si b tiene el factor 2 o el 5 y además algún otro primo, el decimal es **periódico mixto**.
> - Además, en (ii) y (iii) el decimal **nunca** es exacto.
>
> **[demostración pendiente de (ii), (iii) y la última frase: capítulos 2.3 y 14.6]**

Demostración de (i). Si b = 2^i · 5^j (es decir, b es un producto de i doses y j cincos), llama m al mayor de los números i y j. Multiplicando numerador y denominador por los doses o cincos que falten, el denominador se convierte en 2^m · 5^m = 10^m, es decir, un 1 seguido de m ceros. Y una fracción con denominador 10^m es un decimal exacto con m cifras decimales. Por ejemplo, 7/40: 40 = 2^3 · 5, m = 3, faltan dos cincos: 7/40 = (7·25)/(40·25) = 175/1000 = 0,175. ∎

[VERIFICADO POR PROGRAMA] El criterio completo (i)–(iii) coincide con la división larga en **las 124 750 fracciones a/b con 1 ≤ a < b ≤ 500**. Esto es **evidencia**, no demostración: hay infinitas fracciones. La demostración llegará cuando tengas la factorización en primos (Módulo 2).

⚠️ El criterio exige que la fracción sea **irreducible**. 21/56 parece tener un 7 en el denominador (56 = 2^3 · 7), pero 21/56 = 3/8, y 3/8 = 0,375 es exacto.

> **[PROPIEDAD 1.6 — demostrada] Todo decimal exacto o periódico es racional.**

Demostración.

- **Exacto:** si x tiene m cifras decimales, x = N/10^m, donde N es el número que resulta de quitar la coma. Por ejemplo, 0,175 = 175/1000.
- **Periódico:** supongamos que x tiene un anteperiodo de j cifras y un periodo de p cifras. Al multiplicar x por 10^j, la coma se desplaza justo hasta el comienzo del periodo; al multiplicarlo por 10^(j+p), la coma se desplaza justo hasta el comienzo del **siguiente** periodo. Por tanto 10^(j+p)·x y 10^j·x tienen **exactamente las mismas cifras decimales**, y al restarlos las cifras decimales se cancelan: (10^(j+p) − 10^j)·x = N, donde N es un entero. Despejando, x = N / (10^(j+p) − 10^j), que es una fracción de enteros. ∎

Conclusión de las Propiedades 1.4 y 1.6:

> Un número es **racional** ⟺ su expresión decimal es **exacta o periódica**.
>
> Un número es **irracional** ⟺ su expresión decimal es **infinita y no periódica**.

(El símbolo ⟺ se lee «si y solo si» y significa que cada lado implica el otro.)

> **[OBSERVACIÓN 1.1 — demostrada] 0,(9) = 1.**

Demostración. Sea x = 0,999… Entonces 10x = 9,999… Restando, 10x − x = 9,999… − 0,999… = 9, es decir, 9x = 9, luego x = 1. ∎
Segunda comprobación: 1/3 = 0,333…; multiplicando por 3, 3 · (1/3) = 1 y 3 · 0,333… = 0,999… Luego 0,999… = 1.
No es que 0,(9) esté «muy cerca» de 1: **es** 1. Simplemente, el número 1 tiene dos expresiones decimales. (La división larga nunca produce el periodo 9; lo verás en el ejercicio B3.)

> **[TEOREMA 1.1] √2 es irracional.** No existe ninguna fracción a/b tal que (a/b)² = 2.
>
> *Demostración completa en la Ampliación A, al final del capítulo (es opcional en esta primera lectura; el método, «reducción al absurdo», se estudia en el capítulo 10.7).*

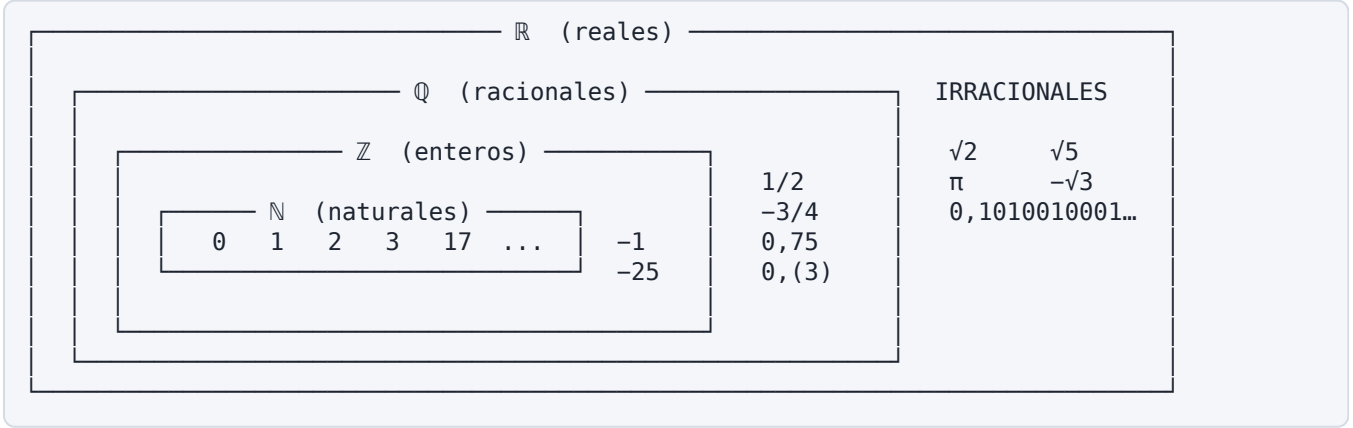
> **[PROPIEDAD 1.7 — demostración pendiente: capítulo 10.7]** Si n es un número natural que **no** es un cuadrado perfecto (no es 0, 1, 4, 9, 16, 25, …), entonces √n es irracional. Si n **sí** es cuadrado perfecto, √n es natural: √49 = 7.

> **[HECHO ACEPTADO 1.1] π es irracional.** Lo demostró Johann Heinrich Lambert en el siglo XVIII. La demostración usa herramientas universitarias y no la veremos. Por eso **π no es 3,14 ni 22/7**: esos números son racionales y solo son aproximaciones de π.

---

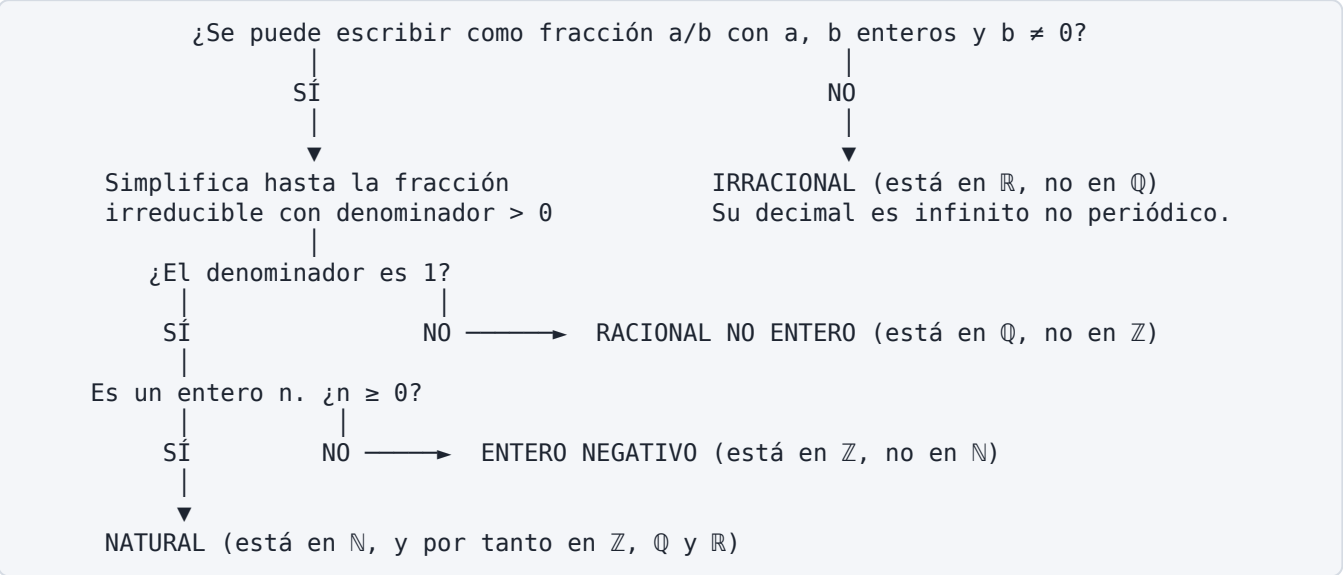
5. Esquema visual y mapa conceptual

5.1. Los conjuntos, unos dentro de otros

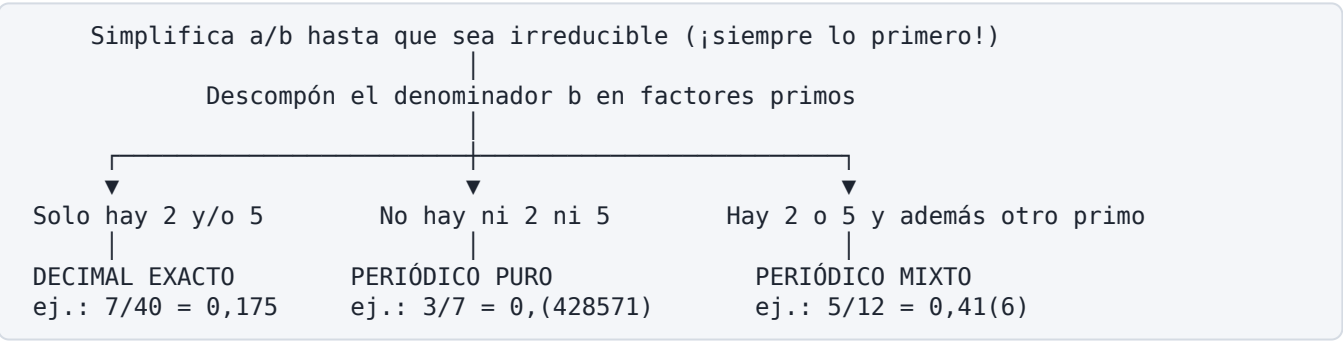


Cómo leerlo: si un número está en una caja, está también en todas las cajas que la rodean. Los irracionales están dentro de $\mathbb{R}$ pero **fuera** de $\mathbb{Q}$: un número no puede ser a la vez racional e irracional.

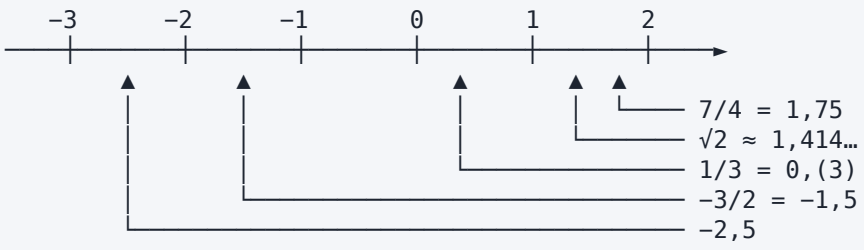
5.2. Árbol de decisión: ¿qué tipo de número es?



5.3. Árbol de decisión: ¿qué decimal dará la fracción a/b?

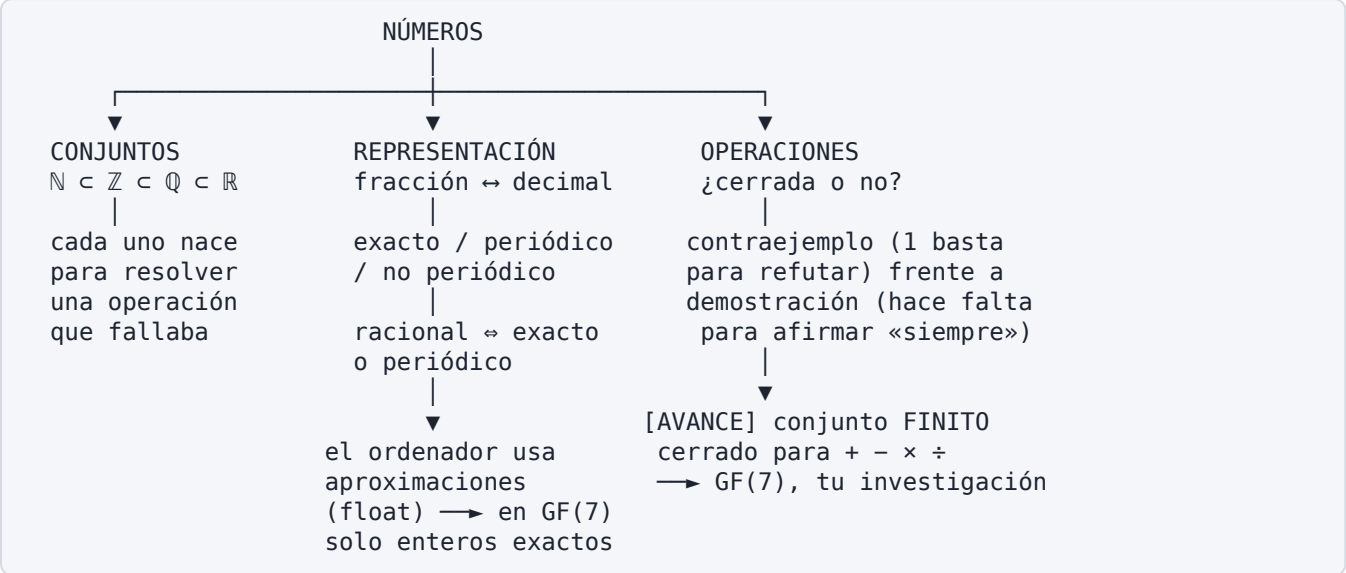


5.4. La recta numérica



Cada número real es un punto de la recta. Los racionales están «por todas partes» (Propiedad 1.3), pero **no la llenan**: entre ellos quedan los irracionales, como √2.

5.5. Mapa conceptual del capítulo



---

6. Fórmulas importantes, símbolo a símbolo

F1. Definición de $\mathbb{Q}$: $\mathbb{Q} = \{\ a/b : a, b \in \mathbb{Z},\ b \neq 0\ \}$

- { … : … } — «el conjunto de los … tales que …».
- $a, b \in \mathbb{Z}$ — a y b son números enteros.
- $b \neq 0$ — el denominador no puede ser cero.

F2. Igualdad de fracciones: $a/b = c/d \iff a \cdot d = b \cdot c$ (con $b \neq 0$, $d \neq 0$)

- Sirve para comprobar si dos fracciones son el mismo número sin simplificarlas.

F3. Suma y resta: $a/b \pm c/d = (a \cdot d \pm c \cdot b)/(b \cdot d)$

- ± significa «más o menos»: la fórmula vale para la suma (+) y para la resta (−).
- En la práctica conviene usar el mínimo común múltiplo de los denominadores (capítulo 1.3), pero esta fórmula **siempre** funciona.

F4. Producto: $(a/b) \cdot (c/d) = (a \cdot c)/(b \cdot d)$

F5. Cociente: $(a/b) : (c/d) = (a \cdot d)/(b \cdot c)$, con $c \neq 0$

F6. Comparación: si $b > 0$ y $d > 0$: $a/b < c/d \iff a \cdot d < c \cdot b$

- La condición $b > 0$, $d > 0$ es esencial: con denominadores negativos el sentido de la desigualdad puede cambiar. Por eso, antes de comparar, se escriben las fracciones con denominador positivo.

F7. Punto medio: m = (a + b)/2

- m está exactamente a mitad de camino entre a y b. Si a y b son racionales, m también.

F8. Fracción generatriz de un decimal exacto: x = N / 10^m

- N: el número que queda al quitar la coma.
- m: el número de cifras decimales.
- 10^m: un 1 seguido de m ceros (10^3 = 1000).
- Ejemplo: 0,175 = 175/1000 = 7/40.

F9. Fracción generatriz de un decimal periódico (regla clásica, que se deduce de la Propiedad 1.6):

```
            N₁ − N₂
   x  =  ──────────────
          99…9 00…0

N₁ = número escrito sin coma hasta el final del PRIMER PERIODO
N₂ = número escrito sin coma hasta el final del ANTEPERIODO
99…9 00…0 = tantos 9 como cifras tiene el periodo,
            seguidos de tantos 0 como cifras tiene el anteperiodo
```

- Ejemplo periódico mixto: 1,2(3) → numerador 123 − 12 = 111; denominador 90 (un 9 por la cifra del periodo, un 0 por la del anteperiodo) → 111/90 = 37/30.
- Ejemplo periódico puro (sin anteperiodo): 2,(18) → (218 − 2)/99 = 216/99 = 24/11.
- **No la memorices sin entenderla:** en el Ejemplo 1.2 verás de dónde sale. Si la olvidas, siempre puedes usar el **método de restar** (multiplicar por potencias de 10 y restar), que es la demostración de la regla.

F10. Cota del periodo: longitud del periodo de a/b ≤ b − 1

- Es una **cota** (un máximo posible), no el valor exacto. 1/13 tiene periodo 6, y 6 ≤ 12.

---

7. Ejemplo básico resuelto paso a paso

> **[EJEMPLO 1.1] Clasifica los siguientes números en ℕ, ℤ, ℚ, 𝕀 y ℝ:**
>
> 7; −3; 0; 2/5; −12/4; 0,75; 0,(6); √9; √5; π; −√16; 1,0100100010000… (cada vez hay un 0 más entre dos unos).

Estrategia. Para cada número: (1) **simplifícalo** todo lo posible; (2) recorre el árbol de decisión de §5.2.

1. **7.** Es un número de contar. Está en $\mathbb{N}$ y, por la Propiedad 1.1, también en $\mathbb{Z}$, $\mathbb{Q}$ y $\mathbb{R}$.
2. **−3.** Es negativo, así que no está en $\mathbb{N}$. Es el opuesto de un natural: está en $\mathbb{Z}$ (y en $\mathbb{Q}$ y $\mathbb{R}$).
3. **0.** Con nuestro convenio, $0 \in \mathbb{N}$.
4. **2/5.** Es una fracción de enteros, así que está en $\mathbb{Q}$. ¿Es un entero disfrazado? 5 no divide a 2, y 2/5 = 0,4 no es entero. Está en $\mathbb{Q}$ y $\mathbb{R}$, **no** en $\mathbb{Z}$.
5. **−12/4.** ¡Primero simplificamos! $-12/4 = -3$, porque $4 \cdot (-3) = -12$. Es un **entero** negativo: está en $\mathbb{Z}$, $\mathbb{Q}$ y $\mathbb{R}$. Error típico: verlo escrito como fracción y decir que «no es entero».
6. **0,75.** Decimal exacto: 0,75 = 75/100 = 3/4 (dividiendo entre 25). Está en $\mathbb{Q}$, no en $\mathbb{Z}$.
7. **0,(6).** Tiene infinitas cifras, pero es **periódico**, así que es racional (Propiedad 1.6). Su fracción: x = 0,666…; 10x = 6,666…; restando, 9x = 6; x = 6/9 = 2/3. Está en $\mathbb{Q}$.
8. **√9.** $\sqrt{9} = 3$, porque $3 \cdot 3 = 9$ y $3 \geq 0$. Es natural.
9. **√5.** $2^2 = 4 < 5 < 9 = 3^2$, así que 5 no es un cuadrado perfecto. Por la Propiedad 1.7, $\sqrt{5}$ es **irracional** ($\sqrt{5} = 2{,}2360679\ldots$).
10. **π.** Irracional (Hecho aceptado 1.1).
11. **−√16.** $\sqrt{16} = 4$, luego $-\sqrt{16} = -4$: entero negativo.
12. **1,0100100010000…** Sus cifras siguen un patrón (bloques de ceros cada vez más largos), pero **ningún bloque fijo** se repite. Razonemos: si fuera periódico con un periodo de p cifras, como aparecen bloques de ceros tan largos como queramos, dentro de la parte periódica acabaría habiendo más de p ceros seguidos; entonces el periodo entero estaría formado solo por ceros y, a partir de ahí, no volvería a aparecer ningún 1. Pero siempre vuelve a aparecer un 1. Luego no es periódico: es infinito no periódico y, por tanto, **irracional**.

Resultado:

Número	Simplificado	$\mathbb{N}$	$\mathbb{Z}$	$\mathbb{Q}$	$\mathbb{I}$	$\mathbb{R}$
7	7	✓	✓	✓		✓
−3	−3		✓	✓		✓
0	0	✓	✓	✓		✓
2/5	0,4			✓		✓
−12/4	−3		✓	✓		✓
0,75	3/4			✓		✓

Número	Simplificado	$\mathbb{N}$	$\mathbb{Z}$	$\mathbb{Q}$	$\mathbb{I}$	$\mathbb{R}$
0,(6)	2/3			✓		✓
√9	3	✓	✓	✓		✓
√5	2,2360679…				✓	✓
π	3,1415926…				✓	✓
−√16	−4		✓	✓		✓
1,0100100010…	—				✓	✓

Comprobación por un segundo procedimiento (regla de la escalera). Por la Propiedad 1.1, en cada fila las marcas de $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ deben formar una «escalera»: si hay ✓ en $\mathbb{N}$, tiene que haberlo en $\mathbb{Z}$ y $\mathbb{Q}$; si hay ✓ en $\mathbb{Z}$, tiene que haberlo en $\mathbb{Q}$. Además, $\mathbb{Q}$ **e** $\mathbb{I}$ **nunca pueden estar marcados a la vez**, y $\mathbb{R}$ **siempre** está marcado. Revisa la tabla fila a fila: se cumple en todas. Si en tu tabla alguna fila rompe la escalera, hay un error seguro.

---

8. Ejemplo intermedio resuelto paso a paso

> **[EJEMPLO 1.2]** (a) Predice, **sin dividir**, qué tipo de decimal dan 7/40, 5/12 y 3/7.
>
> (b) Haz las divisiones y comprueba tu predicción. (c) Partiendo **solo de los decimales**, recupera las fracciones. (d) Comprueba que coinciden con las originales.

(a) Predicción

Paso 1: ¿son irreducibles? 7 y 40 no tienen divisores comunes (7 es primo y no divide a 40). 5 y 12 tampoco (5 es primo y no divide a 12). 3 y 7 son primos distintos. Las tres son irreducibles.

Paso 2: factorizar los denominadores y aplicar la Propiedad 1.5.

Fracción	Denominador	Factores	Predicción
7/40	40	$2 \cdot 2 \cdot 2 \cdot 5 = 2^3 \cdot 5$	Solo 2 y 5 → **exacto**
5/12	12	$2 \cdot 2 \cdot 3 = 2^2 \cdot 3$	Tiene 2 y además 3 → **periódico mixto**
3/7	7	7	Ni 2 ni 5 → **periódico puro**

(b) Divisiones (con todos los restos a la vista)

En cada fila: multiplicamos el resto anterior por 10 («bajamos un cero»), dividimos entre el denominador, anotamos la cifra y el nuevo resto. **Vigilar los restos** es la clave: cuando uno se repite, empieza la repetición.

7 : 40. Como 7 < 40, la parte entera es 0 y el primer resto es 7.

Paso	Resto × 10	: 40	Cifra	Nuevo resto
1	70	$70 = 1 \cdot 40 + 30$	1	30
2	300	$300 = 7 \cdot 40 + 20$	7	20
3	200	$200 = 5 \cdot 40 + 0$	5	**0** → termina

7/40 = **0,175**. Decimal exacto ✓.

5 : 12. Parte entera 0, primer resto 5.

Paso	Resto × 10	: 12	Cifra	Nuevo resto
1	50	$50 = 4 \cdot 12 + 2$	4	2
2	20	$20 = 1 \cdot 12 + 8$	1	8
3	80	$80 = 6 \cdot 12 + 8$	6	**8** ← ya había salido

El resto 8 se repite, así que la cifra 6 se repetirá para siempre: 5/12 = 0,41666… = **0,41(6)**. Periódico mixto: anteperiodo 41, periodo 6 ✓.

3 : 7. Parte entera 0, primer resto 3.

Paso	Resto × 10	: 7	Cifra	Nuevo resto
1	30	30 = 4 · 7 + 2	4	2
2	20	20 = 2 · 7 + 6	2	6
3	60	60 = 8 · 7 + 4	8	4
4	40	40 = 5 · 7 + 5	5	5
5	50	50 = 7 · 7 + 1	7	1
6	10	10 = 1 · 7 + 3	1	**3** ← ¡es el resto inicial!

Volvemos al resto 3, que era el de partida, así que todo se repite desde la primera cifra: 3/7 = **0,(428571)**. Periódico puro ✓. Su periodo tiene 6 cifras, el máximo permitido por la Propiedad 1.4 ($b - 1 = 6$).

👀 **Fíjate en los restos de esta última división: 3, 2, 6, 4, 5, 1.** Han aparecido **todos** los restos no nulos posibles al dividir entre 7. No es casualidad; volveremos a ello en el Ejemplo 1.3 y en el Módulo 14.

(c) Del decimal a la fracción (método de restar)

0,175 (exacto, 3 cifras decimales): 0,175 = 175/1000. Simplificamos: 175 = 5 · 5 · 7 y 1000 = 2 · 2 · 2 · 5 · 5 · 5. Dividimos entre 25: **175/1000 = 7/40**.

0,41(6) (periódico mixto). Llamamos x al número:

```
    x =   0,41666…
 100x =  41,666…      (multiplicamos por 100: la coma salta el anteperiodo "41")
1000x = 416,666…      (multiplicamos por 1000: la coma salta además un periodo "6")
```

Las dos últimas líneas tienen **las mismas cifras decimales** (666…), así que al restarlas desaparecen:

```
1000x − 100x = 416,666… − 41,666…
       900x = 375
          x = 375/900
```

Simplificamos: $375 = 3 \cdot 5^3$ y $900 = 2^2 \cdot 3^2 \cdot 5^2$. El máximo común divisor es $3 \cdot 5^2 = 75$. 375 : 75 = 5 y 900 : 75 = 12. Luego **x = 5/12**.

¿Por qué multiplicamos por 100 y por 1000? Porque queremos dos múltiplos de x cuyas partes decimales sean **idénticas**, para que al restar se cancelen. Eso ocurre cuando, en ambos, la coma queda justo antes del inicio de un periodo.

Comprobación con la regla F9: numerador = 416 − 41 = 375; denominador: un 9 (periodo de 1 cifra) y dos 0 (anteperiodo de 2 cifras) = 900. Sale 375/900 = 5/12 ✓. Ahora ves de dónde sale la regla: es exactamente la resta 1000x − 100x.

0,(428571) (periódico puro, 6 cifras de periodo):

```
      x =      0,428571428571…
1000000x = 428571,428571…
1000000x − x = 428571
    999999x = 428571
          x = 428571/999999
```

Para simplificar: 999999 = 3 · 3 · 3 · 7 · 11 · 13 · 37 y 428571 = 3 · 3 · 3 · 3 · 11 · 13 · 37, que comparten $3^3 \cdot 11 \cdot 13 \cdot 37 = 142857$. Dividiendo: 428571 : 142857 = 3 y 999999 : 142857 = 7. Luego **x = 3/7**.

(d) Comprobación

Hemos recuperado exactamente 7/40, 5/12 y 3/7 ✓. Además, una comprobación independiente rápida para el decimal exacto: 0,175 · 40 = 7 ✓ (multiplicar por el denominador debe devolver el numerador).

💡 **Para un decimal periódico largo**, factorizar 999999 a mano es pesado. Una forma más rápida de comprobar es **al revés**: dividir 3 entre 7 (como en (b)) y ver que sale 0,(428571). Las dos direcciones (fracción → decimal y decimal → fracción) se comprueban mutuamente.

---

9. Ejemplo avanzado aplicado a tu investigación

> **[EJEMPLO 1.3] ¿En qué «mundo de números» viven los códigos sobre GF(7)?**
>
> (a) ¿Por qué un programa de investigación **no** debe usar decimales del ordenador?
>
> (b) ¿Es el conjunto {0, 1, 2, 3, 4, 5, 6} cerrado para la suma y el producto habituales?
>
> (c) ¿Qué pasa si operamos «con reloj de 7»? ¿Y «con reloj de 6»?
>
> (d) ¿Qué conclusión sacamos para tu proyecto?

(a) El ordenador y los decimales

Ya sabes que 1/3 = 0,333… no se puede escribir en decimal con un número finito de cifras. Los ordenadores tienen el mismo problema, pero con **otros números**: internamente no trabajan en base 10 (con las cifras 0–9) sino en **base 2** (solo con las cifras 0 y 1; lo estudiarás en el capítulo 8.7). En base 2, el número 1/10 tiene una expresión **periódica**, igual que 1/3 la tiene en base 10. Como el ordenador solo guarda un número limitado de cifras, **recorta** el periodo y guarda una aproximación de 0,1 (Ampliación C).

Este pequeño programa lo muestra. Todavía no sabes Python; basta con que leas los comentarios (lo que va detrás de `#`).

Para qué sirve: comprobar con tus propios ojos que los decimales del ordenador (llamados `float`) no son exactos, que las fracciones exactas sí lo son, y probar las operaciones «con reloj de 7» del apartado (c).

```
# Archivo: codigo/ejemplos/cap_1_1_rincon_del_ordenador.py

print(0.1 + 0.2)              # suma con decimales "de ordenador" (float)
print(0.1 + 0.2 == 0.3)       # ¿es exactamente igual a 0.3?

from fractions import Fraction
print(Fraction(1, 10) + Fraction(2, 10))              # suma EXACTA de fracciones
print(Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10))

print((4 + 5) % 7)            # suma "con reloj de 7": resto de dividir 9 entre 7
print((3 * 5) % 7)            # producto "con reloj de 7": resto de dividir 15 entre 7
```

Entrada: ninguna (el programa no pide datos).
Salida obtenida al ejecutarlo realmente (Python 3.13.16, 12/10/2026):

```
0.30000000000000004
False
3/10
True
2
1
```

Por qué esa salida es correcta:

- Línea 1: el ordenador guarda aproximaciones de 0,1 y de 0,2; al sumarlas, los pequeños errores se notan en la cifra 17.ª: sale 0,30000000000000004 en lugar de 0,3.
- Línea 2: por eso la comparación «¿es igual a 0,3?» da `False` (falso). **Una investigación que compara decimales del ordenador con `==` puede sacar conclusiones falsas.**

- Líneas 3–4: `Fraction` guarda numerador y denominador como **enteros**, y con enteros Python es exacto: 1/10 + 2/10 = 3/10, y la comparación da `True` (verdadero).
- Líneas 5–6: `%` calcula el **resto** de una división. 9 = 1 · 7 + 2 → resto 2. 15 = 2 · 7 + 1 → resto 1. Esto es lo que usaremos en el apartado (c).

Pruebas: el programa `verificar_cap_1_1.py` comprueba automáticamente estas mismas salidas (y todos los cálculos del capítulo).

Limitaciones: las salidas de las líneas 1–2 dependen de cómo representa el ordenador los `float` (norma internacional IEEE 754, usada por prácticamente todos los ordenadores actuales). Las líneas 3–6 usan solo enteros y son exactas en cualquier ordenador.

Regla para todo el libro: en tu investigación sobre GF(7) **solo se usarán números enteros** (0, 1, 2, 3, 4, 5, 6 y los resultados de operar con ellos), nunca decimales del ordenador. Y **nunca** se dividirá con `/` en Python: en GF(7) dividir significa otra cosa, como verás ahora.

(b) ¿Es {0, 1, 2, 3, 4, 5, 6} cerrado con las operaciones de siempre?

Llamemos S = {0, 1, 2, 3, 4, 5, 6}. Para la **suma habitual**, un solo contraejemplo basta para ver que **no** es cerrado: 4 + 5 = 9 y 9 ∉ S. Para el **producto habitual**, tampoco: 3 · 5 = 15 ∉ S. Con las operaciones de siempre, S no sirve: los resultados «se escapan».

(c) Operar «con reloj de 7» y «con reloj de 6»

Recuerda los días de la semana: después del domingo vuelve el lunes. Definimos dos operaciones nuevas en S (los símbolos ⊕ y ⊗ son solo para distinguirlas de las de siempre):

- **a ⊕ b** = el **resto** de dividir a + b entre 7.
- **a ⊗ b** = el **resto** de dividir a · b entre 7.

Ejemplos: 4 ⊕ 5: 9 = 1 · 7 + **2**, luego 4 ⊕ 5 = 2. 6 ⊕ 6: 12 = 1 · 7 + **5**, luego 6 ⊕ 6 = 5. 3 ⊗ 5: 15 = 2 · 7 + **1**, luego 3 ⊗ 5 = 1.

Ahora S sí es cerrado: el resto de dividir entre 7 es **siempre** uno de los números 0, 1, …, 6. (Esto no es un ejemplo: es un razonamiento general, válido para cualesquiera a y b.)

La tabla completa de ⊗ («multiplicar con reloj de 7») es:

```
 ⊗ │ 0 1 2 3 4 5 6
───┼──────────────
 0 │ 0 0 0 0 0 0 0
 1 │ 0 1 2 3 4 5 6
 2 │ 0 2 4 6 1 3 5
 3 │ 0 3 6 2 5 1 4
 4 │ 0 4 1 5 2 6 3
 5 │ 0 5 3 1 6 4 2
 6 │ 0 6 5 4 3 2 1
```

Mira las filas 1 a 6 (sin la columna del 0): **cada una contiene los números 1, 2, 3, 4, 5, 6 exactamente una vez**, solo que en distinto orden. En particular, **en cada fila aparece el 1**. Eso significa que para cada número a ≠ 0 hay una «pareja» b con a ⊗ b = 1:

a	1	2	3	4	5	6
pareja b (a ⊗ b = 1)	1	4	5	2	3	6

Por ejemplo, 2 ⊗ 4 = 8 → resto 1. ¿Para qué sirve esto? Para **dividir**. En ℚ, dividir entre 2 es lo mismo que multiplicar por 1/2, que es el número que multiplicado por 2 da 1. En el mundo «reloj de 7», el número que multiplicado por 2 da 1 es el **4**. Así que «dividir entre 2» se hace «multiplicando por 4». Comprobación: si «6 entre 2» debe ser 3, entonces 6 ⊗ 4 = 24 → resto 3 ✓, y en efecto 3 ⊗ 2 = 6 ✓.

Comparemos con el reloj de 6. Ahora S′ = {0, 1, 2, 3, 4, 5} y operamos con restos de dividir entre 6:

```
 ⊗ │ 0 1 2 3 4 5       (reloj de 6)
───┼──────────────
 0 │ 0 0 0 0 0 0
 1 │ 0 1 2 3 4 5
 2 │ 0 2 4 0 2 4   ← nunca aparece el 1
 3 │ 0 3 0 3 0 3   ← nunca aparece el 1
 4 │ 0 4 2 0 4 2   ← nunca aparece el 1
 5 │ 0 5 4 3 2 1
```

Aquí pasan dos cosas «raras»: 2 ⊗ 3 = 6 → resto **0**: ¡dos números distintos de cero cuyo producto es cero! Y en las filas de 2, 3 y 4 **no aparece el 1**, así que no se puede «dividir» entre 2, 3 ni 4.

(d) Conclusión para tu proyecto

- **[VERIFICADO POR PROGRAMA]** Con el reloj de 7, el conjunto {0, …, 6} es cerrado para ⊕ y ⊗, y todo número distinto de 0 tiene pareja (por tanto, se puede dividir entre él). Con el reloj de 6, no. (Lo hemos comprobado a mano en las tablas, y `verificar_cap_1_1.py` lo comprueba también.)

- **[AVANCE — demostración pendiente: Módulos 13–14]** Lo que distingue al 7 del 6 es que **7 es primo** y 6 no. En general: con el reloj de n, todo número no nulo tiene pareja **exactamente cuando n es primo**. Investigarás esta afirmación tú mismo en el ejercicio C2.
- El mundo «reloj de 7» es, precisamente, **GF(7)**: un conjunto **finito** con el que se puede sumar, restar, multiplicar y dividir (entre números no nulos) igual que en $\mathbb{Q}$. Los códigos de tu investigación son listas de símbolos de GF(7), y para calcular sus parámetros habrá que resolver sistemas de ecuaciones, para lo cual **hace falta poder dividir**. Por eso se usa GF(7) y no el «reloj de 6».
- Y recuerda los restos de la división 3 : 7 del Ejemplo 1.2: 3, 2, 6, 4, 5, 1. Cada resto se obtiene del anterior multiplicando por 10 y tomando el resto entre 7, es decir, «multiplicando por 3 con reloj de 7» (porque $10 = 1 \cdot 7 + 3$). Que aparezcan todos los restos no nulos está relacionado con una propiedad del número 3 en GF(7) que estudiarás en el capítulo 14.6 (*raíces primitivas*).

---

10. Errores frecuentes

#	Error	Por qué está mal	Cómo corregirlo y detectarlo
E1	«−12/4 no es entero porque está escrito como fracción.»	La forma de escribir no decide el conjunto; decide el **valor**: $-12/4 = -3$.	**Simplifica siempre antes de clasificar.**
E2	«Todo número con infinitas cifras decimales es irracional.»	1/3 = 0,333… es racional. Lo que caracteriza a los irracionales es ser **infinitos y no periódicos**.	Pregúntate: ¿hay un bloque que se repite siempre?
E3	«π = 3,14» o «π = 22/7».	Son racionales; π es irracional. Son **aproximaciones**.	Escribe $\pi \approx 3{,}14$ (con ≈, «aproximadamente»).
E4	«0,(9) es un poco menor que 1.»	0,(9) = 1 exactamente (Observación 1.1).	Haz la resta $10x - x$.
E5	«√ de cualquier número es irracional.»	$\sqrt{9} = 3$, $\sqrt{1/4} = 1/2$.	Comprueba primero si es un cuadrado perfecto.
E6	Escribir «$2 \subset \mathbb{N}$».	$\subset$ es entre conjuntos. Lo correcto es $2 \in \mathbb{N}$ (o $\{2\} \subset \mathbb{N}$).	$\in$: elemento–conjunto; $\subset$: conjunto–conjunto.

#	Error	Por qué está mal	Cómo corregirlo y detectarlo
E7	«$-7 > -3$ porque $7 > 3$.»	Entre negativos, el más alejado del 0 es el menor: $-7 < -3$.	Dibuja la recta, o calcula la diferencia: $-3 - (-7) = 4 > 0$.
E8	Fracción generatriz de 0,41(6) = 416/999.	Esa regla es solo para periódicos **puros**; aquí hay anteperiodo.	Comprueba dividiendo: 416/999 = 0,(416), no 0,41(6).
E9	Decir que 21/56 da decimal periódico «porque 56 tiene un 7».	El criterio exige fracción **irreducible**: 21/56 = 3/8.	**Simplifica primero.**
E10	«He probado 10 ejemplos y funciona, así que es verdad siempre.»	Los ejemplos no demuestran afirmaciones sobre infinitos casos.	Busca una demostración o, al menos, etiquétalo como **conjetura**.
E11	Dividir entre cero, o creer que 0/5 no existe.	a/0 no está definido; 0/5 = 0.	Comprueba: «¿qué número multiplicado por el divisor da el dividendo?».
E12	Escribir `0,75` en Python.	Python usa punto decimal; `0,75` significa otra cosa (dos números separados).	En programas: `0.75`. En el cuaderno: 0,75.
E13	Comparar decimales del ordenador con `==`.	Los `float` son aproximaciones: `0.1 + 0.2 == 0.3` da `False`.	En GF(7), solo enteros. Si necesitas fracciones exactas: `Fraction`.
E14	Confundir «periodo de como mucho b − 1 cifras» con «periodo de exactamente b − 1 cifras».	Es una **cota** (un máximo): 1/13 tiene periodo 6, no 12.	Distingue «≤» de «=».

---

11. Resumen para memorizar

1. $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$; los irracionales están en $\mathbb{R}$ pero no en $\mathbb{Q}$. En este libro, $0 \in \mathbb{N}$.
2. Cada conjunto nace para que una operación deje de fallar: $\mathbb{Z}$ para restar, $\mathbb{Q}$ para dividir, $\mathbb{R}$ para medir ($\sqrt{2}$).
3. **Racional ⟺ decimal exacto o periódico. Irracional ⟺ decimal infinito no periódico.**

4. Fracción **irreducible** a/b: denominador solo con 2 y 5 → exacto; sin 2 ni 5 → periódico puro; mezcla → periódico mixto.
5. El periodo de a/b tiene **como mucho b − 1** cifras, porque solo hay b − 1 restos no nulos.
6. Fracción generatriz: multiplica por potencias de 10 para que coincidan las partes decimales y **resta**.
7. 0,(9) = 1. √n es natural o irracional. π es irracional.
8. Entre dos racionales siempre hay otro (punto medio).
9. **Un contraejemplo basta para refutar; ningún número de ejemplos basta para demostrar.**
10. El ordenador aproxima los decimales: **en GF(7) solo se usan enteros**.
11. [AVANCE] Con el «reloj de 7» se puede sumar, restar, multiplicar y dividir sin salir de {0, …, 6}: eso es GF(7).

---

12. Tabla de fórmulas y propiedades

Código	Enunciado	Condiciones	Estado
F1	$\mathbb{Q} = \{ a/b : a, b \in \mathbb{Z}, b \neq 0 \}$	—	Definición
F2	$a/b = c/d \iff a \cdot d = b \cdot c$	$b, d \neq 0$	Propiedad (se demuestra en 1.3)
F3	$a/b \pm c/d = (a \cdot d \pm c \cdot b)/(b \cdot d)$	$b, d \neq 0$	Propiedad
F4	$(a/b) \cdot (c/d) = (a \cdot c)/(b \cdot d)$	$b, d \neq 0$	Propiedad
F5	$(a/b) : (c/d) = (a \cdot d)/(b \cdot c)$	$b, c, d \neq 0$	Propiedad
F6	$a/b < c/d \iff a \cdot d < c \cdot b$	$b, d > 0$	Propiedad
F7	$a < (a + b)/2 < b$	$a < b$	Prop. 1.3, demostrada
F8	decimal exacto con m cifras = N/10^m	—	Prop. 1.6, demostrada
F9	generatriz = (sin coma hasta fin del 1.er periodo − sin coma hasta fin del anteperiodo) / (9…9 0…0)	decimal periódico	Prop. 1.6, demostrada
F10	longitud del periodo de $a/b \leq b - 1$	$b > 0$	Prop. 1.4, demostrada
P1.1	$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$	—	Demostrada

Código	Enunciado	Condiciones	Estado
P1.2	Cierre: $\mathbb{N}$ (+, ×); $\mathbb{Z}$ (+, −, ×); $\mathbb{Q}$ y $\mathbb{R}$ (+, −, ×, : entre no nulos)	—	Demostrada ($\mathbb{Q}$)
P1.5	Criterio del denominador	fracción irreducible	(i) demostrada; resto pendiente (2.3, 14.6)
T1.1	$\sqrt{2} \notin \mathbb{Q}$	—	Demostrado (Ampliación A)
P1.7	$\sqrt{n} \in \mathbb{N}$ o $\sqrt{n} \notin \mathbb{Q}$	$n \in \mathbb{N}$	Pendiente (10.7)
H1.1	$\pi \notin \mathbb{Q}$	—	Hecho aceptado
O1.1	0,(9) = 1	—	Demostrada

---

13. Ejercicios sin resolver

> **Instrucciones.** Hazlos en tu cuaderno, en orden. En cada uno, **escribe el razonamiento**, no solo el resultado. Cuando termines un bloque, compara con las soluciones (§14). Anota en tu registro (PROGRESO.md) los que falles y por qué.

Nivel A — Comprender y aplicar directamente las definiciones

A1. Clasifica en una tabla como la del Ejemplo 1.1 ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{I}$, $\mathbb{R}$):
a) -8 b) $15/3$ c) $-2/7$ d) 0,125 e) 0,(12) f) $\sqrt{25}$ g) $\sqrt{10}$ h) $-\sqrt{36}/3$ i) 3,141592 j) 0,123456789101112… (se escriben seguidos todos los naturales a partir del 1).

A2. Verdadero o falso. Justifica las verdaderas; da un contraejemplo de las falsas.
a) Todo número natural es entero.
b) Todo número entero es natural.
c) $-5 \in \mathbb{N}$.
d) $\mathbb{Z} \subset \mathbb{Q}$.
e) $0 \in \mathbb{Q}$.
f) Todo número con infinitas cifras decimales es irracional.
g) $\sqrt{2} \in \mathbb{Q}$.
h) Existe un número que es a la vez racional e irracional.

A3. Para cada fracción, **predice sin dividir** el tipo de decimal (exacto, periódico puro o mixto) y después **divide** para comprobarlo: 3/8; 7/9; 13/6; 9/25; 4/11.

A4. Halla la fracción generatriz irreducible y **comprueba dividiendo**:
a) 0,45 b) 0,(5) c) 2,(18) d) 0,3(12) e) 1,0(6).

A5. Ordena de menor a mayor y sitúa aproximadamente en una recta numérica:
$-2{,}5$; $7/4$; $-\sqrt{4}$; 0,(3); $-3/2$; 1,7.

A6. Decide si cada afirmación es verdadera o falsa. Si es falsa, da un contraejemplo; si es verdadera, explica por qué es verdad **en todos los casos**.
a) El producto de dos naturales es natural.
b) La resta de dos naturales es natural.
c) El cociente de dos enteros (con divisor no nulo) es entero.
d) La suma de dos racionales es racional.
e) La suma de dos irracionales es irracional.

Nivel B — Combinar conceptos y resolver problemas nuevos

B1. Encuentra un número racional entre 3/7 y 4/9. Después, encuentra tres más. ¿Cuántos racionales hay entre 3/7 y 4/9? Justifica.

B2. Sin dividir, decide cuáles de estas fracciones dan un decimal exacto: 21/56; 33/120; 9/14; 6/15; 35/42. (Cuidado: hay trampas.)

B3. (a) Demuestra de **dos maneras distintas** que 0,(9) = 1. (b) Halla la fracción generatriz de 0,4(9) y de 2,(9). (c) ¿Qué observas? ¿Qué consecuencia tiene sobre las expresiones decimales de un número?

B4. (a) Sin dividir, justifica que el periodo de 1/13 tiene como mucho 12 cifras. (b) Calcula 1/13 anotando todos los restos y halla la longitud real de su periodo. (c) ¿Contradice el resultado de (b) lo que dijiste en (a)? Explica la diferencia entre una **cota** y un **valor exacto**.

B5. Un compañero escribe: «He programado un ordenador que ha comprobado que ninguna fracción a/b con denominador b entre 1 y 1000 cumple $(a/b)^2 = 2$. Por tanto, $\sqrt{2}$ es irracional.»
(a) ¿Es verdadera su conclusión? (b) ¿Es válido su razonamiento? (c) ¿Qué ha conseguido realmente con su programa?

B6. Para cada ecuación, di cuál es el **conjunto más pequeño** ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ o $\mathbb{R}$) en el que tiene alguna solución, y cuál es esa solución:
a) $x + 9 = 4$ b) $3x = 5$ c) $5x = 35$ d) $x^2 = 9$ e) $x^2 = 7$ f) $2x + 3 = 0$.

Nivel C — Investigación: formular hipótesis, comprobar y justificar

C1. La longitud del periodo de 1/p.

1. Para cada primo $p \in \{3, 7, 11, 13, 17, 19\}$, calcula $1/p$ por división larga **anotando los restos**, y halla la longitud de su periodo. (Si te atreves, añade $p = 23$.)
2. Haz una tabla con p, $p - 1$ y la longitud del periodo.
3. Busca una relación entre la longitud del periodo y $p - 1$. **Formúlala como conjetura**, por escrito.
4. ¿Qué tipo de evidencia tienes: ejemplos, evidencia computacional o demostración?
5. ¿Qué tendrías que hacer para intentar **refutar** tu conjetura?

C2. Mundos de reloj.

1. Construye la tabla de multiplicar «con reloj de n» (multiplicar y quedarse con el resto de dividir entre n) para $n = 4, 5, 6$ y 8. (La de $n = 7$ ya la tienes en el Ejemplo 1.3.)
2. Para cada n, responde: ¿todo número distinto de 0 tiene una «pareja» b tal que $a \otimes b = 1$? Si no, ¿qué números no la tienen?
3. Formula una **conjetura**: ¿para qué valores de n todo número no nulo tiene pareja?
4. **Antes de calcular**, predice qué pasará con $n = 9$, 10 y 11. Después compruébalo.
5. Intenta explicar **por qué** falla cuando n no es primo. (Pista: si $n = a \cdot b$ con a y b entre 2 y $n - 1$, ¿cuánto vale $a \otimes b$?)

Problemas de aplicación práctica

P1. Ascensores y temperaturas.

a) Un ascensor sube de la planta -3 (garaje) a la planta 5. ¿Cuántas plantas ha subido? Después baja de la planta 4 a la -2: expresa ese cambio de planta con un número entero.

b) A las 6:00 un termómetro marca $-4{,}5$ °C y la temperatura sube 0,75 °C cada hora de forma constante. ¿A qué hora marcará 0 °C? ¿Qué temperatura marcará a las 14:00?

c) ¿Qué conjuntos de números has necesitado y por qué?

P2. Céntimos y ordenadores.

Una aplicación de una tienda suma precios.

a) Suma a mano 0,10 € diez veces.

b) Un programa que suma 0.1 diez veces, **una a una**, muestra `0.9999999999999999`. Explica por qué, usando lo aprendido en el Ejemplo 1.3.
c) Propón una forma de evitar el problema trabajando **solo con números enteros**, y úsala para sumar estos precios: 2,35 €; 0,99 €; 12,10 €; 0,56 €.
d) ¿Qué tiene que ver esto con tu investigación sobre GF(7)?

Problema combinado

Como este es el primer capítulo, este problema combina lo aprendido aquí con lo que ya sabes de cursos anteriores (división con resto y fracciones). En los capítulos siguientes, el problema combinado mezclará capítulos del libro.

PC. El calendario y los restos. Hoy es lunes.
a) ¿Qué día de la semana será dentro de 23 días?
b) ¿Y dentro de 100 días?
c) ¿Qué día de la semana fue hace 10 días? Resuélvelo de dos formas distintas.
d) Explica por qué, para responder, solo importa el **resto** de dividir entre 7.
e) Expresa 100 días en semanas como número decimal. ¿Qué relación hay entre la parte decimal y el resto del apartado b)?

---

14. Soluciones razonadas

> ⚠️ Intenta cada ejercicio **antes** de abrir su solución. Si te equivocas, no basta con leer la solución: identifica **qué concepto** fallaba (usa la columna «concepto» del §16) y haz un ejercicio parecido.

▼ Solución A1

	Número	Simplificado / razonamiento	ℕ	ℤ	ℚ	𝕀	ℝ
a	−8	entero negativo		✓	✓		✓
b	15/3	= 5, porque 3 · 5 = 15	✓	✓	✓		✓
c	−2/7	7 no divide a 2: no es entero			✓		✓
d	0,125	= 125/1000 = 1/8			✓		✓
e	0,(12)	= 12/99 = 4/33 (periódico)			✓		✓
f	√25	= 5, porque 5 · 5 = 25	✓	✓	✓		✓

	Número	Simplificado / razonamiento	$\mathbb{N}$	$\mathbb{Z}$	$\mathbb{Q}$	$\mathbb{I}$	$\mathbb{R}$
g	$\sqrt{10}$	$3^2 = 9 < 10 < 16 = 4^2$: no es cuadrado perfecto (Prop. 1.7)				✓	✓
h	$-\sqrt{36}/3$	$= -6/3 = -2$		✓	✓		✓
i	3,141592	decimal **exacto** = 3141592/1000000 = 392699/125000			✓		✓
j	0,123456789101112…	infinito no periódico (ver abajo)				✓	✓

Trampa de i): 3,141592 se parece a π, pero es un decimal exacto de 6 cifras, luego es racional. π tiene infinitas cifras.

Justificación de j): cuando escribimos 10, 100, 1000, 10000, … aparecen bloques de 1, 2, 3, 4, … ceros seguidos, tan largos como queramos. Si el decimal fuera periódico con un periodo de p cifras, al aparecer un bloque de más de p ceros dentro de la parte periódica, el periodo entero tendría que estar formado solo por ceros, y a partir de ahí todas las cifras serían 0. Pero siempre siguen apareciendo cifras distintas de 0 (cada número natural empieza por una cifra no nula). Contradicción: no es periódico. (Este número se llama *constante de Champernowne*.)

Comprobación: todas las filas cumplen la regla de la escalera del Ejemplo 1.1.

▾ Solución A2

a) **Verdadero.** Por definición, $\mathbb{Z}$ contiene a todos los naturales (Prop. 1.1).
b) **Falso.** Contraejemplo: $-1 \in \mathbb{Z}$ pero $-1 \notin \mathbb{N}$.
c) **Falso.** -5 es negativo; los naturales son 0, 1, 2, …
d) **Verdadero.** Todo entero n se escribe n/1, con denominador $1 \neq 0$.
e) **Verdadero.** $0 = 0/1$.
f) **Falso.** Contraejemplo: 1/3 = 0,333… tiene infinitas cifras y es racional.
g) **Falso.** Teorema 1.1: $\sqrt{2}$ es irracional.
h) **Falso.** Por la Definición 1.6, irracional significa «real que **no** es racional»: no se pueden cumplir las dos cosas a la vez.

▾ Solución A3

Todas son irreducibles (compruébalo: numerador y denominador no tienen divisores comunes).

Fracción	Denominador	Predicción	División (restos)	Resultado
3/8	2^3	exacto	30 = 3·8+6; 60 = 7·8+4; 40 = 5·8+0	0,375
7/9	3^2	periódico puro	70 = 7·9+7 → resto 7 se repite	0,(7)
13/6	2·3	periódico mixto	13 = 2·6+1; 10 = 1·6+4; 40 = 6·6+4 → se repite	2,1(6)
9/25	5^2	exacto	90 = 3·25+15; 150 = 6·25+0	0,36
4/11	11	periódico puro	40 = 3·11+7; 70 = 6·11+4 → resto 4 inicial	0,(36)

Segunda comprobación para los exactos: 3/8 = (3·125)/(8·125) = 375/1000 = 0,375 ✓; 9/25 = (9·4)/(25·4) = 36/100 = 0,36 ✓.

▼ Solución A4

a) **0,45** = 45/100 = **9/20** (dividiendo entre 5). Comprobación: 9/20 = 45/100 = 0,45 ✓.

b) x = 0,555…; 10x = 5,555…; 10x − x = 5 → 9x = 5 → **x = 5/9**. Comprobación: 50 = 5·9 + 5, el resto 5 se repite → 0,(5) ✓.

c) x = 2,1818…; 100x = 218,1818…; 100x − x = 216 → 99x = 216 → x = 216/99 = **24/11** (dividiendo entre 9). Con la regla F9: (218 − 2)/99 = 216/99 ✓. Comprobación: 24 = 2·11 + 2; 20 = 1·11 + 9; 90 = 8·11 + 2 → resto 2 repetido → 2,(18) ✓.

d) x = 0,31212…; 10x = 3,1212…; 1000x = 312,1212…; 1000x − 10x = 309 → 990x = 309 → x = 309/990 = **103/330** (dividiendo entre 3). Con la regla F9: (312 − 3)/990 = 309/990 ✓. Comprobación: 1030 = 3·330 + 40; 400 = 1·330 + 70; 700 = 2·330 + 40 → el resto 40 se repite → 0,3(12) ✓.

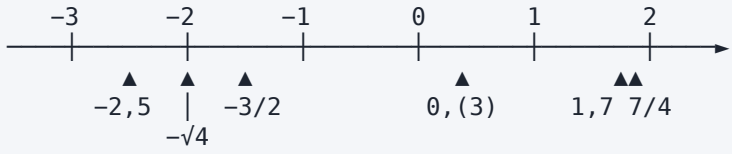
e) x = 1,0666…; 10x = 10,666…; 100x = 106,666…; 100x − 10x = 96 → 90x = 96 → x = 96/90 = **16/15** (dividiendo entre 6). Con la regla F9: (106 − 10)/90 = 96/90 ✓. Comprobación: 16 = 1·15 + 1; 10 = 0·15 + 10; 100 = 6·15 + 10 → se repite → 1,0(6) ✓.

▾ Solución A5

Pasamos todo a decimal: −2,5; 7/4 = 1,75; −√4 = −2; 0,(3) = 0,333...; −3/2 = −1,5; 1,7.

−2,5 < −√4 < −3/2 < 0,(3) < 1,7 < 7/4

Comprobación de las comparaciones delicadas: −2,5 < −2 porque −2 − (−2,5) = 0,5 > 0. 1,7 < 1,75 porque 1,75 − 1,7 = 0,05 > 0.



▾ Solución A6

a) **Verdadera.** Multiplicar naturales es sumar repetidamente naturales (3 · 4 = 4 + 4 + 4), y sumar naturales da un natural; además n · 0 = 0 ∈ ℕ.
b) **Falsa.** Contraejemplo: 2 − 5 = −3 ∉ ℕ.
c) **Falsa.** Contraejemplo: 1 : 2 = 1/2 ∉ ℤ.
d) **Verdadera** para **todos** los casos: es la Propiedad 1.2. a/b + c/d = (a·d + c·b)/(b·d), con numerador entero y denominador entero no nulo.
e) **Falsa.** Contraejemplo: √2 y −√2 son irracionales, pero √2 + (−√2) = 0 ∈ ℚ. ¿Por qué −√2 es irracional? Si fuera −√2 = a/b, entonces √2 = (−a)/b sería racional, lo que contradice el Teorema 1.1.

▾ Solución B1

Punto medio: (3/7 + 4/9)/2. Denominador común 63: 3/7 = 27/63 y 4/9 = 28/63. Suma: 55/63. Mitad: **55/126**.
Comprobación con denominador 126: 3/7 = 54/126 y 4/9 = 56/126, y 54 < 55 < 56 ✓.

Tres más: usamos un denominador mayor para «hacer sitio». Con 504 (= 63 · 8): 3/7 = 216/504 y 4/9 = 224/504. Entre 216 y 224 hay 217, 218, ..., 223. Por ejemplo: **217/504, 218/504 = 109/252, 219/504 = 73/168**.

¿Cuántos hay? Infinitos. Razonamiento: entre 3/7 y 55/126 hay otro punto medio; entre 3/7 y ese nuevo punto, otro; y así sin fin, obteniendo cada vez un número distinto (cada uno es menor que el anterior). Es la Propiedad 1.3

aplicada una y otra vez.

▼ Solución B2

Simplificar primero (Error E9):

Fracción	Irreducible	Denominador	¿Exacto?
21/56	3/8 (÷7)	2^3	**Sí**: 0,375
33/120	11/40 (÷3)	$2^3 \cdot 5$	**Sí**: 0,275
9/14	9/14	$2 \cdot 7$	No: periódico mixto, 0,6(428571)
6/15	2/5 (÷3)	5	**Sí**: 0,4
35/42	5/6 (÷7)	$2 \cdot 3$	No: periódico mixto, 0,8(3)

Las trampas son 21/56 (el 7 «desaparece» al simplificar) y 35/42 (parece tener un 7, pero lo que queda, 6, contiene un 3).

▼ Solución B3

(a) **Método 1 (restar):** x = 0,999…, 10x = 9,999…, 9x = 9, x = 1.
Método 2 (fracciones): 1/3 = 0,333…; multiplicando por 3: 1 = 0,999…
(También: con la regla F9, 0,(9) = 9/9 = 1.)

(b) x = 0,4999…; 10x = 4,999…; 100x = 49,999…; 90x = 45; **x = 1/2 = 0,5**.
x = 2,999…; 10x = 29,999…; 9x = 27; **x = 3**.

(c) Un decimal que termina en «(9)» es otra forma de escribir un decimal exacto: 0,4(9) = 0,5 y 2,(9) = 3. **Algunos números tienen dos expresiones decimales** (los decimales exactos distintos de 0). Por eso, por convenio, se usa la forma sin «(9)».

Curiosidad demostrable: la división larga **nunca** produce el periodo 9. Si un resto r produjera la cifra 9 y otra vez el mismo resto r, tendríamos $10 \cdot r = 9 \cdot b + r$, es decir, $9 \cdot r = 9 \cdot b$, o sea, r = b; pero un resto siempre es menor que el divisor b. Contradicción. **[VERIFICADO POR PROGRAMA]** para todas las fracciones con denominador hasta 300, aunque la demostración anterior ya lo prueba para todas.

▾ Solución B4

(a) Al dividir entre 13 los restos posibles son 0, 1, …, 12. Si en algún momento saliera el resto 0, la división terminaría y no habría periodo. Si no sale, solo quedan 12 restos posibles (1, …, 12) y, por la Propiedad 1.4, el periodo tiene **como mucho 12 cifras**.

(b)

Paso	Resto × 10	: 13	Cifra	Nuevo resto
1	10	10 = 0·13 + 10	0	10
2	100	100 = 7·13 + 9	7	9
3	90	90 = 6·13 + 12	6	12
4	120	120 = 9·13 + 3	9	3
5	30	30 = 2·13 + 4	2	4
6	40	40 = 3·13 + 1	3	**1** ← resto inicial

1/13 = **0,(076923)**: el periodo tiene **6 cifras**.

(c) **No hay contradicción.** En (a) obtuvimos una **cota**: un valor máximo posible («como mucho 12»). En (b) obtuvimos el **valor exacto**: 6, que es menor o igual que 12. Una cota no promete que se alcance. Esta distinción será fundamental en tu investigación: las «cotas» de la teoría de códigos (Singleton, Hamming, Griesmer… en el Módulo 21) dicen hasta dónde **podría** llegar la distancia mínima de un código, no que exista un código que llegue.

▾ Solución B5

(a) **Sí**, la conclusión es verdadera: √2 es irracional (Teorema 1.1).
(b) **No**, el razonamiento no es válido. Hay **infinitos** denominadores posibles; comprobar hasta 1000 no dice nada sobre b = 1001, b = 10^9 o cualquier otro. Sería como decir «he mirado 1000 cajones y no he encontrado la llave, así que la llave no existe».
(c) Ha conseguido **evidencia computacional** (un resultado verificado por programa en un número finito de casos), que hace la afirmación creíble, pero no la demuestra. La demostración de verdad es la de la Ampliación A, que funciona para **todos** los denominadores a la vez.

Conexión con tu investigación: si un programa examina un millón de palabras de un código y no encuentra ninguna de peso menor que 5, eso **no** demuestra que la distancia mínima sea 5, salvo que haya examinado **todas** las palabras (o que exista una demostración). Esta diferencia separa una evidencia de un récord demostrado (Módulos 25 y 28).

▾ Solución B6

a) $x = 4 - 9 = -5 \to \mathbb{Z}$ (no hay solución en $\mathbb{N}$).
b) $x = 5/3 \to \mathbb{Q}$ (3 no divide a 5: no hay solución entera).
c) $x = 35/5 = 7 \to \mathbb{N}$.
d) $x = 3$ o $x = -3$. Como $3 \in \mathbb{N} \to \mathbb{N}$.
e) $x = \sqrt{7}$ o $x = -\sqrt{7}$, irracionales (7 no es cuadrado perfecto) $\to \mathbb{R}$.
f) $2x = -3$, $x = -3/2 \to \mathbb{Q}$.

▾ Solución C1

Datos (1/13 ya lo calculaste en B4; 1/7 tiene un periodo de la misma longitud que el de 3/7, calculado en el Ejemplo 1.2):

p	p − 1	1/p	Longitud del periodo
3	2	0,(3)	1
7	6	0,(142857)	6
11	10	0,(09)	2
13	12	0,(076923)	6
17	16	0,(0588235294117647)	16
19	18	0,(052631578947368421)	18
23	22	0,(0434782608695652173913)	22

Conjetura esperada: «Para todo primo p distinto de 2 y de 5, la longitud del periodo de 1/p **divide** a p − 1.» (1 divide a 2; 6 divide a 6; 2 divide a 10; 6 divide a 12; 16 divide a 16…). Observación adicional: a veces la longitud es **exactamente** p − 1 (7, 17, 19, 23) y a veces es más pequeña (3, 11, 13).

Tipo de evidencia: 6 o 7 ejemplos calculados a mano (y el programa de verificación los confirma). Es **evidencia**, no demostración.

Cómo intentar refutarla: calcular más primos (29, 31, 37, 41…), buscando uno cuya longitud de periodo **no** divida a p − 1. Un solo caso así la refutaría.

Estado real: esta conjetura es **verdadera**: es un teorema que se deduce del *pequeño teorema de Fermat*, que demostrarás en el capítulo 14.5. Que la hayas descubierto a partir de datos es exactamente cómo trabaja un investigador: **datos → patrón → conjetura → demostración**. Y fíjate en que no sabíamos que era verdadera hasta tener la demostración: muchos patrones que se cumplen en los primeros casos fallan más adelante (verás ejemplos célebres en el capítulo 10.10).

▾ Solución C2

Tablas (solo filas y columnas 1 a n − 1, porque el 0 nunca tiene pareja: 0 ⊗ b = 0):

```
n = 4          n = 5          n = 6            n = 8
⊗ | 1 2 3      ⊗ | 1 2 3 4    ⊗ | 1 2 3 4 5    ⊗ | 1 2 3 4 5 6 7
--|-------     --|---------    --|-----------    --|---------------
1 | 1 2 3      1 | 1 2 3 4    1 | 1 2 3 4 5    1 | 1 2 3 4 5 6 7
2 | 2 0 2      2 | 2 4 1 3    2 | 2 4 0 2 4    2 | 2 4 6 0 2 4 6
3 | 3 2 1      3 | 3 1 4 2    3 | 3 0 3 0 3    3 | 3 6 1 4 7 2 5
               4 | 4 3 2 1    4 | 4 2 0 4 2    4 | 4 0 4 0 4 0 4
                              5 | 5 4 3 2 1    5 | 5 2 7 4 1 6 3
                                               6 | 6 4 2 0 6 4 2
                                               7 | 7 6 5 4 3 2 1
```

¿Quién no tiene pareja?

- n = 4: el **2** (su fila es 2, 0, 2: no aparece el 1). Parejas: 1↔1, 3↔3.
- n = 5: **todos tienen pareja**: 1↔1, 2↔3, 4↔4.
- n = 6: **2, 3 y 4** no tienen pareja. Parejas: 1↔1, 5↔5.
- n = 7: **todos tienen pareja** (Ejemplo 1.3).
- n = 8: **2, 4 y 6** no tienen pareja. Parejas: 1↔1, 3↔3, 5↔5, 7↔7.

Conjetura: «Todo número no nulo tiene pareja con el reloj de n **exactamente cuando n es primo**.»

Predicción y comprobación: 9 y 10 no son primos → predicción: fallará; 11 es primo → predicción: funcionará.

- n = 9: el 3 y el 6 no tienen pareja (3·1 = 3, 3·2 = 6, 3·3 = 9 → 0, … los múltiplos de 3 nunca dejan resto 1 al dividir entre 9). ✓ falla.
- n = 10: no tienen pareja 2, 4, 5, 6 y 8. ✓ falla.
- n = 11: todos tienen pareja: 1↔1, 2↔6, 3↔4, 5↔9, 7↔8, 10↔10. (Por ejemplo, 5 · 9 = 45 = 4 · 11 + 1.) ✓ funciona.

[VERIFICADO POR PROGRAMA] Para n de 2 a 11, los valores en los que todos tienen pareja son exactamente 2, 3, 5, 7 y 11, los primos.

¿Por qué falla si n no es primo? Si n = a · b con a y b entre 2 y n − 1, entonces a ⊗ b = 0 (el resto de dividir n entre n es 0). Supongamos que a tuviera pareja c, con a ⊗ c = 1. Entonces, operando con el reloj de n:
b = b ⊗ 1 = b ⊗ (a ⊗ c) = (b ⊗ a) ⊗ c = 0 ⊗ c = 0,
pero b está entre 2 y n − 1, así que no es 0. **Contradicción:** a no puede tener pareja. (Este argumento usa que ⊗ se puede reagrupar y cambiar de orden igual que la multiplicación normal; lo justificaremos en el capítulo 2.8.)

La otra mitad (si n es primo, todos tienen pareja) necesita la *identidad de Bézout* (capítulo 2.7) y se demostrará en el Módulo 13. Por ahora, es una **conjetura bien apoyada** para ti, aunque sea un teorema conocido.

▾ Solución P1

a) De −3 a 5: 5 − (−3) = 5 + 3 = **8 plantas**. De 4 a −2: el cambio es −2 − 4 = **−6** (6 plantas hacia abajo).

b) Le faltan 4,5 °C para llegar a 0. A 0,75 °C por hora: 4,5 : 0,75 = 450 : 75 = **6 horas** → a las **12:00**.
A las 14:00 han pasado 8 horas: −4,5 + 8 · 0,75 = −4,5 + 6 = **1,5 °C**.
Comprobación: a las 12:00 está a 0 °C y en 2 horas más sube 2 · 0,75 = 1,5 °C ✓.

c) ℤ para las plantas bajo tierra y las temperaturas bajo cero (restas como 5 − 8 sin solución en ℕ); ℚ (decimales exactos) para 4,5 y 0,75; ℕ para contar horas.

▾ Solución P2

a) 10 · 0,10 = **1,00 €**.

b) El ordenador no guarda 0,1 exactamente, sino una aproximación (en base 2, 1/10 es periódico y el periodo se recorta). Cada suma acumula un error diminuto, y al final se obtiene 0,9999999999999999 en vez de 1.
[VERIFICADO POR PROGRAMA] con Python 3.13.16.

⚠️ **Un detalle que enseña mucho:** si en vez de sumar una a una se usa la función `sum([0.1] * 10)`, en Python 3.12 o posterior el resultado es `1.0` (comprobado aquí con Python 3.13.16), porque desde la versión 3.12 `sum` usa

un método de suma más preciso; en versiones anteriores daba `0.9999999999999999`. **El mismo cálculo puede dar resultados distintos según la versión del programa**: por eso, en el Módulo 9, aprenderás a anotar siempre la versión de Python con la que obtienes cada resultado. Pero el problema de fondo no desaparece: `0.1 + 0.2 == 0.3` sigue siendo `False`.

c) **Trabajar en céntimos**, que son números enteros: 2,35 € = 235 céntimos; 0,99 € = 99; 12,10 € = 1210; 0,56 € = 56. 235 + 99 + 1210 + 56 = **1600 céntimos = 16,00 €**. Con enteros, el ordenador es exacto. Comprobación sumando en euros: 2,35 + 0,99 = 3,34; 3,34 + 12,10 = 15,44; 15,44 + 0,56 = 16,00 ✓.

d) Es la misma idea: en GF(7) **todo son enteros** (0, …, 6) y los resultados se obtienen con restos de divisiones enteras, así que el ordenador calcula **sin errores de redondeo**. Por eso los cálculos de códigos sobre GF(7) pueden ser exactos y verificables.

▾ Solución PC

Numeramos: lunes = 0, martes = 1, miércoles = 2, jueves = 3, viernes = 4, sábado = 5, domingo = 6.

a) 23 = 3 · 7 + **2**. Tres semanas completas nos devuelven al lunes; faltan 2 días: **miércoles**.

b) 100 = 14 · 7 + **2** → **miércoles** (de nuevo resto 2).

c) **Forma 1 (hacia atrás):** 10 = 7 + 3. Retroceder 7 días nos deja en lunes; retroceder 3 más: domingo, sábado, **viernes**. **Forma 2 (hacia delante):** retroceder 10 días y luego avanzar 14 (dos semanas) nos deja en el mismo día de la semana que avanzar 4 días desde hoy. Lunes + 4 = **viernes** ✓. (Fíjate: −10 = −2 · 7 + 4. El «resto» de −10 entre 7, tomado entre 0 y 6, es 4. Los restos de números negativos se estudian en el capítulo 2.5.)

d) Cada bloque de 7 días nos devuelve exactamente al mismo día de la semana, así que los bloques completos (el cociente) no cambian nada: solo importan los días «sobrantes», es decir, el **resto**.

e) 100/7 = 14,(285714) semanas. La parte decimal es 0,(285714) = **2/7** de semana, es decir, 2 días: ¡exactamente el resto del apartado b! Comprobación: 2/7 = 0,(285714), ya que 20 = 2·7 + 6; 60 = 8·7 + 4; 40 = 5·7 + 5; 50 = 7·7 + 1; 10 = 1·7 + 3; 30 = 4·7 + 2 → resto 2 inicial. En general: a/7 = cociente + resto/7.

---

15. Preguntas de comprensión

Responde con tus palabras, en 2–5 líneas cada una. No son cálculos: son para comprobar que **entiendes**. Las respuestas modelo están en el anexo final.

1. ¿Por qué se dice que $\mathbb{Q}$ «resuelve» todas las divisiones (entre números no nulos) pero no todas las raíces cuadradas?
2. Si un número tiene infinitas cifras decimales, ¿es necesariamente irracional? Explícalo con un ejemplo.
3. ¿Qué diferencia hay entre «$2 \in \mathbb{N}$» y «$\{2\} \subset \mathbb{N}$»? ¿Son verdaderas las dos?
4. Explica, sin hacer la división, por qué 1/7 tiene que acabar repitiéndose.
5. ¿Por qué un solo contraejemplo basta para demostrar que una afirmación «para todo…» es falsa, pero mil ejemplos no bastan para demostrar que es verdadera?
6. ¿Por qué en la investigación con GF(7) no conviene usar los decimales del ordenador?
7. Un compañero dice: «0,(9) y 1 son números distintos porque se escriben distinto». ¿Qué le responderías?
8. ¿Qué tiene de especial el «reloj de 7» frente al «reloj de 6»?

---

16. Prueba de dominio

Instrucciones: sin mirar el capítulo ni las soluciones. Tiempo recomendado: 40 minutos. Cada pregunta vale 1 punto. Las preguntas marcadas con ★ son **críticas**.

Criterio para dar el capítulo por dominado: 8 puntos o más y **todas las preguntas ★ correctas**. Si no lo consigues, no pasa nada: usa la tabla de diagnóstico, haz los ejercicios de refuerzo y repite la prueba otro día (preferiblemente pidiendo a tu tutor una versión con otros números).

D1 ★. Clasifica ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{I}$): $-7/7$; $\sqrt{49}/2$; $0{,}1(23)$; $\sqrt{3}$; $-0{,}5$.

D2 ★. Halla la fracción generatriz irreducible de 0,2(7) y comprueba el resultado dividiendo.

D3. Sin dividir, di qué tipo de decimal dan: 7/20; 5/6; 2/9; 15/48.

D4 ★. Da un contraejemplo de: «La diferencia de dos números naturales es siempre un número natural».

D5. Encuentra un número racional entre $-1/3$ y $-1/4$.

D6. Ordena de menor a mayor: 0,(6); 2/3; 0,67; 0,6.

D7. Verdadero o falso, con justificación: «Todo decimal exacto es un número racional».

D8 ★. Explica en tres líneas como máximo por qué un ordenador responde `False` a `0.1 + 0.2 == 0.3`, y qué consecuencia tiene para tus cálculos con GF(7).

D9. Para cada ecuación, di el conjunto más pequeño ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ o $\mathbb{R}$) en el que tiene solución: $4x = -6$; $x + 10 = 3$; $x^2 = 5$.

D10. ¿Puede el periodo de a/17 tener 20 cifras? Justifica tu respuesta sin hacer ninguna división.

Tabla de diagnóstico

Si fallas una pregunta, el problema está en el concepto indicado. Vuelve a la sección, haz el ejercicio de refuerzo y explica en voz alta (o por escrito) en qué consistía tu error.

Pregunta	Concepto evaluado	Repasa	Refuerzo
D1	Clasificar números (simplificar primero)	§4.2, Ejemplo 1.1, §5.2	RF1
D2	Fracción generatriz de un periódico mixto	§4.3, Prop. 1.6, Ejemplo 1.2 (c), F9	RF2
D3	Criterio del denominador (fracción irreducible)	Prop. 1.5, §5.3	RF3
D4	Operación cerrada y contraejemplo	§4.5, Prop. 1.2	RF4
D5	Densidad de $\mathbb{Q}$ (punto medio)	Prop. 1.3, F7	B1
D6	Orden y equivalencia 0,(6) = 2/3	§4.4, Ejemplo 1.1 (n.º 7)	RF5
D7	Decimal exacto $\Longrightarrow$ racional	Prop. 1.6, F8	A4 a)

Pregunta	Concepto evaluado	Repasa	Refuerzo
D8	Exactitud en el ordenador	Ejemplo 1.3 (a), Ampliación C	P2
D9	Por qué existe cada conjunto	§3.1	B6
D10	Cota del periodo (argumento de los restos)	Prop. 1.4, F10	B4

Ejercicios de refuerzo

RF1. Clasifica: $-20/5$; $0{,}0(5)$; $\sqrt{50}$; $\sqrt{0{,}25}$; $3/1$.
RF2. Fracción generatriz y comprobación: $0{,}1(6)$; $3{,}(7)$; $0{,}(142857)$.
RF3. Sin dividir, predice el tipo de decimal y después compruébalo dividiendo: $11/16$; $7/15$; $5/33$.
RF4. Da un contraejemplo de cada afirmación: (a) «El cociente de dos naturales (divisor no nulo) es natural.» (b) «El producto de dos irracionales es irracional.» (c) «Todo número negativo es entero.»
RF5. Ordena de menor a mayor: $-0{,}(3)$; $-1/4$; $-0{,}3$; $-0{,}33$.

(Soluciones en el anexo final.)

---

17. Relación con el siguiente capítulo

En este capítulo has visto **qué números existen** y **por qué**. En el **capítulo 1.2 (Números enteros: signos, valor absoluto, orden y jerarquía de operaciones)** aprenderás a **operar** con enteros sin errores: reglas de los signos, paréntesis, prioridad de las operaciones y valor absoluto.

¿Por qué es imprescindible para tu objetivo? Porque toda la aritmética de GF(7) se construye sobre la aritmética de los **enteros**: en el Ejemplo 1.3 y en el problema del calendario ya has visto que el «reloj de 7» consiste en operar con enteros y quedarse con un resto. Y cuando aparezcan restas como $2 - 5$ en GF(7), o restos de números negativos (como en el apartado c del problema del calendario), necesitarás dominar los enteros con total seguridad.

A partir del capítulo 1.2, cada capítulo incluirá una sección **«Recupera»** con problemas nuevos sobre capítulos anteriores, para que lo aprendido aquí no se olvide.

Antes de pasar al capítulo 1.2:

- [ ] He hecho todos los ejercicios de nivel A y, al menos, B1, B2, B4 y B5.
- [ ] He intentado C1 y C2 (no es obligatorio acertarlos para avanzar, pero sí intentarlos).
- [ ] He respondido las 8 preguntas de comprensión y las he comparado con las respuestas modelo.
- [ ] He superado la prueba de dominio (≥ 8 puntos y todas las ★).
- [ ] He anotado mis errores en `PROGRESO.md`.

Ampliaciones (opcionales en una primera lectura)

Ampliación A — Demostración de que √2 es irracional (Teorema 1.1)

Idea del método (reducción al absurdo). Suponemos lo **contrario** de lo que queremos demostrar y deducimos, paso a paso, algo imposible. Si de una suposición se deduce algo imposible, la suposición era falsa. (Lo estudiarás con todo detalle en el capítulo 10.7.)

Un hecho que necesitamos: el cuadrado de un número impar es impar.

- *Con las últimas cifras:* un impar termina en 1, 3, 5, 7 o 9; su cuadrado termina, respectivamente, en 1, 9, 5, 9 o 1 (porque 1·1 = 1, 3·3 = 9, 5·5 = 25, 7·7 = 49, 9·9 = 81, y la última cifra de un producto solo depende de las últimas cifras de los factores). Todas son impares.
- *Con álgebra* (usa una identidad notable que repasarás en el capítulo 3.4): un impar es 2m + 1; su cuadrado es $(2m + 1)^2 = 4m^2 + 4m + 1 = 2\cdot(2m^2 + 2m) + 1$, que es impar.

En consecuencia: **si a^2 es par, entonces a es par** (porque si a fuera impar, a^2 sería impar).

Demostración.

1. **Suponemos lo contrario:** √2 es racional. Entonces √2 = a/b con a y b naturales, b ≠ 0, y podemos tomar la fracción **irreducible** (a y b sin divisores comunes mayores que 1).
2. Elevando al cuadrado: $2 = a^2/b^2$, es decir, $\mathbf{a^2 = 2b^2}$.
3. Entonces a^2 es par (es 2 por algo). Por el hecho anterior, **a es par**: a = 2c para algún natural c.
4. Sustituyendo: $(2c)^2 = 2b^2 \rightarrow 4c^2 = 2b^2 \rightarrow \mathbf{b^2 = 2c^2}$.
5. Entonces b^2 es par y, por el mismo hecho, **b es par**.

6. Pero si a y b son pares, ambos son divisibles entre 2, y la fracción a/b **no** era irreducible. Esto contradice el paso 1. ∎

Como la suposición «√2 es racional» lleva a una contradicción, es falsa: **√2 es irracional**. Observa que esta demostración vale para **todas** las fracciones a la vez, cosa que ningún programa de ordenador puede hacer probando casos (ejercicio B5).

Ampliación B — ¿Qué es realmente ℝ?

La Definición 1.6 («los números que se escriben con un decimal») es intuitiva, pero esconde preguntas difíciles: ¿cómo se suman dos decimales infinitos, si no hay «última cifra» por la que empezar? ¿Por qué la recta numérica no tiene «agujeros»? En el siglo XIX, matemáticos como Richard Dedekind y Georg Cantor construyeron ℝ con rigor a partir de ℚ. Esa construcción se estudia en el primer curso universitario de Análisis Matemático.

Para tu investigación no la necesitarás. Los códigos sobre GF(7) viven en conjuntos **finitos**. Los números reales aparecerán solo de forma auxiliar: al comparar tamaños (logaritmos, capítulo 4.5) y en algunas cotas asintóticas (Módulo 21).

Ampliación C — ¿Por qué 0,1 no es exacto en el ordenador?

En base 10, una fracción irreducible da un decimal exacto solo si su denominador tiene únicamente los factores primos de 10, que son 2 y 5 (Propiedad 1.5). En **base 2** pasa lo mismo, pero con los factores primos de 2, que es solo el 2: una fracción da una expresión **exacta en base 2** solo si su denominador es una potencia de 2 (2, 4, 8, 16, …).

Como 1/10 tiene denominador 10 = 2 · 5, y el 5 no es potencia de 2, **en base 2 es periódico**. Haciendo la «división larga en base 2» (multiplicando el resto por 2 en lugar de por 10) se obtiene:

1/10 = 0,0(0011) en base 2 = 0,0001100110011… en base 2.

[VERIFICADO POR PROGRAMA] `verificar_cap_1_1.py` calcula este desarrollo con la misma función que usa para la base 10, cambiando solo la base.

El ordenador guarda solo 53 cifras binarias significativas (en el formato habitual de los `float`), así que recorta el periodo y almacena un número muy próximo a 0,1, pero no igual. Lo estudiarás en el capítulo 8.7.

Anexo — Respuestas de comprensión, prueba de dominio y refuerzo

> ⚠️ No abras este anexo hasta haber hecho la prueba de dominio completa.

▼ Respuestas modelo de las preguntas de comprensión (§15)

1. Porque $\mathbb{Q}$ es cerrado para la división (Prop. 1.2): el cociente de dos fracciones (divisor no nulo) es otra fracción. Pero hay raíces, como $\sqrt{2}$, cuyo valor no es ninguna fracción (Teorema 1.1). Para tenerlas hace falta ampliar hasta $\mathbb{R}$.
2. No. 1/3 = 0,333… tiene infinitas cifras y es racional. Lo que caracteriza a los irracionales es ser infinitos **y no periódicos**.
3. «$2 \in \mathbb{N}$» dice que el **número** 2 es un elemento de $\mathbb{N}$. «$\{2\} \subset \mathbb{N}$» dice que el **conjunto** cuyo único elemento es el 2 está contenido en $\mathbb{N}$. Las dos son verdaderas, pero relacionan objetos de distinto tipo.
4. En la división larga entre 7, los restos solo pueden ser 0, 1, …, 6. Si saliera el 0, la división terminaría; si no sale, solo hay 6 restos posibles (1, …, 6), y en cuanto se repite uno, todo lo que viene después se repite, porque cada paso depende solo del resto anterior.
5. Una afirmación «para todo» dice que **no hay ninguna** excepción: una sola excepción la hace falsa. Pero los casos posibles suelen ser infinitos, y por muchos que compruebes siempre quedan infinitos sin comprobar; solo un razonamiento general cubre todos los casos a la vez.
6. Porque los decimales del ordenador (`float`) son aproximaciones y pueden dar resultados falsos ($0.1 + 0.2 \neq 0.3$). En GF(7) todo son enteros y restos, con los que el ordenador es exacto; así los resultados son verificables.
7. Que un mismo número puede tener varias escrituras (como 1/2 y 2/4). La resta $10x - x$ con $x = 0{,}999\ldots$ da $9x = 9$, $x = 1$. Además, intuitivamente: si fueran distintos, habría algún número entre ellos (su punto medio), y no hay ninguna expresión decimal que quede entre 0,999… y 1.
8. Con el reloj de 7, todo número distinto de 0 tiene pareja (un número que multiplicado por él da 1), así que se puede dividir entre cualquier número no nulo, y el producto de dos números no nulos nunca es 0. Con el reloj de 6, $2 \otimes 3 = 0$ y el 2, el 3 y el 4 no tienen pareja. La diferencia es que 7 es primo y 6 no.

▾ Soluciones de la prueba de dominio (§16)

D1 ★. $-7/7 = -1 \to \mathbb{Z}$ (y $\mathbb{Q}$, $\mathbb{R}$; no $\mathbb{N}$). $\sqrt{49}/2 = 7/2 \to \mathbb{Q}$ (no $\mathbb{Z}$). 0,1(23) → periódico → $\mathbb{Q}$. $\sqrt{3}$ → 3 no es cuadrado perfecto → $\mathbb{I}$. $-0{,}5 = -1/2 \to \mathbb{Q}$ (no $\mathbb{Z}$).

D2 ★. $x = 0{,}2777\ldots$; $10x = 2{,}777\ldots$; $100x = 27{,}777\ldots$; $100x - 10x = 25 \to 90x = 25 \to x = 25/90 =$ **5/18**. (Regla F9: $(27 - 2)/90 = 25/90$ ✓.)
Comprobación: $50 = 2\cdot18 + 14$; $140 = 7\cdot18 + 14$ → el resto 14 se repite → 0,2(7) ✓.

D3. 7/20: $20 = 2^2\cdot5$ → **exacto** (0,35). 5/6: $6 = 2\cdot3$ → **periódico mixto** (0,8(3)). 2/9: $9 = 3^2$ → **periódico puro** (0,(2)). 15/48 = 5/16 (¡simplificar!): $16 = 2^4$ → **exacto** (0,3125).

D4 ★. Por ejemplo, $3 - 7 = -4 \notin \mathbb{N}$. (Cualquier resta de un natural menos otro mayor sirve.)

D5. Con denominador 24: $-1/3 = -8/24$ y $-1/4 = -6/24$. Entre ellos: **−7/24** (también vale el punto medio, que es justamente $-7/24$). Comprobación: $-8/24 < -7/24 < -6/24$ ✓.

D6. 0,(6) = 2/3 = 0,666… Entonces **0,6 < 0,(6) = 2/3 < 0,67**. (0,6 < 0,666… porque la segunda cifra decimal 0 < 6; 0,666… < 0,67 porque la segunda cifra decimal 6 < 7.)

D7. Verdadero. Un decimal exacto con m cifras decimales es N/10^m (por ejemplo, 2,35 = 235/100), que es una fracción de enteros con denominador no nulo (Prop. 1.6).

D8 ★. Respuesta modelo: «El ordenador guarda los decimales en base 2, donde 0,1 y 0,2 son periódicos, así que solo guarda aproximaciones; al sumar, el pequeño error hace que el resultado no sea exactamente 0,3. En GF(7) hay que usar solo enteros (y restos), con los que el ordenador es exacto.» Se considera correcta si menciona **aproximación/redondeo** y **usar enteros**.

D9. $4x = -6 \to x = -3/2 \to \mathbb{Q}$. $x + 10 = 3 \to x = -7 \to \mathbb{Z}$. $x^2 = 5 \to x = \pm\sqrt{5} \to \mathbb{R}$.

D10. No. Al dividir entre 17 hay como mucho 16 restos no nulos (1, …, 16), y en cuanto uno se repite empieza el periodo. Por tanto, el periodo tiene **como mucho 16 cifras** (Prop. 1.4). (De hecho, para 1/17 tiene exactamente 16.)

▾ Soluciones de los ejercicios de refuerzo

RF1. $-20/5 = -4 \to \mathbb{Z}$. 0,0(5) → periódico → $\mathbb{Q}$ (es 1/18). $\sqrt{50}$ → $49 < 50 < 64$, no es cuadrado perfecto → $\mathbb{I}$. $\sqrt{0{,}25} = 0{,}5$, porque $0{,}5 \cdot 0{,}5 = 0{,}25 \to \mathbb{Q}$ (no $\mathbb{Z}$). $3/1 = 3 \to \mathbb{N}$.

RF2.

- 0,1(6): $10x = 1{,}666\ldots$; $100x = 16{,}666\ldots$; $90x = 15$; $x = 15/90 =$ **1/6**. Comprobación: $10 = 1 \cdot 6 + 4$; $40 = 6 \cdot 6 + 4 \to 0{,}1(6)$ ✓.
- 3,(7): $x = 3{,}777\ldots$; $10x = 37{,}777\ldots$; $9x = 34$; $x =$ **34/9**. Comprobación: $34 = 3 \cdot 9 + 7$; $70 = 7 \cdot 9 + 7 \to 3{,}(7)$ ✓.
- 0,(142857): $x = 142857/999999 =$ **1/7**, porque $999999 = 7 \cdot 142857$. Comprobación: es el desarrollo de 1/7 (ver tabla de C1) ✓.

RF3. 11/16: $16 = 2^4 \to$ exacto: **0,6875**. 7/15: $15 = 3 \cdot 5 \to$ mixto: **0,4(6)**. 5/33: $33 = 3 \cdot 11 \to$ puro: **0,(15)**.

RF4. (a) $3 : 2 = 3/2 \notin \mathbb{N}$. (b) $\sqrt{2} \cdot \sqrt{2} = 2 \in \mathbb{Q}$. (c) $-1/2$ es negativo y no es entero.

RF5. En decimal: −0,333…; −0,25; −0,3; −0,33. Entre negativos, el que está más lejos del 0 es el menor: **−0,(3) < −0,33 < −0,3 < −1/4**.

Referencias del capítulo

- G. H. Hardy y E. M. Wright, *An Introduction to the Theory of Numbers*, Oxford University Press (6.ª ed., revisada por D. R. Heath-Brown y J. H. Silverman, 2008). Contiene la demostración de la irracionalidad de √2 y un estudio completo de las expresiones decimales de los racionales (longitud del periodo). *Referencia clásica; datos bibliográficos citados de memoria, no verificados en línea en esta sesión.*
- D. Goldberg, «What Every Computer Scientist Should Know About Floating-Point Arithmetic», *ACM Computing Surveys*, 23(1), 1991, pp. 5–48. Explicación detallada de los `float`. *Referencia clásica; no verificada en línea en esta sesión.*
- Documentación oficial de Python, «Floating-Point Arithmetic: Issues and Limitations»: https://docs.python.org/3/tutorial/floatingpoint.html — *el enlace no se pudo abrir desde el entorno de redacción; la página existe en la*

documentación oficial, pero comprueba el enlace tú mismo.

- Ministerio de Educación y Formación Profesional, *Real Decreto 217/2022, de 29 de marzo*, por el que se establece la ordenación y las enseñanzas mínimas de la ESO (saberes básicos de Matemáticas: sentido numérico). *Citado de memoria; no verificado en línea en esta sesión.*

Todas las afirmaciones «verificadas por programa» de este capítulo se pueden reproducir ejecutando:

```
python3 codigo/verificaciones/verificar_cap_1_1.py
```

Registro personal del capítulo 1.1

Copia esta ficha en `PROGRESO.md` cuando termines:

```
Capítulo 1.1 — Conjuntos de números
Fecha de inicio:                Fecha de la prueba de dominio:
Puntuación prueba de dominio:  __/10    ¿Todas las ⋆ correctas? Sí / No
Ejercicios hechos:  A1 A2 A3 A4 A5 A6 | B1 B2 B3 B4 B5 B6 | C1 C2 | P1 P2 PC
Ejercicios fallados y concepto:
Preguntas de comprensión que me costaron:
¿Dominado?  Sí / No (repetir prueba el día: ______)
```

Apéndice A — Glosario

> El glosario **crece con cada capítulo**. Cada término indica el capítulo donde se define. Los términos marcados con **[AVANCE]** se han mencionado, pero se definen con rigor más adelante. Orden alfabético.

Término	Significado	Capítulo
Anteperiodo	En un decimal periódico mixto, cifras decimales que hay entre la coma y el periodo y que no se repiten. En 0,41(6), el anteperiodo es 41.	1.1
Aproximación	Número que se usa en lugar de otro porque está cerca de él, pero no es igual. Se escribe con ≈: π ≈ 3,14.	1.1
Cerrado (conjunto cerrado para una operación)	Un conjunto A es cerrado para una operación si, operando dos elementos cualesquiera de A, el resultado siempre está en A. ℕ es cerrado para la suma, pero no para la resta.	1.1
Conjetura	Afirmación que parece verdadera (por ejemplos o por evidencia computacional) pero que no está demostrada.	1.1
Conjunto	Colección de objetos (elementos) en la que está claro qué objetos pertenecen y cuáles no.	1.1 (formal en 11.1)
Contraejemplo	Caso concreto que demuestra que una afirmación del tipo «para todo…» es falsa. Uno solo basta.	1.1
Cota	Valor máximo (cota superior) o mínimo (cota inferior) que se sabe que una cantidad no puede sobrepasar. No garantiza que se alcance.	1.1
Cuadrado perfecto	Número natural que es el cuadrado de otro natural: 0, 1, 4, 9, 16, 25, …	1.1
Decimal exacto	Expresión decimal con un número finito de cifras: 0,175.	1.1

Término	Significado	Capítulo
Decimal periódico puro / mixto	Decimal infinito en el que un bloque de cifras (periodo) se repite indefinidamente, justo después de la coma (puro: 0,(36)) o tras un anteperiodo (mixto: 0,41(6)).	1.1
Demostración	Razonamiento que prueba que una afirmación es verdadera en **todos** los casos a los que se refiere.	1.1 (formal en Módulo 10)
Densidad (de $\mathbb{Q}$)	Propiedad de que entre dos racionales distintos siempre hay otro racional.	1.1
Elemento	Cada uno de los objetos que forman un conjunto. «x ∈ A» significa que x es elemento de A.	1.1
Entero (número)	…, −2, −1, 0, 1, 2, … Conjunto $\mathbb{Z}$.	1.1
Evidencia computacional	Comprobación por ordenador en un número finito de casos. Apoya una afirmación, pero no la demuestra (salvo que cubra todos los casos posibles).	1.1
`float`	Tipo de número decimal aproximado que usan los ordenadores (coma flotante). No es exacto: en Python `0.1 + 0.2 == 0.3` da `False`.	1.1 (a fondo en 6.2 y 8.7)
Fracción generatriz	Fracción irreducible que da un decimal exacto o periódico dado. La de 0,41(6) es 5/12.	1.1
Fracción irreducible	Fracción cuyo numerador y denominador no tienen divisores comunes mayores que 1.	1.1
Fracciones equivalentes	Fracciones que representan el mismo número: a/b = c/d cuando a·d = b·c.	1.1
GF(7) [AVANCE]	Campo finito de 7 elementos: el conjunto {0, 1, 2, 3, 4, 5, 6} con la suma y el producto «con reloj de 7» (restos de dividir entre 7).	1.1 (definición formal en 13.3)
Irracional (número)	Número real que no se puede escribir como fracción de enteros. Su expresión decimal es infinita y no periódica: √2, π. Conjunto: $\mathbb{I} = \mathbb{R} \setminus \mathbb{Q}$.	1.1
Natural (número)	0, 1, 2, 3, … Conjunto $\mathbb{N}$. En este libro, 0 ∈ $\mathbb{N}$.	1.1

Término	Significado	Capítulo
Opuesto	El opuesto de a es −a, el número que sumado con a da 0.	1.1
Orden (a < b)	a < b significa que b − a es positivo; en la recta, a está a la izquierda de b.	1.1
Parte entera	En una expresión decimal, la parte anterior a la coma. (Las funciones ⌊x⌋ y ⌈x⌉ se definen en 1.4.)	1.1
Periodo	Bloque de cifras que se repite indefinidamente en un decimal periódico.	1.1
Primo (número)	Natural mayor que 1 cuyos únicos divisores son 1 y él mismo.	1.1 (a fondo en 2.2)
Principio del palomar [AVANCE]	Si se reparten más objetos que cajones, algún cajón recibe al menos dos objetos.	1.1 (formal en 11.6)
Racional (número)	Número que se puede escribir como a/b con a, b enteros y b ≠ 0. Conjunto $\mathbb{Q}$.	1.1
Real (número)	Número que se puede escribir con una expresión decimal (finita o infinita). Corresponde a un punto de la recta numérica. Conjunto $\mathbb{R}$.	1.1
Recta numérica (recta real)	Recta en la que cada punto corresponde a un número real y viceversa.	1.1
Reducción al absurdo [AVANCE]	Método de demostración: se supone lo contrario de lo que se quiere probar y se llega a una contradicción.	1.1 (formal en 10.7)
Resto	En la división de a entre b, lo que «sobra»: a = cociente · b + resto, con 0 ≤ resto < b.	1.1 (formal en 2.5)
Subconjunto (A ⊂ B)	A es subconjunto de B si todo elemento de A es también elemento de B.	1.1

Apéndice B — Hoja de fórmulas

> Crece con cada capítulo. Cada fórmula indica sus **condiciones** (cuándo se puede usar) y su **estado** (demostrada, pendiente o definición). Una fórmula usada fuera de sus condiciones es una de las fuentes de error más frecuentes.

Capítulo 1.1 — Conjuntos de números

Código	Fórmula	Condiciones	Estado
F1	$\mathbb{Q} = \{ a/b : a, b \in \mathbb{Z}, b \neq 0 \}$	—	Definición
—	$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$; $\mathbb{I} = \mathbb{R} \setminus \mathbb{Q}$	—	Demostrada (Prop. 1.1)
F2	$a/b = c/d \iff a \cdot d = b \cdot c$	$b \neq 0, d \neq 0$	Propiedad
F3	$a/b \pm c/d = (a \cdot d \pm c \cdot b)/(b \cdot d)$	$b \neq 0, d \neq 0$	Propiedad
F4	$(a/b) \cdot (c/d) = (a \cdot c)/(b \cdot d)$	$b \neq 0, d \neq 0$	Propiedad
F5	$(a/b) : (c/d) = (a \cdot d)/(b \cdot c)$	$b \neq 0, c \neq 0, d \neq 0$	Propiedad
F6	$a/b < c/d \iff a \cdot d < c \cdot b$	**$b > 0, d > 0$**	Propiedad
F7	$a < (a + b)/2 < b$	$a < b$	Demostrada (Prop. 1.3)
F8	Decimal exacto con m cifras decimales = N/10^m (N: el número sin la coma)	—	Demostrada (Prop. 1.6)
F9	Generatriz de un periódico = (número sin coma hasta el final del primer periodo − número sin coma hasta el final del anteperiodo) / (tantos 9 como cifras del periodo, seguidos de tantos 0 como cifras del anteperiodo)	decimal periódico	Demostrada (Prop. 1.6)
F10	Longitud del periodo de $a/b \leq b - 1$	$b > 0$ entero	Demostrada (Prop. 1.4)
—	Racional $\iff$ decimal exacto o periódico	—	Demostrada (Props. 1.4 y 1.6)

Código	Fórmula	Condiciones	Estado
—	Criterio del denominador (fracción **irreducible** a/b): solo 2 y 5 → exacto; ni 2 ni 5 → periódico puro; mezcla → periódico mixto	fracción irreducible, b > 1	Exacto: demostrada. Resto: pendiente (2.3, 14.6)
—	$\sqrt{2} \notin \mathbb{Q}$	—	Demostrada (Teorema 1.1)
—	$\sqrt{n} \in \mathbb{N}$ o $\sqrt{n} \notin \mathbb{Q}$	$n \in \mathbb{N}$	Pendiente (10.7)
—	$0,(9) = 1$	—	Demostrada (Obs. 1.1)
—	[AVANCE] $a \oplus b$ = resto de $(a + b)$ entre 7; $a \otimes b$ = resto de $(a \cdot b)$ entre 7	$a, b \in \{0, \ldots, 6\}$	Definición (formal en 2.8 y 13.3)

Apéndice C — Errores frecuentes y cómo corregirlos

> Lista acumulada de todos los capítulos. Para cada error: qué se hace mal, por qué está mal y cómo detectarlo. Úsala como lista de comprobación antes de entregar un ejercicio o un resultado. Los errores **personales** (los tuyos) se anotan aparte, en `PROGRESO.md`.

Capítulo 1.1 — Conjuntos de números

Código	Error	Por qué está mal	Cómo detectarlo / corregirlo
1.1-E1	Clasificar $-12/4$ como «no entero» por estar escrito como fracción.	Lo que decide es el valor: $-12/4 = -3$.	Simplifica siempre antes de clasificar.
1.1-E2	«Infinitas cifras decimales $\Longrightarrow$ irracional.»	$1/3 = 0{,}333\ldots$ es racional.	¿Hay un bloque que se repite? Entonces es racional.
1.1-E3	$\pi = 3{,}14$ o $\pi = 22/7$.	Son aproximaciones racionales de un irracional.	Usa $\approx$.
1.1-E4	$0{,}(9) < 1$.	$0{,}(9) = 1$.	Resta $10x - x$.
1.1-E5	«Toda raíz cuadrada es irracional.»	$\sqrt{9} = 3$.	Comprueba si es un cuadrado perfecto.
1.1-E6	Escribir $2 \subset \mathbb{N}$.	$\subset$ relaciona conjuntos; $\in$ relaciona elemento y conjunto.	$2 \in \mathbb{N}$, $\{2\} \subset \mathbb{N}$.
1.1-E7	$-7 > -3$.	Entre negativos, el más alejado de 0 es el menor.	Calcula la diferencia: $-3 - (-7) = 4 > 0$, luego $-7 < -3$.
1.1-E8	Generatriz de un periódico mixto como si fuera puro $(0{,}41(6) = 416/999)$.	Se ignora el anteperiodo.	Comprueba dividiendo la fracción obtenida.

Código	Error	Por qué está mal	Cómo detectarlo / corregirlo
1.1-E9	Aplicar el criterio del denominador sin simplificar (21/56).	El criterio exige fracción irreducible.	Simplifica primero.
1.1-E10	Tomar ejemplos como demostración.	Los casos son infinitos.	Etiqueta como conjetura hasta tener una demostración.
1.1-E11	Dividir entre 0 o creer que 0/5 no existe.	a/0 no está definido; 0/5 = 0.	¿Qué número multiplicado por el divisor da el dividendo?
1.1-E12	Escribir `0,75` en Python.	Python usa punto decimal.	En el código: `0.75`.
1.1-E13	Comparar `float` con `==`.	Los `float` son aproximaciones.	En GF(7), solo enteros; para fracciones exactas, `Fraction`.
1.1-E14	Confundir una cota con el valor exacto.	«Como mucho b − 1» no significa «exactamente b − 1».	Distingue ≤ de =.
1.1-E15	Suponer que un programa da el mismo resultado en cualquier versión.	`sum([0.1]*10)` da `1.0` en Python ≥ 3.12 y `0.9999999999999999` en versiones anteriores.	Anota siempre la versión con la que obtienes un resultado.

Apéndice D — Bibliografía comentada

> **Regla de honestidad:** cada referencia indica su **estado de verificación**. No se ha inventado ninguna referencia, pero no todas se han podido comprobar en línea al redactar el libro. Antes de citar una fuente en un trabajo propio, **ábrela y compruébala tú mismo** (autores, título, año, páginas). Aprender a hacerlo forma parte del Módulo 28.

Leyenda de estado

Estado	Significado
🟢 **Verificada (búsqueda web, 12/10/2026)**	Se ha confirmado por búsqueda web que existe y los datos indicados coinciden. Lo que no se pudo confirmar se señala aparte.
🟡 **Clásica, citada de memoria**	Obra de referencia muy conocida en su campo; los datos se han escrito de memoria y **no** se han comprobado en línea en esta sesión.
🔴 **Enlace no comprobado**	El enlace no se pudo abrir desde el entorno de redacción.

---

1. Teoría de códigos (libros de texto)

Ordenados de más accesible a más avanzado. Los usarás a partir de la Parte V.

1. 🟡 **R. Hill**, *A First Course in Coding Theory*, Oxford University Press (Clarendon Press), 1986. Probablemente el texto más amable para empezar; trata códigos lineales sobre GF(q), matrices generadoras y de control, y cotas. **Recomendado como primer libro de códigos.**
2. 🟡 **S. Ling y C. Xing**, *Coding Theory: A First Course*, Cambridge University Press, 2004. Muy claro; incluye campos finitos, códigos cíclicos, BCH y Reed–Solomon con muchos ejercicios.
3. 🟡 **W. C. Huffman y V. Pless**, *Fundamentals of Error-Correcting Codes*, Cambridge University Press, 2003. Referencia estándar de nivel universitario: cotas, códigos cíclicos, códigos sobre GF(q), enumeradores de pesos. Lo usarás en los Módulos 21, 22 y 27.

4. 🟡 **J. H. van Lint**, *Introduction to Coding Theory*, Springer (Graduate Texts in Mathematics 86), 3.ª ed., 1999. Conciso y riguroso; para cuando domines las demostraciones (Módulo 10 en adelante).
5. 🟡 **F. J. MacWilliams y N. J. A. Sloane**, *The Theory of Error-Correcting Codes*, North-Holland, 1977. La «enciclopedia» clásica de la teoría de códigos. Para consultar, no para aprender desde cero.
6. 🟡 **A. E. Brouwer**, «Bounds on the size of linear codes», en V. S. Pless y W. C. Huffman (eds.), *Handbook of Coding Theory*, Elsevier, 1998. Origen de las tablas de mejores códigos lineales conocidos que después continuó M. Grassl (ver §3).

2. Campos finitos, álgebra y matemáticas discretas

7. 🟡 **R. Lidl y H. Niederreiter**, *Introduction to Finite Fields and Their Applications*, Cambridge University Press, ed. revisada, 1994. Para el Módulo 16. (Existe una obra más extensa de los mismos autores, *Finite Fields*, en la colección *Encyclopedia of Mathematics and its Applications*, vol. 20, CUP.)
8. 🟡 **G. H. Hardy y E. M. Wright**, *An Introduction to the Theory of Numbers*, Oxford University Press, 6.ª ed. (revisada por D. R. Heath-Brown y J. H. Silverman), 2008. Teoría de números clásica: irracionalidad, decimales periódicos, congruencias, Fermat. Capítulos 1.1, Módulos 2 y 14.
9. 🟡 **G. Strang**, *Introduction to Linear Algebra*, Wellesley-Cambridge Press (varias ediciones). Álgebra lineal con mucha intuición, para el Módulo 12 (trabaja sobre los reales; la versión sobre campos finitos la verás en el Módulo 13).
10. 🟡 **K. H. Rosen**, *Discrete Mathematics and Its Applications*, McGraw-Hill (varias ediciones). Lógica, conjuntos, combinatoria, inducción, aritmética modular. Módulos 5, 10, 11 y 14.
11. 🟡 **D. J. Velleman**, *How to Prove It: A Structured Approach*, Cambridge University Press (3.ª ed., 2019). El mejor libro para aprender a leer y escribir demostraciones. Módulo 10.
12. 🟡 **G. Pólya**, *How to Solve It*, Princeton University Press, 1945. Traducción española: *Cómo plantear y resolver problemas*, Editorial Trillas. Método general de resolución de problemas. Capítulo 3.11 y Módulo 30.

3. Tablas de códigos, bases de datos y artículos relacionados con GF(7)

13. 🟢 **M. Grassl**, *Bounds on the minimum distance of linear codes and quantum codes* (tablas en línea), http://www.codetables.de **Verificado:** búsquedas web confirman que las tablas existen, que las mantiene Markus Grassl y que continúan las tablas de A. E. Brouwer; la documentación de SageMath indica que su función de consulta a esta base admite los cuerpos de 2, 3, 4, 5, **7**, 8 y 9 elementos. 🔴 La página no se pudo abrir directamente desde el entorno de redacción. **Es la referencia principal para saber si un código $[n, k, d]_7$ mejora lo conocido** (Módulos 21.8, 28.8).
14. 🟢 **«Some results on the structure of constacyclic codes and new linear codes over GF(7) from quasi-twisted codes»**, *Advances in Mathematics of Communications*, 11(1), 2017, pp. 245–258. DOI: 10.3934/amc.2017016. Página del editor: https://www.aimsciences.org/article/doi/10.3934/amc.2017016 **Verificado:** revista, volumen, páginas y DOI; según el resumen, los autores encuentran 41 códigos cuasitwisted sobre GF(7) con mejores parámetros que los mejores códigos lineales conocidos entonces, y obtienen otros mediante la Construcción X, acortamiento y pinchado. 🟡 **Autores no confirmados en la búsqueda**; según mi conocimiento son N. Aydin, N. Connolly y M. Grassl: compruébalo en la página del editor. **Directamente relacionado con tu investigación** (Módulos 22.9, 27, 29).
15. 🟢 **E. Z. Chen y N. Aydin**, «A database of linear codes over F_{13} with minimum distance bounds and new quasi-twisted codes from a heuristic search algorithm», *Journal of Algebra Combinatorics Discrete Structures and Applications*, 2015. **Verificado:** título y revista. Ejemplo de cómo se construye una base de datos de códigos con búsquedas heurísticas (Módulo 26). Trata otro cuerpo (13 elementos), pero el método es el mismo.

4. Complejidad del cálculo de la distancia mínima

16. 🟢/🟡 **A. Vardy**, «The intractability of computing the minimum distance of a code», *IEEE Transactions on Information Theory*, 1997. **Verificado:** existencia y resultado (calcular la distancia mínima de un código lineal es un problema NP-difícil, como habían conjeturado Berlekamp, McEliece y van Tilborg en 1978), por las citas de otros artículos. 🟡 Volumen y páginas no confirmados (de memoria: vol. 43, n.º 6, pp. 1757–1766). Capítulo 19.8.
17. 🟢 **I. Dumer, D. Micciancio y M. Sudan**, «Hardness of approximating the minimum distance of a linear code», *IEEE Transactions on Information Theory*, 49(1), 2003, pp. 22–37. **Verificado.** Extiende el resultado de Vardy: ni siquiera

aproximar la distancia mínima es fácil (bajo hipótesis de complejidad). Capítulo 19.8 (ampliación).

5. Programación y cálculo numérico

18. 🔴 **Documentación oficial de Python**: https://docs.python.org/3/ — en particular, el tutorial «Floating-Point Arithmetic: Issues and Limitations», https://docs.python.org/3/tutorial/floatingpoint.html El enlace no se pudo abrir desde el entorno de redacción; es la documentación oficial y debería estar disponible. Módulos 6–9.
19. 🟡 **D. Goldberg**, «What Every Computer Scientist Should Know About Floating-Point Arithmetic», *ACM Computing Surveys*, 23(1), 1991, pp. 5–48. Por qué los `float` no son exactos. Capítulos 1.1 y 8.7 (lectura avanzada).
20. 🟡 **T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein**, *Introduction to Algorithms*, MIT Press (4.ª ed., 2022). Referencia de algoritmos y complejidad. Módulo 8 (ampliación).

6. Software de álgebra que se usará para verificación independiente (Módulo 28)

21. 🟡 **SageMath** (software libre de matemáticas), https://www.sagemath.org — incluye funciones de teoría de códigos y una consulta a las tablas de codetables.de (ver n.º 13).
22. 🟡 **Magma** (sistema comercial de álgebra computacional, Universidad de Sídney). Es el programa que se usa habitualmente para calcular las tablas de mejores códigos conocidos. Referencia: W. Bosma, J. Cannon y C. Playoust, «The Magma algebra system I: The user language», *Journal of Symbolic Computation*, 24, 1997.

7. Currículo escolar

23. 🟡 **Real Decreto 217/2022, de 29 de marzo**, por el que se establece la ordenación y las enseñanzas mínimas de la Educación Secundaria Obligatoria (BOE). Marco de los saberes básicos de Matemáticas de ESO. La concreción para Andalucía la establece la normativa autonómica correspondiente (no consultada en esta sesión).

Fuentes por capítulo

Capítulo	Fuentes
1.1	Hardy y Wright (8); Goldberg (19); documentación de Python (18); Real Decreto 217/2022 (23). Todos los cálculos: verificados con `codigo/verificaciones/verificar_cap_1_1.py`.

Registro de verificaciones de fuentes

Fecha	Fuente	Método	Resultado
12/10/2026	codetables.de (13)	Búsqueda web + intento de acceso directo	Existencia y mantenimiento confirmados por fuentes secundarias; acceso directo fallido (DNS).
12/10/2026	Artículo GF(7) cuasitwisted (14)	Búsqueda web	Revista, volumen, páginas, DOI y resumen confirmados; autores no confirmados.
12/10/2026	Chen y Aydin, F_{13} (15)	Búsqueda web	Título y revista confirmados.
12/10/2026	Vardy 1997 (16)	Búsqueda web	Existencia y resultado confirmados; volumen y páginas no.
12/10/2026	Dumer–Micciancio–Sudan (17)	Búsqueda web	Confirmado.
12/10/2026	docs.python.org (18)	Acceso directo	Fallido (DNS) desde el entorno de redacción.

Registro de progreso

> Este archivo tiene **dos registros separados**, porque escribir un capítulo y dominarlo son cosas distintas:
>
> - **Registro de redacción:** qué partes del libro están escritas (lo actualiza quien redacta).
> - **Registro de aprendizaje:** qué capítulos has **dominado tú**, con tus puntuaciones y errores (lo actualizas tú, o tu tutor con tus resultados).
>
> Última actualización: **12/10/2026 — Entrega 1**.

---

1. Índice general (resumen)

El índice completo, con enlaces, está en `libro/00-indice-general.md`.

Parte	Módulos	Capítulos	Redactados
I. Fundamentos de ESO	1–5	46	1 (1.1)
II. Programación	6–9	38	0
III. Discretas y álgebra lineal	10–13	45	0
IV. Números y polinomios	14–16	25	0
V. Teoría de códigos	17–23	62	0
VI. Buscador de códigos	24–28	38	0
VII. Investigación autónoma	29–32	32	0
Total	32	**286**	**1**

---

2. Registro de redacción

Campo	Estado
Último capítulo redactado	1.1 — Los conjuntos de números: $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$ y $\mathbb{R}$
Próximo capítulo a redactar	1.2 — Números enteros: signos, valor absoluto, orden y jerarquía de operaciones
Documentos de apoyo creados	Índice general · Mapa de prerrequisitos · Cómo estudiar este libro · Glosario · Hoja de fórmulas · Errores frecuentes · Bibliografía · Este registro
Código creado	`codigo/verificaciones/verificar_cap_1_1.py` · `codigo/ejemplos/cap_1_1_rincon_del_ordenador.py`
Versiones descargables	`descargas/libro-gf7-entrega-1.pdf`, `.html` y `-fuentes.zip`, generadas con `herramientas/construir_libro.py` (13/10/2026).
Verificaciones ejecutadas	`verificar_cap_1_1.py` con Python 3.13.16 el 12/10/2026: **todas las comprobaciones superadas** (incluida la comprobación del criterio del denominador en 124 750 fracciones).

Entregas

Entrega	Fecha	Contenido
1	12/10/2026	Índice general completo, mapa de prerrequisitos, sistema de estudio, apéndices iniciales, **capítulo 1.1 completo** con verificación por programa.
1 (descargas)	13/10/2026	Primeras versiones descargables (PDF, HTML, ZIP) con todo lo redactado en la Entrega 1.
2 (prevista)	—	Capítulos 1.2 y 1.3 (enteros; fracciones).
3 (prevista)	—	Capítulos 1.4 y 1.5 (decimales y partes enteras; potencias).
4 (prevista)	—	Capítulos 1.6–1.8 y Repaso R1.
Siguientes	—	Se seguirá el **orden de estudio recomendado** del mapa de prerrequisitos (Etapa 1: M1 → M2, con M6 en paralelo desde 2.5).

---

3. Conceptos ya explicados en el libro

Concepto	Capítulo	Nivel de tratamiento
Conjunto, elemento, $\in$, $\notin$, subconjunto $\subset$	1.1	Intuitivo (formal en 11.1)
$\mathbb{N}$ (con 0), $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$, irracionales $\mathbb{I}$	1.1	Completo para ESO; construcción rigurosa de $\mathbb{R}$ solo mencionada
Opuesto	1.1	Completo
Fracciones equivalentes, fracción irreducible	1.1	Presentado (operaciones a fondo en 1.3)
Decimales exactos, periódicos puros y mixtos; periodo, anteperiodo	1.1	Completo
Fracción generatriz (método de restar y regla clásica)	1.1	Completo
Criterio del denominador (2 y 5)	1.1	Enunciado; demostrada la parte «exacto»; resto pendiente (2.3, 14.6)
Cota del periodo ($\leq b-1$) y argumento de los restos	1.1	Completo, con demostración
Orden en $\mathbb{R}$; comparación de fracciones	1.1	Completo
Densidad de $\mathbb{Q}$ (punto medio)	1.1	Completo, con demostración
Operación cerrada; contraejemplo	1.1	Completo
Ejemplo, evidencia computacional y demostración	1.1	Introducido (a fondo en 10.10)
Irracionalidad de $\sqrt{2}$ (reducción al absurdo)	1.1	Demostración en ampliación (método formal en 10.7)
$0{,}(9) = 1$	1.1	Completo
`float` frente a aritmética exacta; versiones de Python	1.1	Introducido (a fondo en 6.2, 8.7, 9.5)
[AVANCE] «Reloj de 7» (GF(7)): $\oplus$, $\otimes$, parejas (inversos)	1.1	Solo avance (formal en 2.8, 13.1–13.4)
[AVANCE] Principio del palomar	1.1	Solo idea (formal en 11.6)

---

4. Decisiones de organización del libro

N.º	Decisión	Motivo
D1	Formato Markdown en un repositorio Git, un archivo por capítulo.	Versionable, legible en GitHub, fácil de ampliar por entregas.
D2	Notación con símbolos Unicode, no LaTeX.	Se lee en cualquier visor, también fuera de GitHub.
D3	$0 \in \mathbb{N}$.	Convenio habitual en informática y en teoría de códigos (peso 0, índice 0). Se avisa de que otros libros usan otro convenio.
D4	Coma decimal en el texto y punto en el código; periodo entre paréntesis: 0,41(6).	Convenio español + compatible con teclado y programas.
D5	Etiquetas obligatorias: DEFINICIÓN, PROPIEDAD (demostrada / pendiente), HECHO ACEPTADO, CONJETURA, VERIFICADO POR PROGRAMA, EJEMPLO, AVANCE.	Requisito del proyecto: no presentar nunca una conjetura como teorema.
D6	Soluciones plegadas (`<details>`) dentro de §14 y respuestas de la prueba de dominio en un anexo al final del capítulo.	Respetar el orden obligatorio de las 17 secciones y, a la vez, evitar ver las soluciones por accidente.
D7	Cada capítulo con cálculos tiene un programa de verificación independiente; se indica versión de Python y fecha de ejecución.	Comprobar cada resultado por un segundo procedimiento y no afirmar nunca salidas no ejecutadas.
D8	Solo biblioteca estándar de Python.	Evitar dependencias externas innecesarias.
D9	Se añaden 29 temas marcados con [+] (28 capítulos o apartados y un apéndice) que faltaban en el temario (ver mapa de prerrequisitos, §4).	Ningún concepto debe usarse antes de haberse enseñado.

N.º	Decisión	Motivo
D10	La numeración de módulos respeta el temario original; el **orden de estudio** recomendado intercala módulos (etapas 1–7).	Mantener la correspondencia con el temario pedido y, a la vez, llegar antes a la práctica con códigos sobre GF(7).
D11	Criterio de dominio: ≥ 8/10 y todas las preguntas ★; «prueba de salto» con ≥ 9/10.	Progresar por comprensión demostrada, no por capítulos leídos; no hacer perder tiempo con lo ya sabido.
D12	Marcas ★ (núcleo) y ◇ (ampliación) en el índice.	Indicar qué es imprescindible para el proyecto y qué es extensión.
D13	Cada referencia bibliográfica lleva su estado de verificación.	No inventar ni dar por comprobadas fuentes que no se han comprobado.
D14	Versiones descargables (PDF, HTML de un solo archivo y ZIP) generadas automáticamente desde el Markdown con `herramientas/construir_libro.py` en cada entrega.	Poder estudiar sin conexión y en papel sin mantener copias a mano que se desincronicen.
D15	En el Markdown, las líneas que deben verse separadas (apartados a), b)…, enunciado y datos) terminan en «\» (salto de línea forzado), y los recuadros de definición separan título, fórmula y explicación en párrafos.	Que se lean igual de bien en GitHub, en el HTML y en el PDF.

---

5. Registro de aprendizaje (lo rellenas tú)

Último capítulo dominado: — (ninguno todavía)
Capítulo que estoy estudiando: 1.1
Próximo capítulo a estudiar: 1.1 → después 1.2

5.1. Estado por capítulo

Capítulo	Empezado	Prueba de dominio (fecha y nota)	¿Todas las ★?	¿Dominado?	Repasar el día
1.1					

5.2. Ejercicios realizados

Capítulo	Ejercicios hechos	Acertados	Fallados
1.1			

5.3. Errores que debo repasar

Fecha	Capítulo / ejercicio	Qué hice	Concepto que fallaba	Ejercicio de refuerzo hecho	¿Superado?

5.4. Módulos completados y pendientes de repaso

Módulo	Estado	Repaso acumulativo (R*n*)	Nota
1	En curso	Pendiente	

Código del libro

Todos los programas usan **solo la biblioteca estándar de Python 3**. No hay que instalar nada más.

Carpetas

Carpeta	Contenido
`ejemplos/`	Programas cortos que aparecen dentro de los capítulos (sección «Rincón del ordenador» o ejemplos de código).
`verificaciones/`	Un programa por capítulo que **recalcula por un procedimiento independiente** todos los resultados numéricos del capítulo (ejemplos, ejercicios, soluciones, prueba de dominio).

A partir del Módulo 13 aparecerá una carpeta con la **biblioteca del proyecto** (operaciones sobre GF(7), matrices, rango, distancia mínima) y sus pruebas unitarias.

Cómo ejecutar

Desde la carpeta raíz del repositorio:

```
python3 codigo/ejemplos/cap_1_1_rincon_del_ordenador.py
python3 codigo/verificaciones/verificar_cap_1_1.py
```

Si un programa de verificación termina con `TODAS LAS COMPROBACIONES SUPERADAS`, los cálculos del capítulo coinciden con el segundo procedimiento. Si alguno falla, el programa se detiene e indica qué cálculo no coincide.

Registro de ejecuciones

Fecha	Programa	Versión de Python	Resultado
12/10/2026	`verificaciones/verificar_cap_1_1.py`	3.13.16	Todas las comprobaciones superadas
12/10/2026	`ejemplos/cap_1_1_rincon_del_ordenador.py`	3.13.16	Salida idéntica a la mostrada en el capítulo 1.1

Limitaciones conocidas

- Algunas salidas con números decimales (`float`) pueden variar entre versiones de Python. Ejemplo documentado en el capítulo 1.1: `sum([0.1] * 10)` da `1.0` desde Python 3.12 y `0.9999999999999999` en versiones anteriores. Los programas de verificación tienen esto en cuenta.
- Los programas de verificación comprueban los **cálculos** del libro; no sustituyen a las demostraciones. Cuando una comprobación cubre solo un número finito de casos de una afirmación general, el libro lo etiqueta como **[VERIFICADO POR PROGRAMA]** (evidencia, no demostración).

Apéndice de código: programas del libro

Copia íntegra de los programas incluidos en el repositorio, para poder leerlos sin conexión. Para ejecutarlos, usa los archivos `.py` del ZIP (no copies desde el PDF: los saltos de línea y espacios podrían alterarse).

codigo/ejemplos/cap_1_1_rincon_del_ordenador.py

```
# Rincón del ordenador — Capítulo 1.1
# Objetivo: ver con tus propios ojos que los decimales del ordenador (float)
# no son exactos, que las fracciones exactas sí lo son, y probar la
# "suma y el producto con reloj de 7".

print(0.1 + 0.2)              # suma con decimales "de ordenador" (float)
print(0.1 + 0.2 == 0.3)       # ¿es exactamente igual a 0.3?

from fractions import Fraction
print(Fraction(1, 10) + Fraction(2, 10))              # suma EXACTA de fracciones
print(Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10))

print((4 + 5) % 7)            # suma "con reloj de 7": resto de dividir 9 entre 7
print((3 * 5) % 7)            # producto "con reloj de 7": resto de dividir 15 entre 7
```

codigo/verificaciones/verificar_cap_1_1.py

```
#!/usr/bin/env python3
"""
verificar_cap_1_1.py — Comprobación independiente de los cálculos del
Capítulo 1.1 («Los conjuntos de números: N, Z, Q y R»).

¿PARA QUÉ SIRVE?
    Todos los resultados numéricos del capítulo se han calculado a mano.
    Este programa los vuelve a calcular por un procedimiento distinto
    (el ordenador, con aritmética EXACTA de fracciones) para detectar
    posibles errores. Es la misma filosofía que usarás en tu investigación:
    un resultado importante se comprueba siempre por un segundo camino.

¿CÓMO SE USA?
    python3 verificar_cap_1_1.py

    Si todo es correcto, cada bloque imprime «OK» y al final aparece
    «TODAS LAS COMPROBACIONES SUPERADAS». Si algo no coincide, el programa
    se detiene con un AssertionError que dice qué cálculo ha fallado.

REQUISITOS
    Solo la biblioteca estándar de Python 3 (no hay que instalar nada).

NOTA PARA EL ESTUDIANTE
    Todavía no necesitas entender este código. Podrás leerlo línea a línea
```

```
    cuando termines los Módulos 6 y 7. Por ahora basta con ejecutarlo y ver
    que todas las comprobaciones salen «OK».
"""

from fractions import Fraction as F
from math import isqrt


# ---------------------------------------------------------------------------
# 1. HERRAMIENTAS
# ---------------------------------------------------------------------------

def desarrollo(fraccion, base=10):
    """Desarrollo de una fracción en la base indicada mediante la DIVISIÓN
    LARGA, exactamente como se hace a mano.

    Devuelve (signo, parte_entera, anteperiodo, periodo), donde anteperiodo
    y periodo son cadenas de cifras ('' si no hay).

    Idea clave (Propiedad 1.4 del capítulo): al dividir entre b solo hay b
    restos posibles (0, 1, ..., b-1). En cuanto un resto se repite, las
    cifras empiezan a repetirse: ahí empieza el periodo.
    """
    a, b = fraccion.numerator, fraccion.denominator   # Fraction garantiza b > 0
    signo = "-" if a < 0 else ""
    a = abs(a)
    parte_entera, resto = divmod(a, b)
    cifras = []
    posicion_del_resto = {}           # resto -> posición de la cifra que genera
    while resto != 0 and resto not in posicion_del_resto:
        posicion_del_resto[resto] = len(cifras)
        cifra, resto = divmod(resto * base, b)
        cifras.append(str(cifra))
    if resto == 0:                    # la división termina: decimal exacto
        return signo, parte_entera, "".join(cifras), ""
    inicio = posicion_del_resto[resto]
    return signo, parte_entera, "".join(cifras[:inicio]), "".join(cifras[inicio:])


def restos_division_larga(a, b):
    """Lista de restos que aparecen al dividir a entre b (0 < a < b), empezando
    por el propio a, hasta que un resto se repite o sale 0."""
    restos = []
    r = a
    while r != 0 and r not in restos:
        restos.append(r)
        r = (r * 10) % b
    return restos


def a_texto(fraccion):
    """Escribe el decimal con coma española y el periodo entre paréntesis.
    Ejemplos: 0,41(6)   2,(18)   0,175   -3"""
    signo, entera, ante, per = desarrollo(fraccion)
    if not ante and not per:
        return f"{signo}{entera}"
    texto = f"{signo}{entera}," + ante
    if per:
        texto += f"({per})"
    return texto


def tipo(fraccion):
    """'entero', 'exacto', 'periódico puro' o 'periódico mixto'."""
    _, _, ante, per = desarrollo(fraccion)
    if not per:
        return "exacto" if ante else "entero"
    return "periódico mixto" if ante else "periódico puro"
```

```python
def tipo_predicho(fraccion):
    """Predicción SIN dividir, usando solo el denominador de la fracción
    irreducible (Propiedad 1.5). Es un procedimiento distinto de tipo()."""
    b = fraccion.denominator          # Fraction ya está simplificada
    if b == 1:
        return "entero"
    resto_b = b
    tiene_2_o_5 = False
    for p in (2, 5):
        while resto_b % p == 0:
            resto_b //= p
            tiene_2_o_5 = True
    if resto_b == 1:
        return "exacto"
    return "periódico mixto" if tiene_2_o_5 else "periódico puro"


def generatriz(parte_entera, anteperiodo, periodo):
    """Fracción generatriz de  parte_entera,anteperiodo(periodo)  (número >= 0).

    Regla general (se deduce en el capítulo restando 10^k·x - 10^j·x):
        x = E + (AP - A) / (10^len(A) · (10^len(P) - 1))
    donde AP es el número formado por anteperiodo y periodo seguidos.
    Es un procedimiento INDEPENDIENTE de la división larga de desarrollo().
    """
    E = F(parte_entera)
    A = int(anteperiodo) if anteperiodo else 0
    if not periodo:                                  # decimal exacto
        return E + F(A, 10 ** len(anteperiodo))
    AP = int(anteperiodo + periodo)
    return E + F(AP - A, 10 ** len(anteperiodo) * (10 ** len(periodo) - 1))


def es_cuadrado_perfecto(n):
    return n >= 0 and isqrt(n) ** 2 == n


def ok(mensaje):
    print(f"OK  {mensaje}")


# ---------------------------------------------------------------------------
# 2. COMPROBACIONES DEL CAPÍTULO
# ---------------------------------------------------------------------------

def comprobar_ejemplo_1_1():
    # Números de la lista que son racionales (se representan como fracciones exactas)
    assert F(-12, 4) == -3
    assert F(75, 100) == F(3, 4)
    assert generatriz(0, "", "6") == F(2, 3)
    assert isqrt(9) == 3 and es_cuadrado_perfecto(9)
    assert not es_cuadrado_perfecto(5)          # sqrt(5) irracional (Prop. 1.7)
    assert isqrt(16) == 4
    assert F(2, 5) == F(4, 10)                  # 2/5 = 0,4: racional no entero
    ok("Ejemplo 1.1 (clasificación)")


def comprobar_ejemplo_1_2():
    casos = {F(7, 40): ("0,175", "exacto"),
             F(5, 12): ("0,41(6)", "periódico mixto"),
             F(3, 7): ("0,(428571)", "periódico puro")}
    for fr, (texto, t) in casos.items():
        assert a_texto(fr) == texto, (fr, a_texto(fr))
        assert tipo(fr) == t == tipo_predicho(fr), fr
    # Restos de la división 3 : 7 (aparecen todos los restos no nulos 1..6)
    assert restos_division_larga(3, 7) == [3, 2, 6, 4, 5, 1]
    # Fracciones generatrices calculadas a mano
    assert generatriz(0, "41", "6") == F(375, 900) == F(5, 12)
```

```
    assert generatriz(0, "175", "") == F(7, 40)
    assert generatriz(0, "", "428571") == F(3, 7)
    assert F(7, 40) == F(175, 1000)            # 40·25 = 1000
    ok("Ejemplo 1.2 (fracción <-> decimal)")


def comprobar_ejemplo_1_3():
    # Coma flotante: el ordenador NO es exacto con 0,1 y 0,2
    assert 0.1 + 0.2 != 0.3
    assert repr(0.1 + 0.2) == "0.30000000000000004"
    assert F(1, 10) + F(2, 10) == F(3, 10)       # con fracciones exactas sí
    # Sumar 0.1 diez veces, una a una, con un bucle:
    total = 0.0
    for _ in range(10):
        total += 0.1
    assert total != 1.0 and repr(total) == "0.9999999999999999"
    # ¡Ojo con las versiones! Desde Python 3.12, la función sum() usa una suma
    # "compensada" más precisa y en este caso concreto devuelve 1.0.
    # (Comprobado al redactar el libro con Python 3.13.)
    import sys
    if sys.version_info >= (3, 12):
        assert sum([0.1] * 10) == 1.0
    # 1/10 en base 2 es periódico: 0,0(0011)
    assert desarrollo(F(1, 10), base=2) == ("", 0, "0", "0011")
    # Tablas "con reloj de 7" y "con reloj de 6"
    assert (4 + 5) % 7 == 2 and (6 + 6) % 7 == 5 and (3 * 5) % 7 == 1
    tabla7 = [[(a * b) % 7 for b in range(7)] for a in range(7)]
    assert tabla7[2] == [0, 2, 4, 6, 1, 3, 5]
    assert tabla7[3] == [0, 3, 6, 2, 5, 1, 4]
    assert tabla7[4] == [0, 4, 1, 5, 2, 6, 3]
    assert tabla7[5] == [0, 5, 3, 1, 6, 4, 2]
    assert tabla7[6] == [0, 6, 5, 4, 3, 2, 1]
    for a in range(1, 7):                        # cada fila no nula es una
        assert sorted(tabla7[a][1:]) == [1, 2, 3, 4, 5, 6]   # reordenación de 1..6
    inversos7 = {a: b for a in range(1, 7) for b in range(1, 7) if a * b % 7 == 1}
    assert inversos7 == {1: 1, 2: 4, 3: 5, 4: 2, 5: 3, 6: 6}
    tabla6 = [[(a * b) % 6 for b in range(6)] for a in range(6)]
    assert tabla6[2] == [0, 2, 4, 0, 2, 4]
    assert tabla6[3] == [0, 3, 0, 3, 0, 3]
    assert (2 * 3) % 6 == 0
    ok("Ejemplo 1.3 (exactitud, tablas módulo 7 y módulo 6)")


def comprobar_ejercicios_nivel_A():
    # A1
    assert F(15, 3) == 5 and F(125, 1000) == F(1, 8)
    assert generatriz(0, "", "12") == F(4, 33)
    assert es_cuadrado_perfecto(25) and not es_cuadrado_perfecto(10)
    assert F(-6, 3) == -2                        # -sqrt(36)/3
    assert F(3141592, 1000000) == F(392699, 125000)
    # A3
    esperado = {F(3, 8): "0,375", F(7, 9): "0,(7)", F(13, 6): "2,1(6)",
                F(9, 25): "0,36", F(4, 11): "0,(36)"}
    for fr, texto in esperado.items():
        assert a_texto(fr) == texto, (fr, a_texto(fr))
        assert tipo(fr) == tipo_predicho(fr)
    # A4
    assert generatriz(0, "45", "") == F(9, 20)
    assert generatriz(0, "", "5") == F(5, 9)
    assert generatriz(2, "", "18") == F(24, 11)
    assert generatriz(0, "3", "12") == F(309, 990) == F(103, 330)
    assert generatriz(1, "0", "6") == F(96, 90) == F(16, 15)
    for fr, texto in {F(9, 20): "0,45", F(5, 9): "0,(5)", F(24, 11): "2,(18)",
                      F(103, 330): "0,3(12)", F(16, 15): "1,0(6)"}.items():
        assert a_texto(fr) == texto, (fr, a_texto(fr))
    # A5: -2,5 < -sqrt(4) < -3/2 < 0,(3) < 1,7 < 7/4
    lista = [F(-25, 10), F(-2), F(-3, 2), F(1, 3), F(17, 10), F(7, 4)]
    assert lista == sorted(lista)
    # A6 (contraejemplos)
```

```
    assert 2 - 5 < 0 and F(1, 2).denominator != 1
    ok("Ejercicios de nivel A")


def comprobar_ejercicios_nivel_B():
    # B1
    m = (F(3, 7) + F(4, 9)) / 2
    assert m == F(55, 126) and F(3, 7) < m < F(4, 9)
    assert F(3, 7) == F(216, 504) and F(4, 9) == F(224, 504)
    assert F(218, 504) == F(109, 252) and F(219, 504) == F(73, 168)
    for k in (217, 218, 219):
        assert F(3, 7) < F(k, 504) < F(4, 9)
    # B2
    assert F(21, 56) == F(3, 8) and F(33, 120) == F(11, 40)
    assert F(6, 15) == F(2, 5) and F(35, 42) == F(5, 6)
    for fr, t in {F(21, 56): "exacto", F(33, 120): "exacto", F(9, 14): "periódico mixto",
                  F(6, 15): "exacto", F(35, 42): "periódico mixto"}.items():
        assert tipo(fr) == tipo_predicho(fr) == t, fr
    assert a_texto(F(9, 14)) == "0,6(428571)" and a_texto(F(5, 6)) == "0,8(3)"
    # B3: 0,(9) = 1 ; 0,4(9) = 1/2 ; 2,(9) = 3
    assert generatriz(0, "", "9") == 1
    assert generatriz(0, "4", "9") == F(1, 2)
    assert generatriz(2, "", "9") == 3
    # La división larga nunca produce periodo 9 (comprobado para b <= 300)
    for b in range(2, 301):
        for a in range(1, b):
            periodo = desarrollo(F(a, b))[3]
            assert set(periodo) != {"9"}, (a, b)
    # B4: 1/13 = 0,(076923); restos 1, 10, 9, 12, 3, 4
    assert a_texto(F(1, 13)) == "0,(076923)"
    assert restos_division_larga(1, 13) == [1, 10, 9, 12, 3, 4]
    # B6
    assert F(5, 3).denominator == 3 and F(-3, 2) == F(-15, 10)
    ok("Ejercicios de nivel B")


def comprobar_ejercicios_nivel_C():
    # C1: longitud del periodo de 1/p
    periodos = {}
    for p in (3, 7, 11, 13, 17, 19, 23):
        periodos[p] = len(desarrollo(F(1, p))[3])
        assert (p - 1) % periodos[p] == 0       # el periodo divide a p - 1
    assert periodos == {3: 1, 7: 6, 11: 2, 13: 6, 17: 16, 19: 18, 23: 22}
    assert a_texto(F(1, 17)) == "0,(0588235294117647)"
    assert a_texto(F(1, 19)) == "0,(052631578947368421)"
    assert a_texto(F(1, 23)) == "0,(043478260869565217391 3)"
    # C2: ¿para qué n todo número no nulo tiene "pareja" b con a·b ≡ 1 (mod n)?
    def todos_tienen_pareja(n):
        return all(any(a * b % n == 1 for b in range(1, n)) for a in range(1, n))
    resultado = {n: todos_tienen_pareja(n) for n in range(2, 12)}
    assert [n for n, v in resultado.items() if v] == [2, 3, 5, 7, 11]
    sin_pareja = {n: [a for a in range(1, n)
                      if not any(a * b % n == 1 for b in range(1, n))]
                  for n in (4, 6, 8, 9, 10)}
    assert sin_pareja == {4: [2], 6: [2, 3, 4], 8: [2, 4, 6], 9: [3, 6],
                          10: [2, 4, 5, 6, 8]}
    parejas11 = {a: b for a in range(1, 11) for b in range(1, 11) if a * b % 11 == 1}
    assert parejas11 == {1: 1, 2: 6, 3: 4, 4: 3, 5: 9, 6: 2, 7: 8, 8: 7, 9: 5, 10: 10}
    parejas5 = {a: b for a in range(1, 5) for b in range(1, 5) if a * b % 5 == 1}
    assert parejas5 == {1: 1, 2: 3, 3: 2, 4: 4}
    ok("Ejercicios de nivel C")


def comprobar_problemas():
    # Problema de aplicación 1
    assert 5 - (-3) == 8 and -2 - 4 == -6
    assert F(45, 10) / F(75, 100) == 6
    assert F(-45, 10) + 8 * F(75, 100) == F(3, 2)
    # Problema de aplicación 2 (céntimos)
```

```
    assert 235 + 99 + 1210 + 56 == 1600
    assert F(235, 100) + F(99, 100) + F(1210, 100) + F(56, 100) == 16
    # Problema combinado (calendario). 0 = lunes, 1 = martes, ...
    dias = ["lunes", "martes", "miércoles", "jueves", "viernes", "sábado", "domingo"]
    assert divmod(23, 7) == (3, 2) and dias[(0 + 23) % 7] == "miércoles"
    assert divmod(100, 7) == (14, 2) and dias[(0 + 100) % 7] == "miércoles"
    assert dias[(0 - 10) % 7] == "viernes"       # Python da restos >= 0
    assert a_texto(F(100, 7)) == "14,(285714)" and a_texto(F(2, 7)) == "0,(285714)"
    ok("Problemas de aplicación y problema combinado")


def comprobar_prueba_de_dominio_y_refuerzo():
    # D1
    assert F(-7, 7) == -1 and F(7, 2) == F(isqrt(49), 2)
    assert not es_cuadrado_perfecto(3)
    # D2
    assert generatriz(0, "2", "7") == F(25, 90) == F(5, 18)
    assert a_texto(F(5, 18)) == "0,2(7)"
    # D3
    for fr, t in {F(7, 20): "exacto", F(5, 6): "periódico mixto",
                  F(2, 9): "periódico puro", F(15, 48): "exacto"}.items():
        assert tipo(fr) == tipo_predicho(fr) == t, fr
    assert F(15, 48) == F(5, 16)
    # D5
    assert F(-1, 3) < F(-7, 24) < F(-1, 4)
    # D6: 0,6 < 0,(6) = 2/3 < 0,67
    assert generatriz(0, "", "6") == F(2, 3) and F(6, 10) < F(2, 3) < F(67, 100)
    # D10: el periodo de a/17 tiene como mucho 16 cifras
    assert max(len(desarrollo(F(a, 17))[3]) for a in range(1, 17)) == 16
    # Refuerzo RF1
    assert F(-20, 5) == -4 and es_cuadrado_perfecto(49) and not es_cuadrado_perfecto(50)
    assert F(5, 10) ** 2 == F(25, 100)
    assert generatriz(0, "0", "5") == F(1, 18)
    # RF2
    assert generatriz(0, "1", "6") == F(1, 6)
    assert generatriz(3, "", "7") == F(34, 9)
    assert generatriz(0, "", "142857") == F(1, 7)
    # RF3
    for fr, texto in {F(11, 16): "0,6875", F(7, 15): "0,4(6)", F(5, 33): "0,(15)"}.items():
        assert a_texto(fr) == texto and tipo(fr) == tipo_predicho(fr)
    # RF5: -0,(3) < -0,33 < -0,3 < -1/4
    lista = [F(-1, 3), F(-33, 100), F(-3, 10), F(-1, 4)]
    assert lista == sorted(lista)
    ok("Prueba de dominio y ejercicios de refuerzo")


def comprobar_propiedad_1_5_masivamente():
    """Resultado verificado por programa: la predicción por el denominador
    coincide con la división larga para TODAS las fracciones a/b con
    1 <= a < b <= 500 (evidencia computacional, no demostración)."""
    total = 0
    for b in range(2, 501):
        for a in range(1, b):
            fr = F(a, b)
            assert tipo(fr) == tipo_predicho(fr), fr
            total += 1
    ok(f"Propiedad 1.5 comprobada en {total} fracciones (evidencia, no demostración)")


def comprobar_raiz_de_2_sin_denominador_pequeno():
    """Ejercicio B5: ninguna fracción a/b con 1 <= b <= 1000 cumple (a/b)^2 = 2.
    Esto es EVIDENCIA, no una demostración (hay infinitos denominadores)."""
    for b in range(1, 1001):
        a = isqrt(2 * b * b)          # único candidato posible: a^2 cerca de 2b^2
        assert a * a != 2 * b * b
    ok("Ningún a/b con b <= 1000 tiene cuadrado 2 (evidencia del ejercicio B5)")


if __name__ == "__main__":
```

```
    comprobar_ejemplo_1_1()
    comprobar_ejemplo_1_2()
    comprobar_ejemplo_1_3()
    comprobar_ejercicios_nivel_A()
    comprobar_ejercicios_nivel_B()
    comprobar_ejercicios_nivel_C()
    comprobar_problemas()
    comprobar_prueba_de_dominio_y_refuerzo()
    comprobar_propiedad_1_5_masivamente()
    comprobar_raiz_de_2_sin_denominador_pequeno()
    print("\nTODAS LAS COMPROBACIONES SUPERADAS")
```